BSCS FINAL PROJECT
<Design and Test Specification>

Intelligent Secure Meditation Counselor-Patient Care System with AI Virtual Assistant

Project Advisor

Ms. Misha Asif

Presented by:
Group ID: F25CS186

Student Reg#	Student Name
L1F22BSCS1111	**Faizyab Ahmad**
L1F22BSCS0956	**Sameed Saeed**
L1F22BSCS0976	**Hassan Hayat**

Faculty of Information Technology & Computer Science

University of Central Punjab

Design and Test Specification

SDP Phase III

Intelligent Secure Meditation Counselor-Patient Care System with AI Virtual Assistant

Advisor: Misha Asif

Team F25CS186

Member Name	Primary Responsibility
Faizyab Ahmad	UI/UX Design/ Frontend/ Documentation
Sameed Saeed	Backend / AI / Documentation
Hassan Hayat	Automation / Testing / Documentation

Table of Contents

Previous Phases Feedback

Phase 1 Idea Defense Feedback (Screenshot)

10:28 4G 78

Project Office Sep 1
To L1F22BSCS1111 - FAIZYAB AHMAD, +4

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	**CS-98**
Project Title	**INTELLIGENT SECURE MADITATION COUNSELOR-PATIENT CARE SYSTEM WITH AI VIRTUAL ASSISTANT**

Security concerns. At the end web app with AI support.

Prepare your Proposal Document in the light of comments provided. Last date to submit the Proposal Document is **Monday, September 15, 2025** not later than 04:00 PM. The proposal document should be prepared using the template available at http://smrms.ucp.edu.pk/All_Phases_Templates.html. S ubmit the hard copy to Project Office in F-304 (Main Building).

When submitting the hard copy of the proposal document, use the duly signed Group Allocation Yellow Page (available at http://smrms.ucp.edu.pk/Forms.html) as the first page of the submission document. Also get the proposal tape bind before submission.

Reply All

Mail Calendar Apps

Phase 2 (SDS) Feedback (Screenshot)

Project Office Reply Reply all Forward

To: L1F22BSCS0956- SAMEED SAEED; L1F22BSCS1111 - FAIZYAB AHMAD; **+1 other** Thu 4/30/2026 1:58 PM

Cc: Misha Asif

Dear Students,

Following is the feedback regarding your phase 2 (SDS):

F25CS186	**INTELLIGENT SECURE MADITATION COUNSELOR-PATIENT CARE SYSTEM WITH AI VIRTUAL ASSISTANT**

USD design is not as per the standard diagram. DFD is not well decomposed. Needs revision.

Overall good.

FoIT&CS Projects Office
F-304 (Cabin # 7)
042-35880007 ext: 207
Project Office Website http://smrms.ucp.edu.pk/

Reply Reply all Forward

Abstract

This project discusses the design and development of a dependable, AI-powered online platform intended to improve and speed up the functioning of counselling and patient care. The system is built around three unique role-based interfaces to satisfy the demands of all significant stakeholders. The platform allows therapists to handle patient appointment schedules, medical records, and session notes with the aid of artificial intelligence insights. Patients can more easily locate and book therapy appointments, participate in fully encrypted video sessions, track therapeutic tasks and individual health metrics, and interact with a personalized chatbot assistant anytime, anywhere because of the user interface. Administrative tasks are combined into a specialized portal for professional account management projects. One of the system's key innovations is its context-aware AI virtual assistant. This module analyses longitudinal patient data, including medical history, allergies, clinical notes, and personal preferences, to produce personalized, helpful recommendations for patients and therapists. Gamified goal tracking with a points system, automated reminders, integrated payment processing, an anonymous interaction mode, and visual progress analytics are additional features that improve engagement and favorable results. The platform's fundamental security and privacy development includes secure encryption, robust role-based access control (RBAC), and secure authentication procedures. By ensuring the security and privacy of sensitive health information, this comprehensive security framework fosters a trusted environment for the provision of digital healthcare.

1. Introduction

The web-based, highly secure Intelligent Therapist-Patient Management System with AI Virtual Assistant was designed specifically to enhance patient-therapist communication, expedite therapy session administration, and ensure secure communication. As the significance of mental health care increases, effective therapy management remains a significant challenge. Many therapists still rely on time-consuming, labor-intensive manual procedures that are prone to scheduling errors, such as phone calls and paper documentation. Patients often find it difficult to plan sessions, track their progress, and maintain their privacy while undergoing therapy. This project addresses these concerns by providing a centralized, intelligent system that enables therapists to register patients, manage comprehensive medical records, create flexible schedules, and conduct encrypted online sessions. By removing stigma and enabling greater access to care, patients will be able to find a therapist quickly, view real-time availability for appointments, schedule and pay for appointments easily, and remain anonymous while receiving care. An example of how the app will help patients sustain their therapy is through its AI-powered virtual assistant who will provide patients with ongoing assistance by providing them with customized suggestions and reminders that are tailored to their individual profiles, including information that consists of medical history, allergies, and therapist notes. The combination of automation, Artificial Intelligence driven personalization, and privacy protections that are at the core of the system results in a reduction of the administrative burden on therapists as well as greater patient engagement in therapy, improved adherence to treatment plans, and higher overall treatment outcomes. This innovative new app platform will provide a more effective, efficient, and reliable environment for patients to receive therapy while simultaneously satisfying the increasing demand for digital, accessible, and private mental health care needs. The complete software specifications that follow will be defined in this document.

1.1 Product

There are many major obstacles that are complicating the effective delivery of therapy and the engagement of patients by mental health care organizations' leadership today. Due to the use of manual procedures, most therapists are using phone-based scheduling and paper records to track patient progress, which introduces inefficiencies, takes a great deal of therapists' time, increases the likelihood of inaccurate appointment schedules, and ultimately results in the avoidance of therapy by some patients. In addition to that, if patients have difficulty scheduling their own appointments, tracking their progress towards recovery, finding privacy during sensitive appointments, etc.; many patients will avoid any type of mental health care completely. Additionally, the current digital platforms usually do not provide strong privacy protections, such as providing anonymous communication options and personalized support based on each individual user's medical history. Consequently, these factors lead to missed appointments, decreased adherence to therapy, increased burnout for therapists, and decreased overall mental health outcomes for patients. The Intelligent Therapist-Patient Management System will provide a secure automated intelligent web-based platform, to support the real-time scheduling of appointments and provide encrypted communication and utilize AI to generate a customised support experience for each patient. By doing so, the Intelligent Therapist-Patient Management System will improve the accessibility, effectiveness and reliability of mental health therapy by not only providing enhanced privacy and improving patient engagement but also eliminating all of the inefficiencies associated with manual scheduling procedures.

1.2 Background

This project focusses on modern digital mental healthcare, a developing field that addresses significant inefficiencies in conventional therapy by combining telehealth and medication management. Although remote sessions have made existing platforms more accessible, they frequently function in silos and provide separate solutions with little to no integration between a patient's continuing care journey and a therapist's clinical tools. This gap leads to impersonal patient care, stressful administrative workloads for professionals, and enduring privacy issues that discourage people from getting treatment. Within this framework, our project seeks to go beyond traditional teletherapy by developing a single, safe ecosystem that intelligently automates processes, encourages team progress monitoring, and provides genuinely customized, context-aware AI assistance to improve patient engagement and therapeutic results.

1.3 Objective(s)/Aim(s)/Target(s)

The objective of the proposed system is to develop an AI-based Secure Meditation Consultant Patient Support System that will provide cutting-edge technological enhancements to the way Mental Health professionals deliver consultation services by providing virtual automated support for Individualized Counselling, enhanced security and protection of confidential information about both patients and counsellors. The Secure Meditation Consultant Patient Support System provides Counsellors with tools for creating, tracking and communicating with their clients via a secure online environment. Clients will receive the latest information about therapy sessions and other resources related to their progress in therapy anonymously or anonymously via a unique virtual identity. Clients also get access to AI-based recommendations for their service delivery process through a single, highly secure web-based platform. In summary, the following represents the key objectives of the Secure Meditation Consultant Patient Support System:

1.3.1 Develop a Secure Therapist–Patient Management Platform

To create a centralized portal with encrypted storage and restricted access that enables therapists to register, manage, and keep an eye on patient profiles and therapy sessions while ensuring total data confidentiality.

1.3.2 Implement Automated Appointment Scheduling

To reduce scheduling conflicts and no-shows, a dynamic booking system that automatically verifies therapist availability, avoids double booking, and provides real-time confirmations should be designed.

1.3.3 Integrate Secure Communication and Consultation Channels

To reduce scheduling conflicts and no-shows, a dynamic booking system that automatically verifies therapist availability, avoids double booking, and provides real-time confirmations should be designed.

1.3.4 Incorporate an AI-Powered Virtual Assistant

To incorporate an intelligent assistant that can evaluate session data, therapist notes, and patient medical histories to offer tailored, context-aware advice and reminders about mental health.

1.3.5 Enable Goal Assignment and Progress Tracking

To enable patients to track their progress using a gamified points-based system that promotes engagement, and to let therapists assign customizable wellness or therapeutic goals.

1.3.6 Establish Secure Medical Record Storage

To keep a secure database of therapy notes, prescriptions, and clinical reports that is only accessible by authorized personnel and complies with accepted data security procedures.

1.3.7 Implement Robust Authentication and Access Control

To create a secure authentication system that protects user data from unwanted access by utilizing session management, role-based access control (RBAC), and encrypted credentials.

1.3.8 Ensure Comprehensive System Security

Enforcing secure API communication, end-to-end encryption for data in transit and resting, and ongoing monitoring to preserve information integrity, confidentiality, and dependability.

1.3.9 Automate Notifications and Reminders

To enhance communication by using integrated email and SMS services to automatically notify people about therapy exercises, upcoming appointments, and medication schedules.

1.3.10 Provide Data-Driven Progress Analytics

To produce AI-based reports and visual progress dashboards that let patients and therapists assess progress, spot trends in behaviors, and modify treatment plans as necessary.

1.3.11 Enhance Accessibility and User Experience

To create a user-friendly, responsive, and intuitive web interface that is optimized for a variety of devices, guaranteeing a smooth experience for patients and therapists of all technical backgrounds.

1.4 Scope

This Software Requirements Specification (SRS) defines an extensive, web-based platform to monitor, safeguard, and support the entire therapist-to-client continuum of care (including the interface between therapists/clients), as well as protect and enhance the overall connection between therapists and clients. The Therapist Portal will allow therapists to schedule appointments, store secure medical records, and register patients; while the Patient Portal will enable patients to view their medical records, book real-time appointments, pay using integrated payment systems, and track their progress through a gamified goal-setting system. The core of the platform will utilize an artificial intelligence (AI) virtual assistant that will evaluate the patient profile and provide personalized recommendations based on the context of the patient's treatment. The platform will also include shared feedback mechanisms between PCPs and patients, allow patients to interact anonymously with their therapist or PCP, and provide secure video calls to conduct therapy, which will be encrypted end-to-end. The platform will not include AI automated therapy, support for native mobile applications or native mobile interfaces (other than a responsive web application), integration of third-party medical devices, provision of in-person direct medical diagnosis, and/or handling and filing insurance claims.

1.5 Business Goals

The **AI-Integrated Secure Meditation Counsellor–Patient Care System** is designed not only as a technical solution but also as a sustainable and scalable digital healthcare product that fulfils clear business and corporate objectives. One of the primary business goals of the system is to enable mental health professionals to manage and grow their practices more efficiently through automation of core operational processes such as patient registration, appointment scheduling, session management, billing, and progress tracking. By reducing administrative workload and manual effort, therapists can dedicate more time to patient care, thereby increasing service quality and professional productivity.

Another key business goal is to establish a **subscription-based revenue model** for therapists and counselling professionals. Through tiered subscription plans and optional free trial periods, the platform aims to generate consistent recurring revenue while allowing professionals to evaluate system value before full adoption. This model supports long-term financial sustainability and continuous system improvement. Additionally, integrated payment processing for therapy sessions simplifies financial transactions, reduces payment delays, and improves cash flow for service providers.

The system also serves as a **digital discovery and engagement platform**, helping patients easily locate licensed therapists based on availability and specialization. This increases therapists' visibility and client reach, expanding their professional network beyond geographical limitations. From a corporate perspective, the platform aims to position itself as a trusted, privacy-focused mental health technology brand by prioritizing strong security, ethical AI usage, and regulatory compliance (HIPAA/GDPR), which are critical factors for user trust and market credibility.

Furthermore, the business strategy emphasizes **scalability and future expansion**. The system is architected to support future enhancements such as mobile application development, advanced AI analytics, and additional wellness services without major redesign. By leveraging cloud infrastructure and modular design, the platform can scale with increasing user demand while maintaining performance and reliability. Overall, the business goals focus on creating a commercially viable, secure, and intelligent ecosystem that benefits therapists, patients, and stakeholders while supporting long-term growth in the digital mental healthcare market.

1.6 Document Conventions

The IEEE Software Requirements Specification (IEEE 830-1998) guidelines served as the model for the structured and standardised format used in this Software Requirements Specification (SRS) document. For readability, accuracy, and convenience of reference, it keeps its layout simple and uniform. Throughout the document, the following conventions have been adopted:

1.7.1 Font Style and Size:

The document is written using *Times* font, with headings in bold (size 14) and main text Arial font with 11 size and sub heading in regular (size 12) for professional presentation and clarity.

1.7.2 Numbering Scheme:

All sections and subsections are numbered hierarchically (e.g., 1.1, 1.2, 1.3, etc.) to provide clear organization and logical flow of information.

1.7.3 Terminology:

Key terms such as *Therapist*, *Patient*, *System*, *AI Assistant*, and *Anonymous Mode* are capitalized to indicate system entities or core components within the project.

1.7.4 Symbols and Notations:

Standard symbols (✓, ✗) are used in comparison tables to indicate the presence or absence of specific features. Acronyms such as AI (Artificial Intelligence), API (Application Programming Interface), and RBAC (Role-Based Access Control) are expanded when first introduced.

1.7.5 Language Style:

The document uses formal, technical, and concise language suitable for academic and professional contexts. Passive voice is used where appropriate to emphasise process and functionality.

1.7.6 Figures and Tables:

Figures and tables are labelled sequentially (e.g., *Figure 1*, *Table 1*) and referenced within the text where relevant to maintain coherence and traceability.

1.7.7 Emphasis:

Important terms and definitions are either bolded or italicized to draw attention to key points or technical terminology.
These conventions ensure that the document maintains a consistent format, allowing readers, developers, and evaluators to interpret the content accurately and efficiently.

1.7 Miscellaneous

This section provides additional information and guidelines that are intended to support the successful organization, development, and execution of the project. The following points should be considered throughout the project lifecycle:

1.7.1 Project Management Practices:

Clear assignment of roles and responsibilities among team members is essential. Regular progress reviews, milestone tracking, and documentation updates should be maintained to ensure timely delivery of project tasks.

1.7.2 Version Control and Collaboration:

The use of a version control system (e.g., Git) is recommended to manage source code, documentation, and collaborative development efficiently. Proper branching strategies, code reviews, and commit documentation should be followed.

1.7.3 Testing and Quality Assurance:

Comprehensive testing strategies, including unit testing, integration testing, system testing, and user acceptance testing, should be implemented. Test plans, test cases, and bug tracking procedures must be documented to ensure system reliability and performance.

1.7.4 Documentation Standards:

All project artifacts, including design diagrams, code documentation, and user manuals, should follow the conventions outlined in this SRS. This ensures consistency, readability, and ease of use for developers, evaluators, and end users.

1.7.5 Communication Channels:

Effective communication among team members, stakeholders, and supervisors is critical. Scheduled meetings, progress reports, and feedback loops should be maintained to ensure alignment with project objectives.

1.7.6 Risk Management:

Potential project risks, such as technical challenges, resource limitations, or security vulnerabilities, should be identified early. Contingency plans and mitigation strategies should be documented and periodically reviewed.

1.7.7 Learning and Knowledge Sharing:

Team members should document lessons learned, innovative approaches, and insights gained during project development. This knowledge base will aid future maintenance, upgrades, and similar projects.

1.7.8 Ethical Considerations:

Given the sensitive nature of patient data, adherence to privacy laws, ethical standards, and security best practices (HIPAA/GDPR compliance) is mandatory throughout the development and deployment of the system.

By incorporating these miscellaneous guidelines, the project ensures a structured, professional, and secure approach to software development, facilitating a smooth workflow, effective collaboration, and the delivery of a reliable and user-centric Intelligent Therapist–Patient Management System.

2. Technical Architecture

BENZI.AI is a fully custom-built software system designed and developed from the ground up to address the specialized requirements of secure mental health counselling, therapist–patient interaction, and AI-assisted meditation guidance. No Commercial Off-The-Shelf (COTS) solution was adopted, as no existing product sufficiently addresses the combined requirements of healthcare data privacy, AI personalization, and role-based multi-portal access. The following subsections describe the complete architecture of the system, covering application components, data management, interaction patterns, design patterns, tools and technologies, and infrastructure.

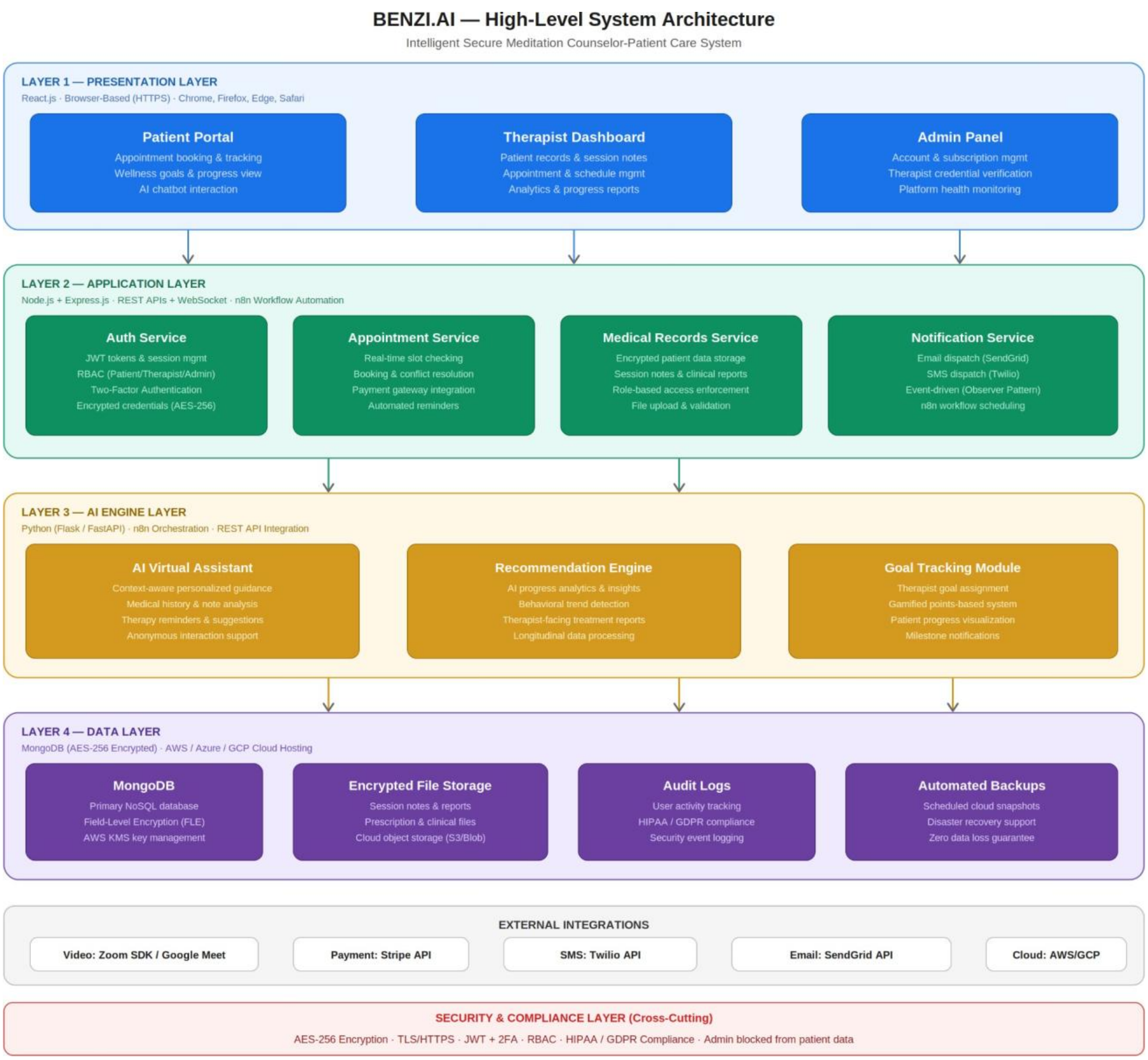


Figure 1.1: BENZI.AI High-Level System Architecture — Layered Client-Server Model

The high-level architecture of BENZI.AI follows a four-layer client-server model: the Presentation Layer (React.js web interfaces), the Application Layer (Node.js + Express.js RESTful backend), the AI Engine Layer (Python with n8n automation), and the Data Layer (MongoDB cloud database). All inter-layer communication is performed over HTTPS using secure REST APIs and WebSocket connections. The system supports both online and batch processing. Online processing governs real-time interactions including user authentication, appointment booking, live therapy sessions, AI chatbot queries, and payment handling. Batch processing manages background tasks such as automated reminders, progress report generation, AI analytics processing, and scheduled database backups. The system combines transaction processing for managing appointments, therapy sessions, and payments with analytical reporting, including AI-generated progress dashboards and therapist-facing treatment insights.

2.1 Application and Data Architecture

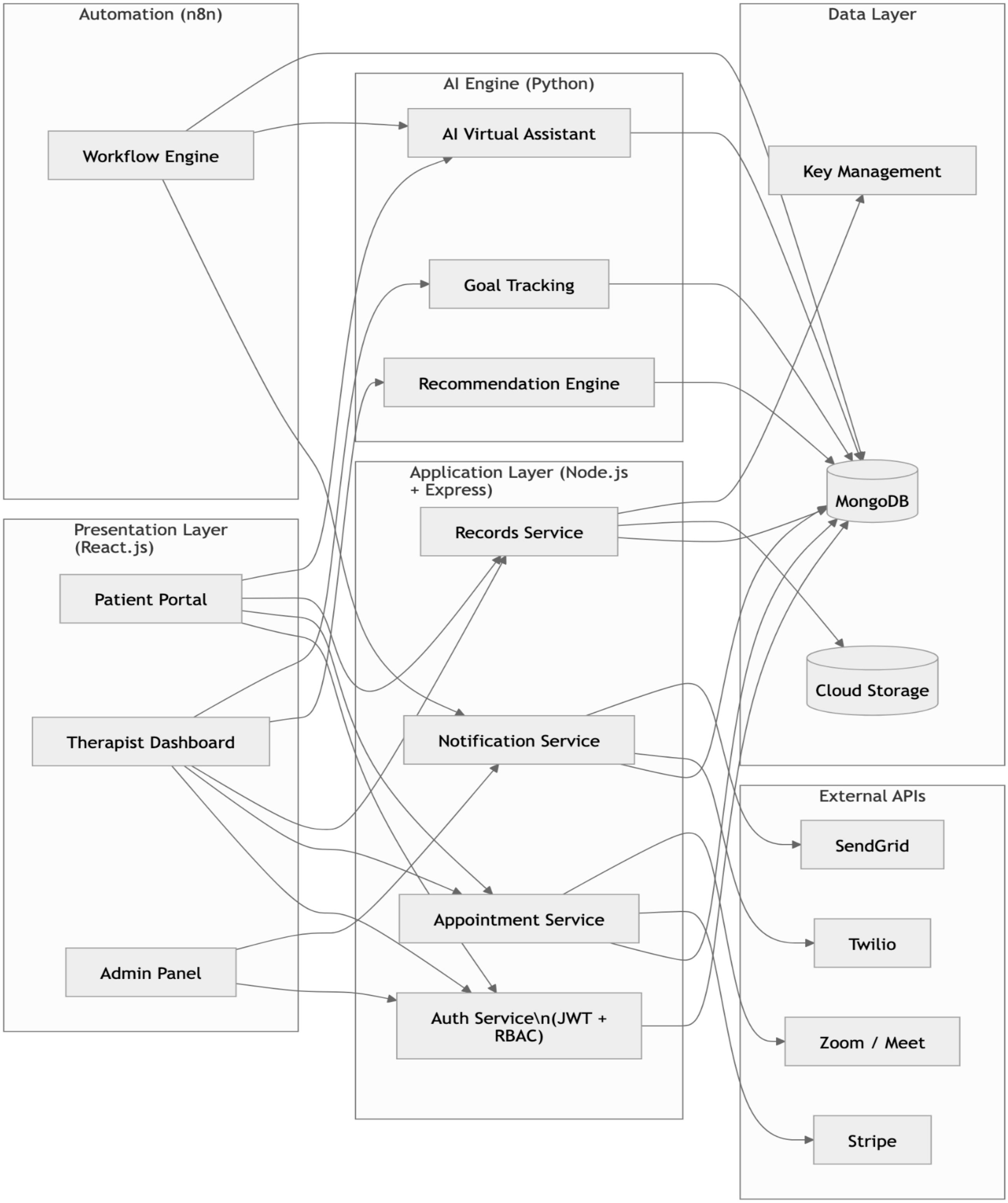


Fig 2.2 Component Diagram

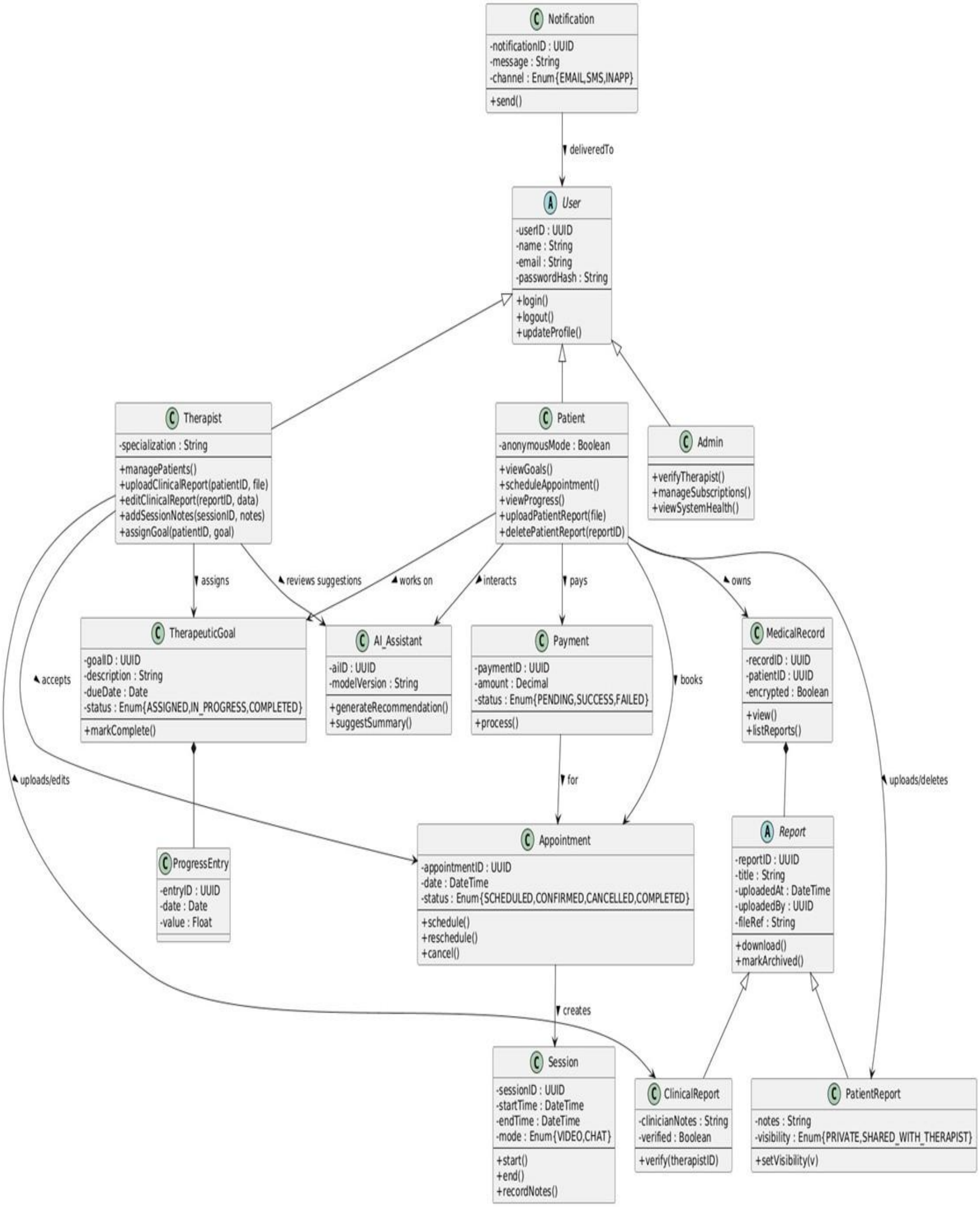


Fig 2.3 Abstract Class Diagram

The application architecture of BENZI.AI consists of six major components operating across four logical layers, as illustrated in Figure 2.2 These components collectively serve three distinct user roles: Patient, Therapist, and Administrator.

Application Components

The frontend web application is implemented in React.js and serves as the browser-based interface for all three roles. Three dedicated portals are provided: the Patient Portal, which enables appointment booking, goal tracking, AI assistant interaction, and progress visualization; the Therapist Dashboard, which supports patient registration, session management, medical record maintenance, and analytics; and the Admin Panel, which provides account management, therapist credential verification, subscription control, and platform health monitoring.

The backend application server is implemented using Node.js and Express.js. It provides RESTful API endpoints consumed by all frontend portals and enforces all business logic and security policies. Core backend services include the Authentication and Authorization Service (managing JWT tokens, RBAC, and session control), the Appointment Management Service (real-time slot checking, booking, and conflict resolution), the Medical Records Service (encrypted storage and retrieval of patient data, session notes, and clinical reports), and the Notification Service (automated email and SMS dispatching via Stripe, Twilio, and SendGrid).

The AI Engine is developed in Python, using either Flask or FastAPI as the service framework, and is integrated with the backend through secure RESTful APIs. AI workflows and scheduling are orchestrated through n8n, a self-hosted workflow automation platform. The AI Engine consists of three sub-modules: the AI Virtual Assistant (providing context-aware, personalized recommendations based on patient medical history, therapist notes, and session data), the Recommendation Engine (generating progress insights and behavioral trend analysis for therapist-facing dashboards), and the Goal Tracking Module (supporting the gamified, points-based therapeutic goal system).

The Automation Engine is implemented through n8n workflows and is responsible for all background and scheduled operations, including automated reminder dispatch, report generation, periodic AI analytics execution, and workflow orchestration between the Python AI modules and the Node.js backend.

Data Components

The system collects and manages the following categories of data: user registration and authentication credentials, therapist professional profiles and verification documents, patient medical histories (allergies, diagnoses, and treatment plans), therapy session notes and clinical annotations, AI-generated recommendations and interaction logs, appointment records and payment transactions, wellness goal assignments and completion histories, system audit logs, and platform usage analytics.

All sensitive healthcare data is stored using MongoDB field-level encryption (FLE), with encryption key management handled through a cloud-based Key Management Service (KMS) such as AWS KMS. Object storage services on the cloud host are used for file-based records (reports, session recordings, uploaded documents). Regular automated backups are configured to ensure data durability and disaster recovery compliance.

2.2 Component Interactions and Collaborations

All components of BENZI.AI interact through a well-defined communication architecture. The frontend portals communicate exclusively with the backend application server over HTTPS using RESTful APIs. For features requiring real-time bidirectional communication — specifically live therapy sessions and chat — WebSocket connections are established between the client and the backend. The backend server acts as the central orchestrator, routing requests to the appropriate service module (authentication, records, appointments, or notifications) and invoking the AI Engine through secured API calls when AI processing is required.The AI Engine receives patient profile data and session context from the backend, processes it through the recommendation and assistant modules, and returns structured outputs that are stored in MongoDB and surfaced to the relevant user dashboards. The n8n automation platform acts as the scheduling and workflow layer, triggering AI pipeline executions, sending notifications through third-party APIs, and managing retry logic for failed background operations.Third-party integration components connect to the backend through authenticated API clients. Video conferencing is embedded through the Zoom SDK and Google Meet API; payment processing is handled through Stripe; and messaging services (email and SMS) are managed through Twilio and SendGrid. All external API calls are made over HTTPS, with credentials stored securely using environment variables and cloud secret management services.The Sequence Diagram and Activity Diagram for key system workflows are provided in Figures X and X respectively, illustrating the design-level interaction flows for appointment booking, therapy session conduct, AI assistant interaction, and goal tracking. These diagrams extend the high-level models presented in Phase 2 with full method signatures, return types, and inter-service message flows.

The Service-Oriented Architecture (SOA) Pattern is applied to core backend functions. Authentication, appointment scheduling, medical records management, AI recommendation delivery, and notification dispatching are each implemented as independent, self-contained services that communicate through

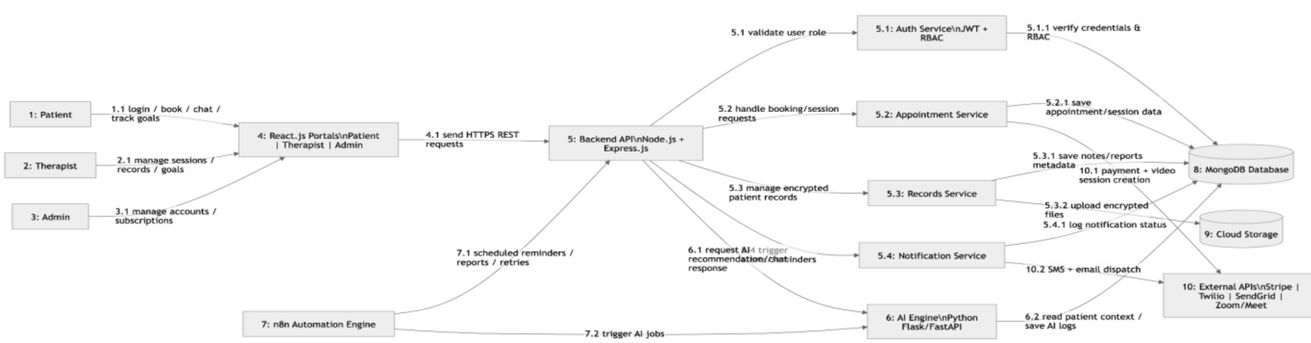


Fig 2.4 Collaboration Diagram

DFD:

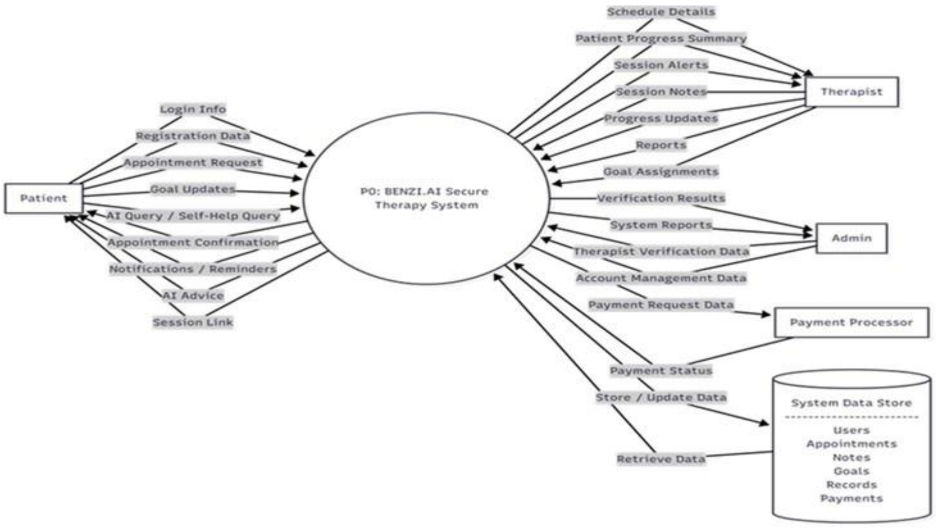


Fig 2.5 Level 0 DFD

Fig 2.5 DFD Level 0

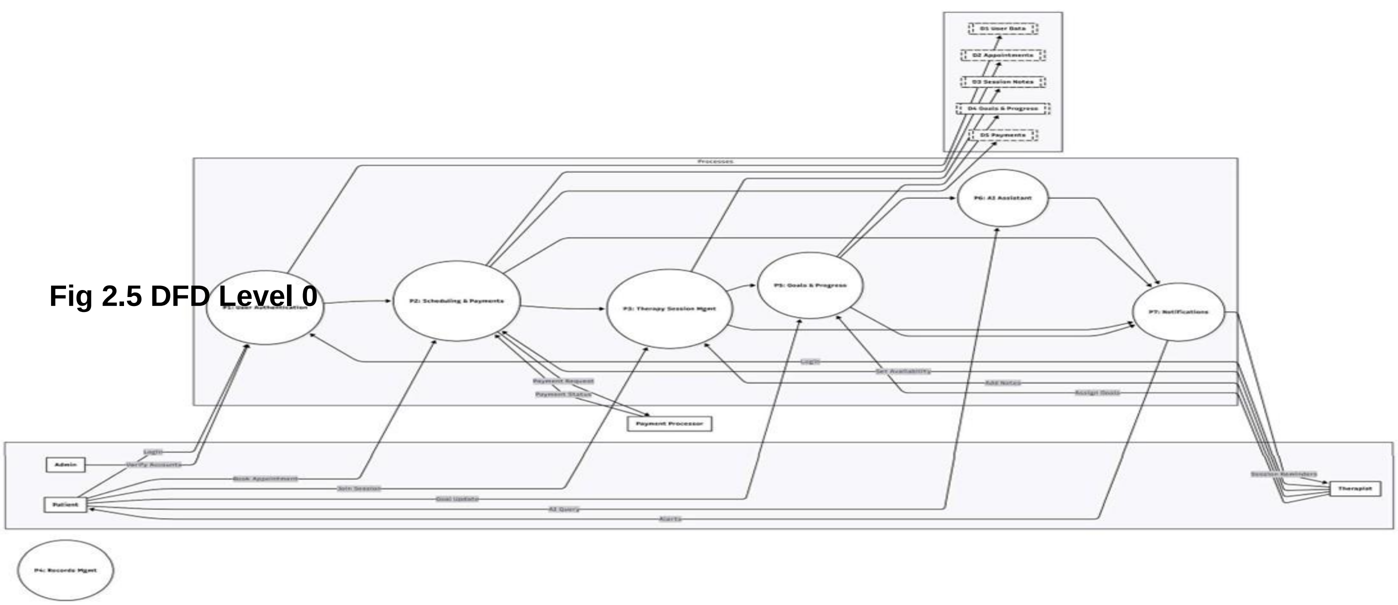


Fig 2.6 Level 1 DFD

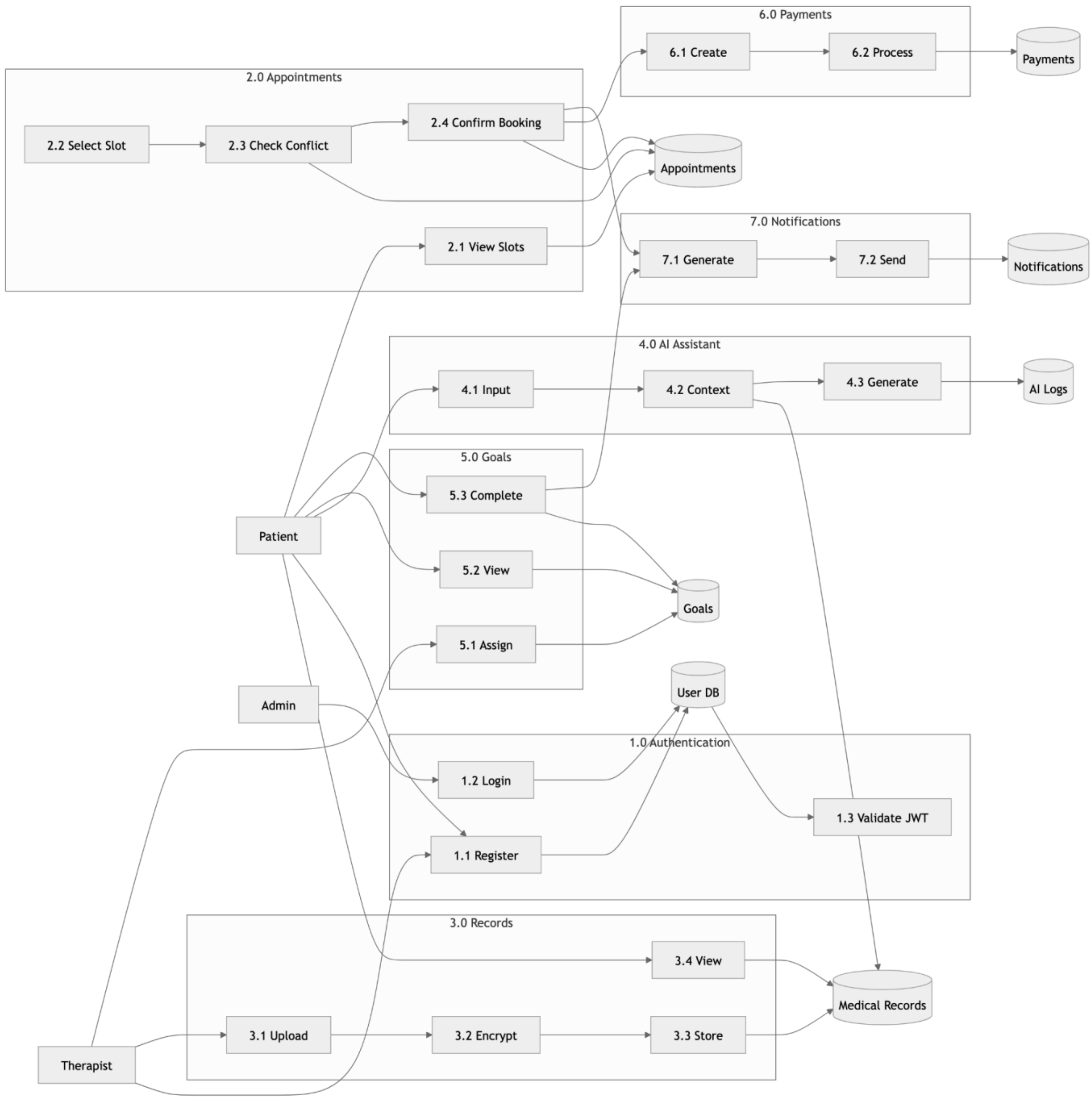


Fig 2.7 Level 2 DFD

Activity Diagram:

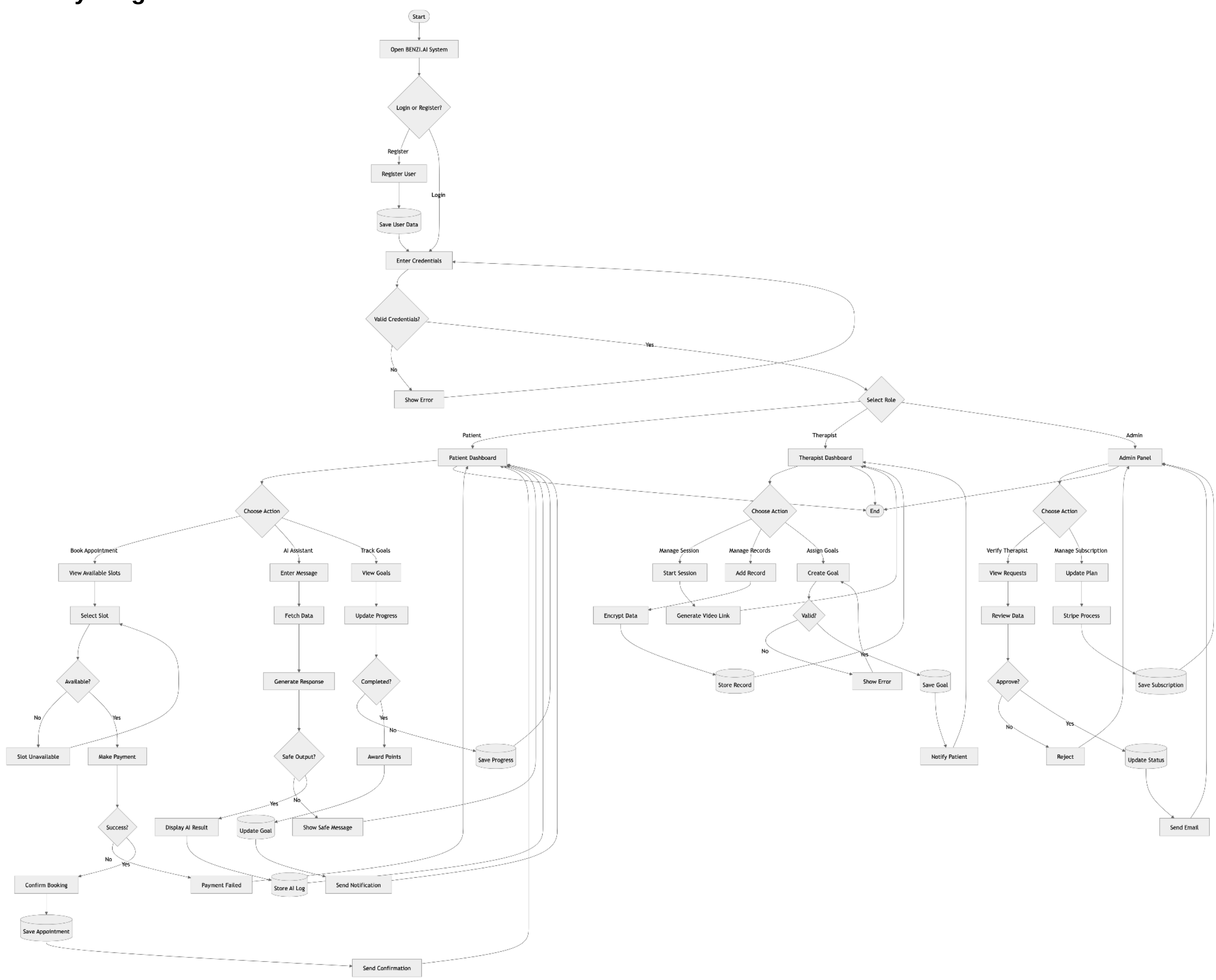


Fig 2.8 Activity Diagram

2.3 Design Reuse and Design Patterns

The BENZI.AI system has been architected with a strong emphasis on modularity and reusability, employing the following established software design patterns throughout its implementation.

The Layered Architecture Pattern governs the overall system structure by enforcing strict separation of concerns across the Presentation, Application, AI, and Data layers. No layer communicates with a non-adjacent layer directly, ensuring maintainability and independent scalability.

The Model-View-Controller (MVC) Pattern organizes the backend codebase. Database entity definitions serve as Models, Express.js controllers encapsulate business logic and API request handling, and the React.js frontend components constitute the View layer.

2.3 Design Reuse and Design Patterns

REST APIs. This enables individual services to be scaled, replaced, or extended without affecting other system components.

The Observer Pattern governs the notification subsystem. User-facing events — including appointment confirmations, goal completion milestones, and session reminders — automatically trigger notification dispatch without requiring direct coupling between the business logic and the messaging infrastructure.

The Factory Pattern is applied to user role instantiation and AI response handler creation. The system dynamically generates the appropriate user context object (Patient, Therapist, or Admin) and AI response type at runtime based on authenticated session data.

The Singleton Pattern ensures that system-wide resources including the MongoDB database connection, configuration managers, and logging services are instantiated once and shared consistently across all modules, preventing resource duplication and ensuring uniform state management.

Reusable Components: A centralized Authentication and RBAC module is reused across all three portals. React.js UI components including navigation bars, appointment cards, modal dialogs, notification banners, and form elements are developed once and reused across all screens. RESTful API endpoints for user profiles, appointments, and analytics are designed as shared services consumed by multiple modules. The AI recommendation logic is implemented as a standalone, reusable service invokable from therapy sessions, goal tracking, and progress dashboards. The n8n automation workflows for reminders and report generation are designed as reusable templates applicable across multiple system functions.

2.4 Technology Architecture

BENZI.AI is designed to operate as a cloud-based, scalable, and highly available platform. The complete technology stack is summarized in Table X below.

Table 1: Technology Stack Summary

Layer	Component	Technology / Tool
Presentation	Web Application	React.js
Application	Backend Server	Node.js, Express.js
Application	Workflow Automation	n8n
AI Engine	AI Module	Python (Flask / FastAPI)
Data	Primary Database	MongoDB
Data	Key Management	AWS KMS / Azure Key Vault
Integration	Video Conferencing	Zoom SDK, Google Meet API
Integration	Payment Processing	Stripe API
Integration	Messaging (SMS)	Twilio API
Integration	Messaging (Email)	SendGrid API
Infrastructure	Cloud Hosting	AWS / Microsoft Azure / GCP
Security	Data Encryption	AES-256, TLS/HTTPS, E2EE
Security	Authentication	JWT, 2FA, RBAC
DevOps	Version Control	Git / GitHub

System Hosting: The system is hosted on an external cloud data center — Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) — providing elastic scalability, high availability, automated backups, and disaster recovery. Containerized or virtualized deployment of all application services, AI modules, databases, and automation workflows is supported.

Hardware Infrastructure: The system operates on cloud-based virtual machines providing scalable CPU and memory resources for concurrent user sessions, optional GPU support for AI model inference, secure encrypted storage volumes, and load balancers to distribute incoming traffic across application instances. No dedicated on-premises hardware is required.

Network Architecture: BENZI.AI operates entirely over the Internet using secure HTTPS communication. All client-server interactions are encrypted using TLS 1.2 or higher. Real-time features (live chat, video sessions) require stable broadband connectivity with low latency and are supported through WebSocket connections. The system is not available on a LAN or private WAN —

it is a public-internet, cloud-hosted platform accessible from any modern web browser without client-side software installation.

End-User Interface: The system provides a browser-based end-user interface accessible through all major modern browsers (Chrome, Firefox, Edge, Safari). No thick-client installation is required. The interface is responsive and supports desktops, tablets, and mobile browsers.

Modes of Operation: The system supports three operational modes. Online Mode enables real-time interactions including user authentication, appointment booking, therapy sessions, AI chatbot communication, and dashboard updates. Background Mode manages automated workflows for reminders, notifications, analytical report generation, and scheduled AI tasks. Maintenance Mode supports system updates, security patches, and performance monitoring with minimal service inte*rruption.*

2.5 Architecture Evaluation

The architectural decisions made for BENZI.AI were guided by the specific requirements of a secure, scalable, AI-integrated mental health platform. This subsection justifies the key technology choices and evaluates each against alternative options.

React.js vs Angular / Vue.js: React.js was selected for the frontend due to its component-based architecture, which directly supports the creation of reusable UI elements across the three portals, its large ecosystem, and its flexibility for integration with third-party libraries. Angular, while offering a more opinionated structure, introduces higher complexity and steeper learning curve unsuitable for the project timeline. Vue.js, although lightweight, has a comparatively smaller ecosystem and less community support for enterprise-scale healthcare UI patterns.

Node.js + Express.js vs Django / Spring Boot: Node.js was selected as the backend framework for its non-blocking, event-driven architecture, which is well-suited to real-time operations such as WebSocket-based therapy sessions and concurrent API requests. Django (Python) was considered but excluded because it would require maintaining two separate Python environments — one for Django and one for the AI modules — adding unnecessary operational complexity. Spring Boot (Java) was excluded due to its greater infrastructure overhead and mismatch with the team's primary skill set.

MongoDB vs PostgreSQL / MySQL: MongoDB was selected as the primary database due to its schema flexibility, which is critical for managing semi-structured healthcare data (session notes, AI-generated recommendations, and variable-length medical histories), and its native support for field-level encryption. PostgreSQL, while more mature for relational data, would require more complex schema migrations when the data model evolves. The system's data is predominantly document-oriented, making a NoSQL solution structurally appropriate.

Python (Flask / FastAPI) for AI vs Node.js-native AI: Python was selected for all AI components due to its unmatched ecosystem of machine learning libraries (TensorFlow, PyTorch, scikit-learn, HuggingFace Transformers). Implementing AI logic natively in Node.js would significantly limit the availability of trained model architectures and community support. The separation of the AI module as an independent Python service, integrated through REST APIs, also supports independent scaling and model replacement without affecting the main application backend.

n8n vs AWS Step Functions / Apache Airflow: n8n was selected for workflow automation due to its self-hosted nature (supporting HIPAA/GDPR compliance by keeping data processing within the controlled environment), its visual workflow designer that simplifies maintenance, and its native REST

API integration. AWS Step Functions would introduce vendor lock-in and higher operational cost. Apache Airflow, while powerful, is over-engineered for the scheduling complexity required by this project and would require significant DevOps investment to maintain.

Cloud Hosting (AWS / Azure / GCP) vs On-Premises: Cloud hosting was selected to provide elastic scalability, high availability, and built-in disaster recovery without the capital investment of on-premises infrastructure. HIPAA and GDPR compliance is achievable on all three major cloud providers through their respective Business Associate Agreements (BAA) and compliance certification programs (e.g., AWS HIPAA-eligible services, Azure Healthcare APIs).

3. Detailed Component Design

ERD:

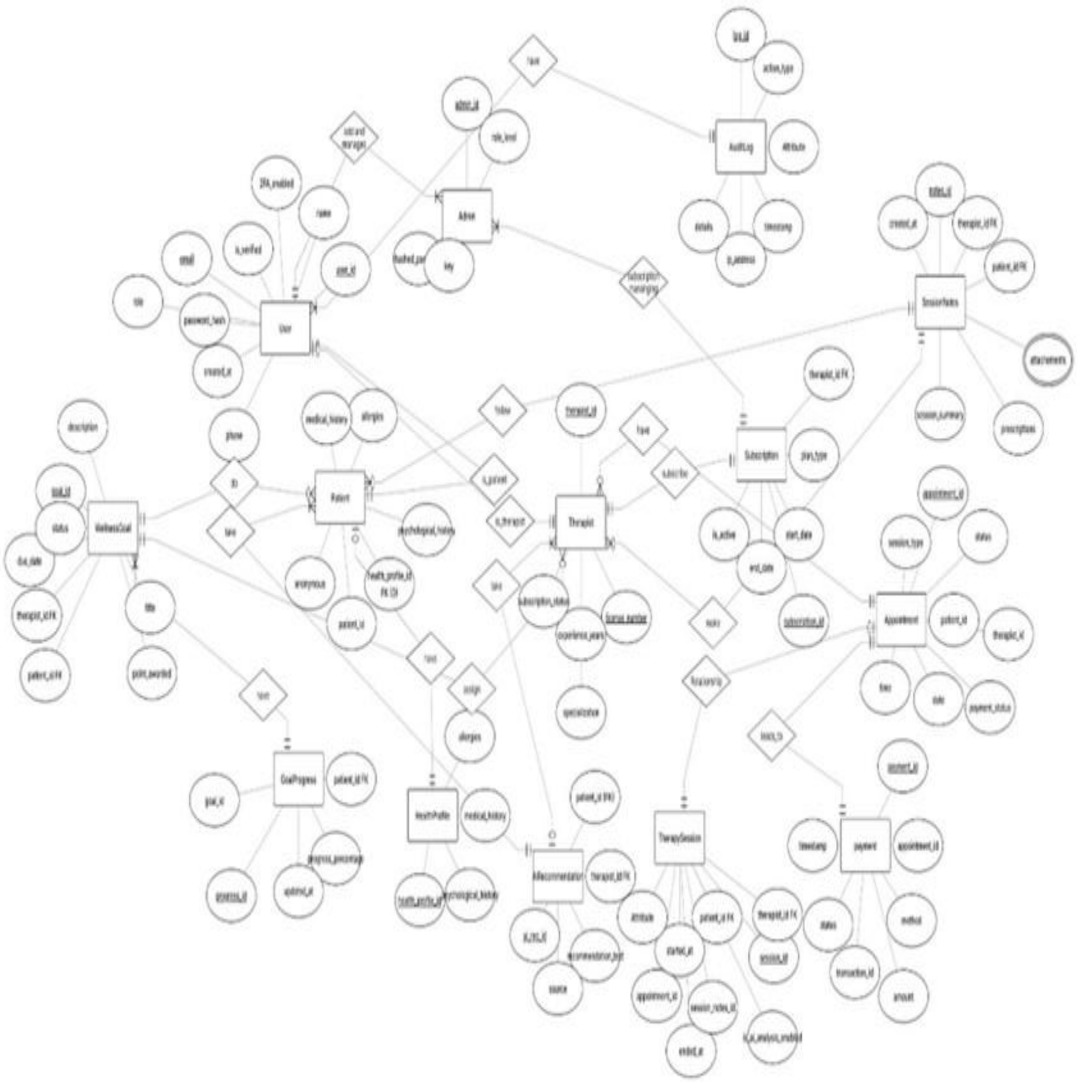


Fig 3.1 ERD

3.1 Component-Component Interface

This section describes in detail how the internal components of BENZI.AI communicate, collaborate, and exchange data with one another. All inter-component communication within the system follows a strict layered protocol: the frontend React.js portals communicate exclusively with the Node.js + Express.js backend through secured HTTPS REST API calls; the backend routes requests to the appropriate internal service module; the AI Engine communicates with the backend through dedicated REST endpoints over HTTPS; and the n8n automation engine invokes internal backend API endpoints on scheduled or event-driven triggers. No component communicates directly with the database except through its designated backend service — the frontend has zero direct database access at all times.

The following design-level sequence diagrams document the precise message flows, API endpoint calls, method invocations, database operations, return types, and alternate flows for each of the six primary use cases of the system. Each diagram uses filled arrowheads for synchronous calls and dashed open arrowheads for return messages, consistent with UML 2.0 sequence diagram notation.

Deployment Diagram

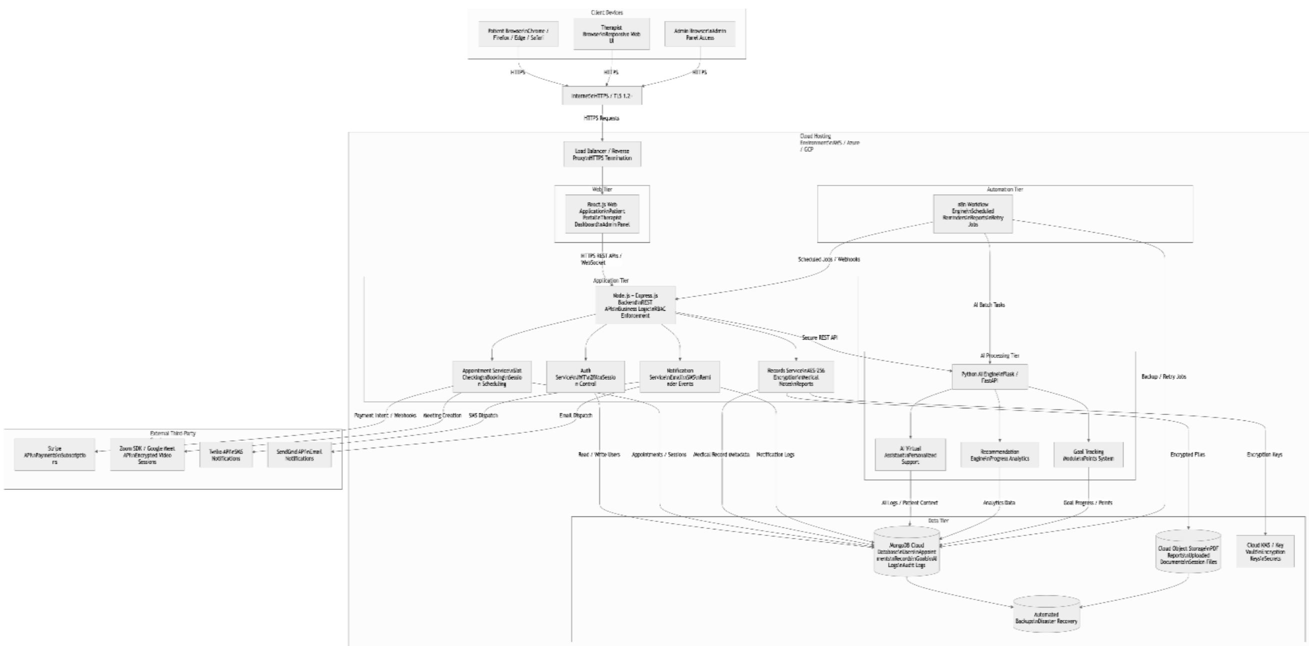


Fig 3.2 Deployment Diagram

Database Diagram :

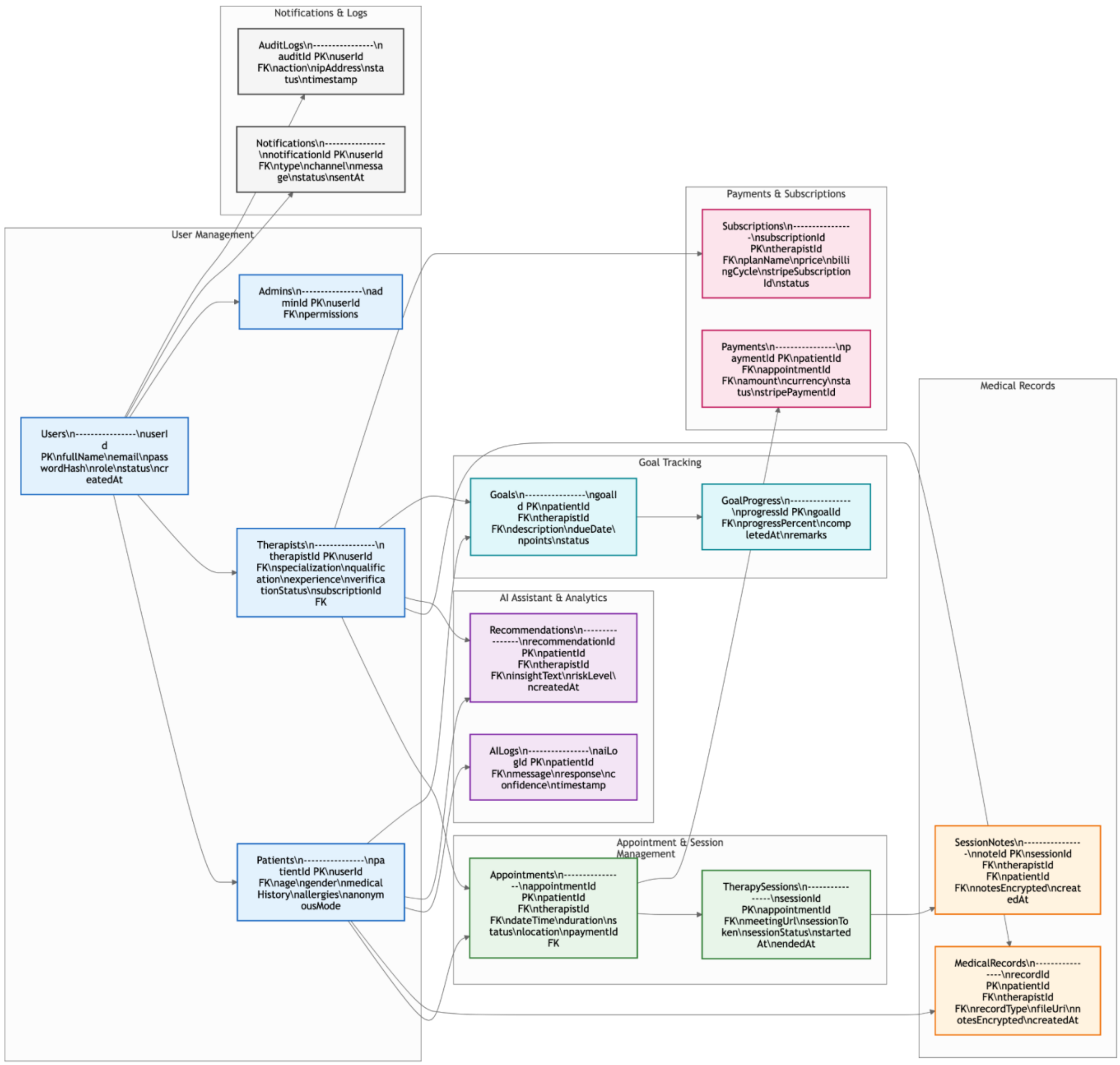


Fig 3.3 Database Diagram

State Diagram:

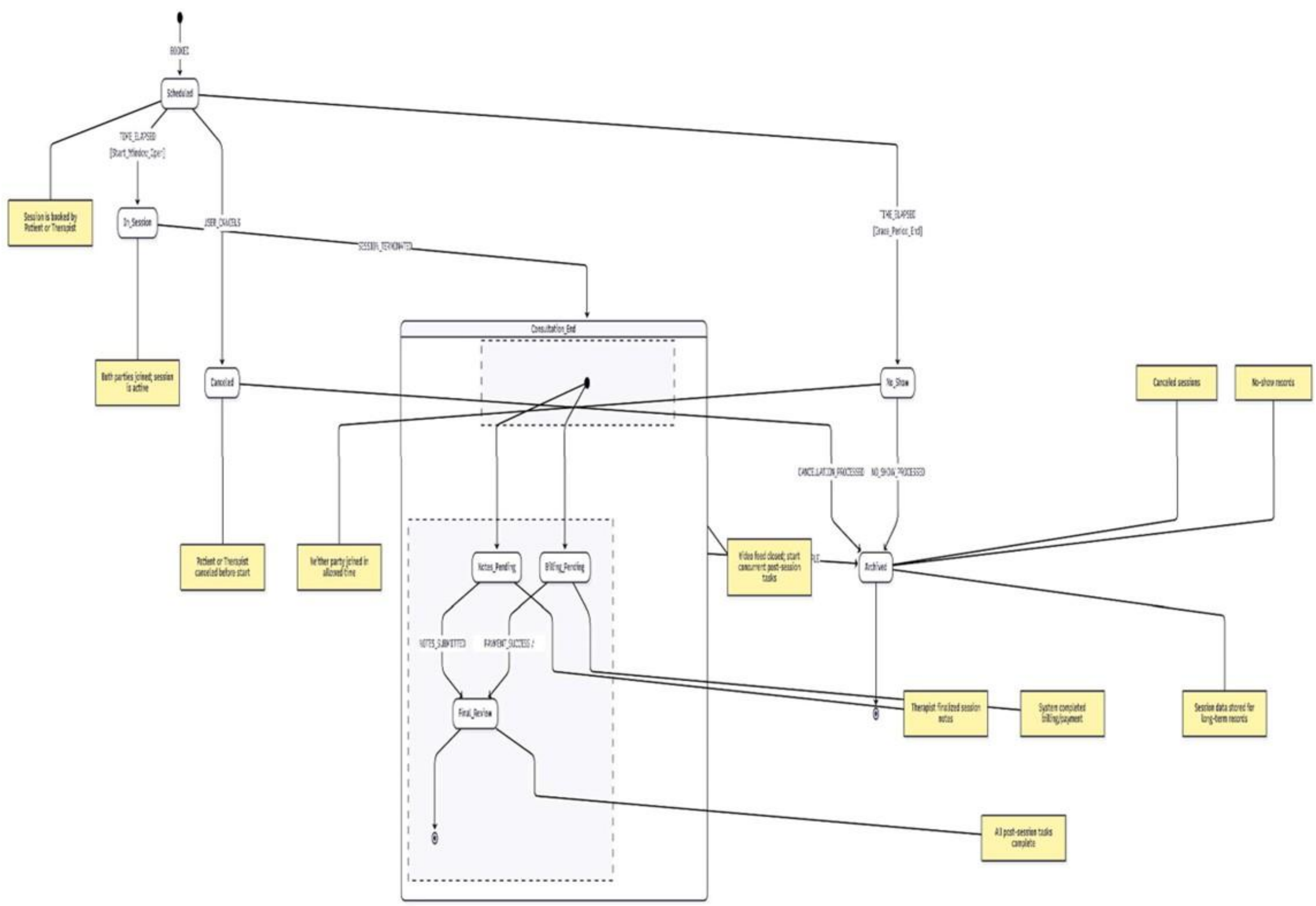


Fig 2.8 State Diagram

Petrinet diagram:

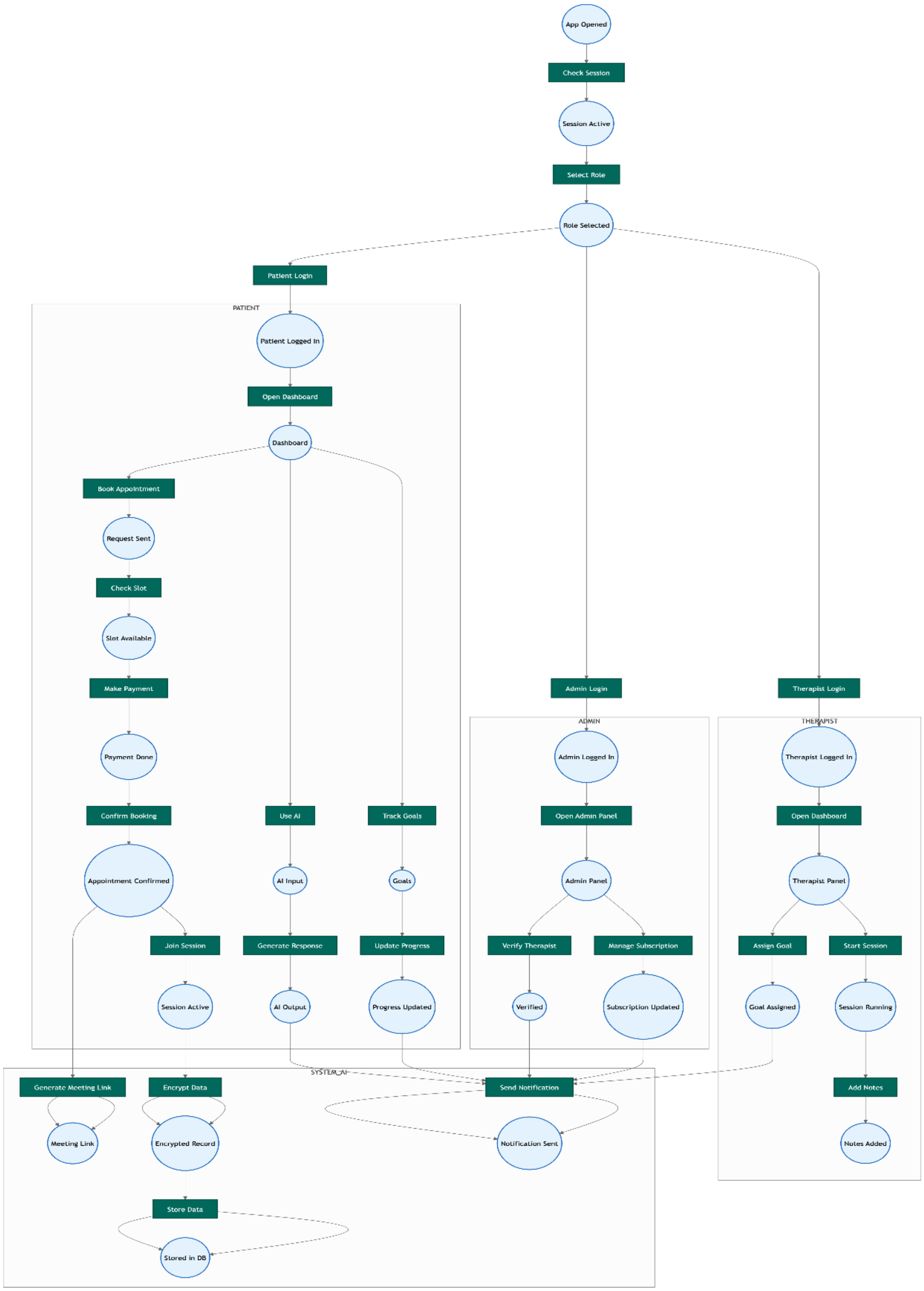


Fig 2.9 Petrinet Diagram

3.1.1 UC-1: Book Appointment with Therapist

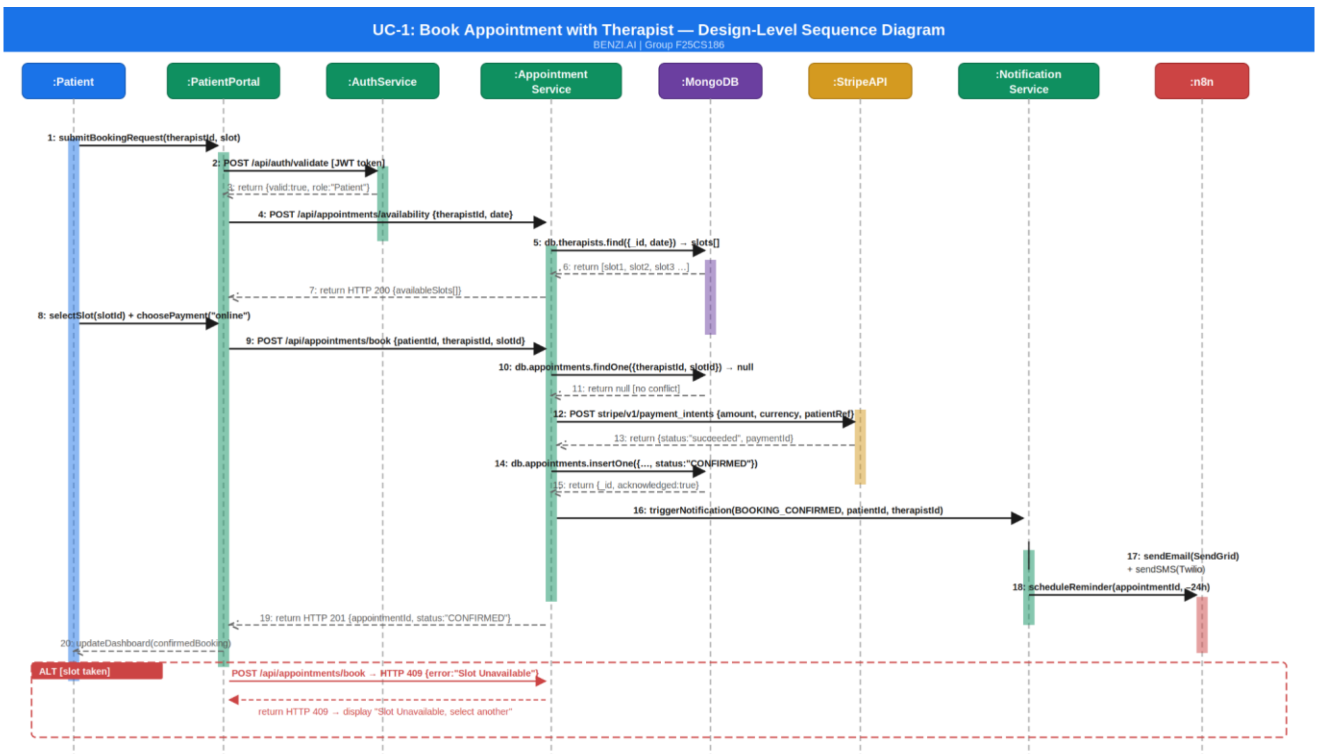


Figure 2: UC-1 Book Appointment — Design-Level Sequence Diagram

The appointment booking flow involves six components: PatientPortal, AuthService, AppointmentService, MongoDB, StripeAPI, and NotificationService. The Patient initiates a booking request from the Patient Portal, which first validates the session token with the AuthService via `POST /api/auth/validate`, receiving confirmation of role `"Patient"` and a valid JWT. The portal then calls `POST /api/appointments/availability` on the AppointmentService, which queries MongoDB using `db.therapists.find({_id, date})` and returns the available slot array to the frontend for display.

Once the patient selects a slot and chooses online payment, the portal calls `POST /api/appointments/book` with `{patientId, therapistId, slotId}`. The AppointmentService performs a conflict check via `db.appointments.findOne({therapistId, slotId})` — a null return confirms no double-booking. The service then calls Stripe's `POST stripe/v1/payment_intents` endpoint with `{amount, currency, patientRef}`. On receiving `{status:"succeeded", paymentId}`, the AppointmentService persists the appointment via `db.appointments.insertOne({…, status:"CONFIRMED"})`. The Observer Pattern then triggers the NotificationService, which dispatches a confirmation email via SendGrid and an SMS via Twilio to both Patient and Therapist. Finally, the n8n automation engine is invoked to schedule a 24-hour reminder workflow for the appointment. The portal receives `HTTP 201 {appointmentId, status:"CONFIRMED"}` and updates the patient dashboard.

In the alternate flow where the selected slot is already taken, the AppointmentService returns `HTTP 409 {error:"Slot Unavailable"}` and the patient is redirected to select a different time. If payment fails, Stripe returns an error event, the appointment record is not persisted, and the patient receives a failure notification.

3.1.2 UC-2: Conduct Therapy Session (Encrypted + AI Support)

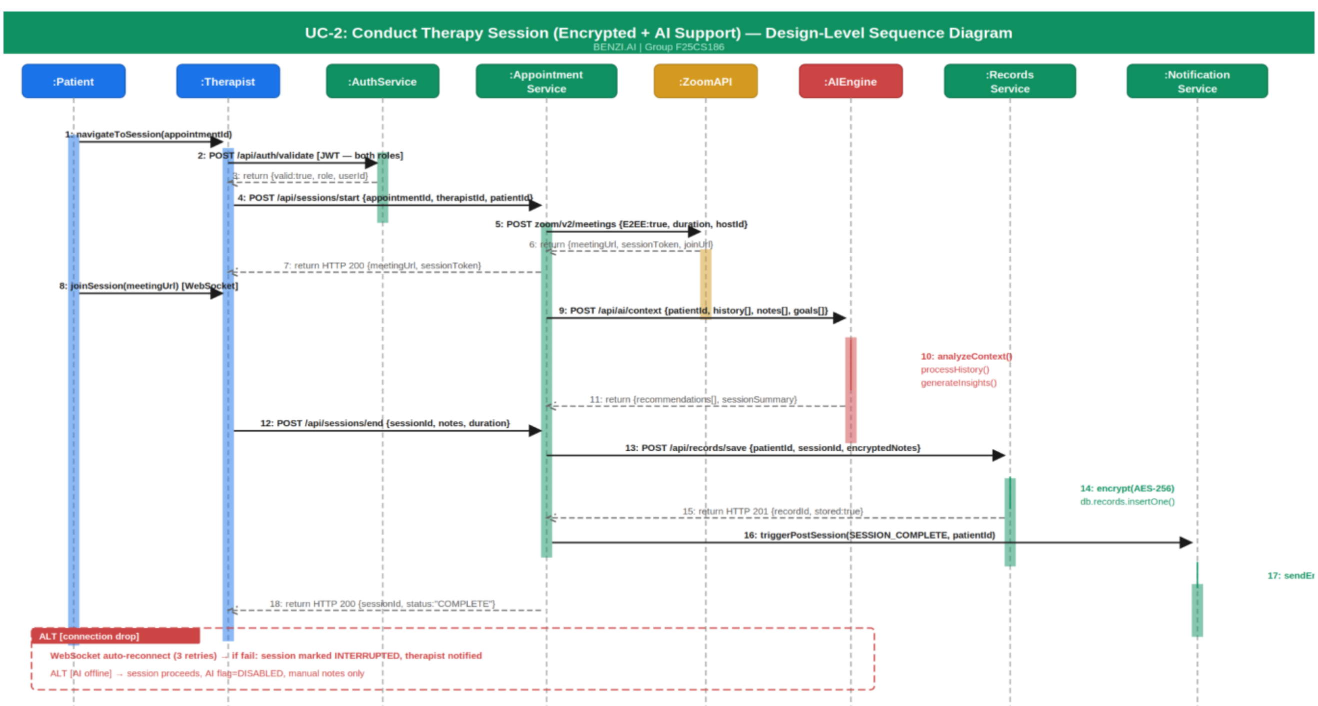


Figure 3: UC-2 Conduct Therapy Session — Design-Level Sequence Diagram

The therapy session flow involves eight components: Patient, Therapist, AuthService, AppointmentService, ZoomAPI, AIEngine, RecordsService, and NotificationService. Both the Patient and Therapist authenticate independently through `POST /api/auth/validate`, each receiving their respective role-confirmed JWT tokens. The Therapist Dashboard calls `POST /api/sessions/start {appointmentId, therapistId, patientId}` on the AppointmentService, which in turn calls Zoom's `POST zoom/v2/meetings` with `{E2EE:true, duration, hostId}`. The Zoom API returns `{meetingUrl, sessionToken, joinUrl}`, which the backend distributes to both portals. The Patient joins via `joinSession(meetingUrl)` over a live WebSocket connection.

If AI assistance is enabled, the backend calls `POST /api/ai/context {patientId, history[], notes[], goals[]}` on the AIEngine. The engine executes `analyzeContext()`, `processHistory()`, and `generateInsights()` and returns `{recommendations[], sessionSummary}` to the Therapist Dashboard for real-time display alongside the video session.

At session end, the Therapist submits notes via `POST /api/sessions/end {sessionId, notes, duration}`. The RecordsService encrypts the notes using AES-256 and persists them via `db.records.insertOne()`, returning `HTTP 201 {recordId, stored:true}`. The NotificationService then dispatches a post-session summary email to the Patient via SendGrid.

In the alternate flow where the internet connection drops, the WebSocket layer attempts auto-reconnect up to three times before marking the session as `INTERRUPTED` and notifying the Therapist. If the AI module is offline, the session continues in manual mode with the AI flag set to `DISABLED` in the session record.

3.1.3 UC-3: Manage Patient Records and Notes

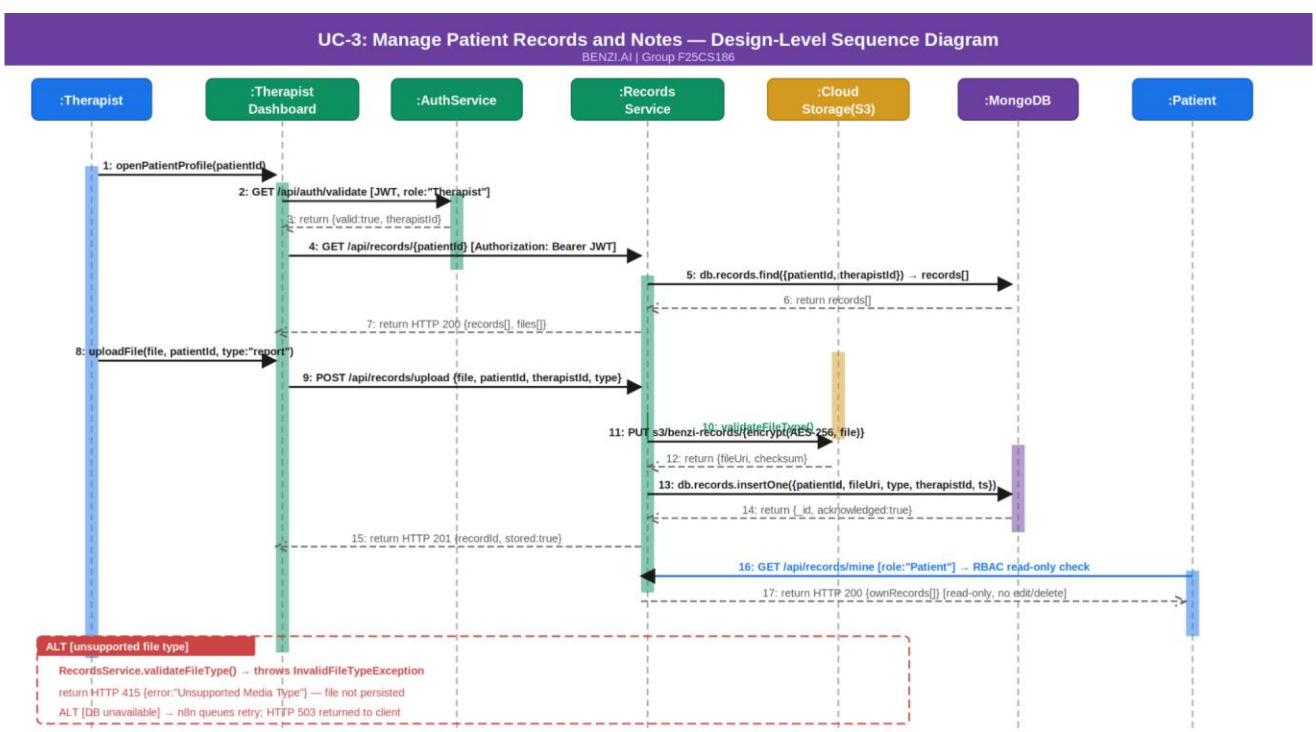


Figure 4: UC-3 Manage Patient Records — Design-Level Sequence Diagram

The records management flow involves seven components: Therapist, TherapistDashboard, AuthService, RecordsService, CloudStorage (AWS S3 / Azure Blob), MongoDB, and Patient. The Therapist opens a patient profile, triggering `GET /api/auth/validate` to confirm `role:"Therapist"`. The dashboard calls `GET /api/records/{patientId}` on the RecordsService, which executes `db.records.find({patientId, therapistId})` and returns the existing records array.

For a file upload, the Therapist submits the file with `POST /api/records/upload {file, patientId, therapistId, type}`. The RecordsService first calls `validateFileType()` — accepted formats are PDF, DOCX, and JPG. On successful validation, the file is encrypted using AES-256 and uploaded to cloud object storage via `PUT s3/benzi-records/{encryptedFile}`, which returns `{fileUri, checksum}`. The file URI and metadata are then persisted in MongoDB via `db.records.insertOne({patientId, fileUri, type, therapistId, timestamp})`. The service returns `HTTP 201 {recordId, stored:true}` to the dashboard.

When the Patient accesses their own records, the RecordsService enforces RBAC — the Patient role is granted read-only access to their own records only, receiving `HTTP 200 {ownRecords[]}` with no edit or delete permissions. Any Admin-role request to the RecordsService returns `HTTP 403 FORBIDDEN` immediately, as the Admin is architecturally blocked from all patient data.

In the alternate flow where an unsupported file type is uploaded, `validateFileType()` throws `InvalidFileTypeException` and the service returns `HTTP 415 {error:"Unsupported Media Type"}` without persisting any data. If the database is unavailable, n8n queues the operation for retry and the client receives `HTTP 503`.

3.1.4 UC-4: Assign and Track Wellness Goals

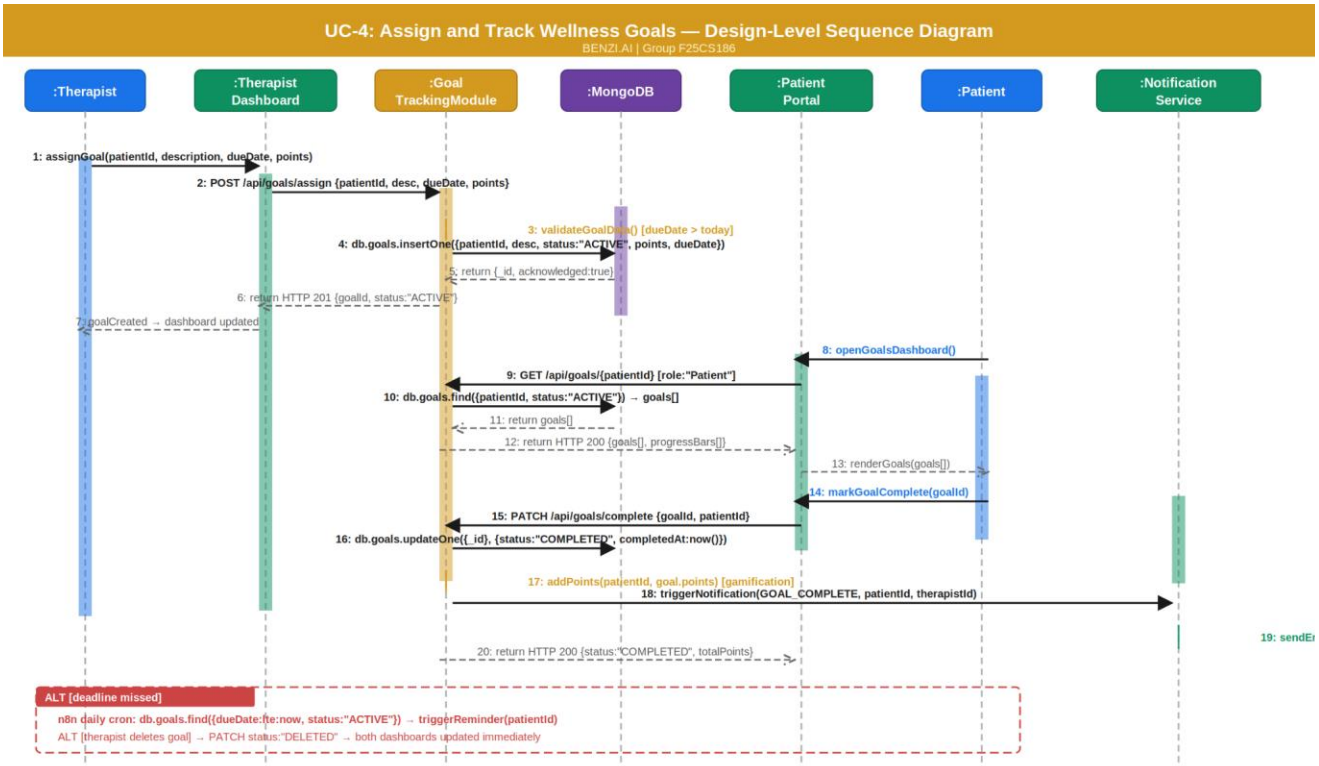


Figure 5: UC-4 Assign and Track Wellness Goals — Design-Level Sequence Diagram

The wellness goal flow involves seven components: Therapist, TherapistDashboard, GoalTrackingModule, MongoDB, PatientPortal, Patient, and NotificationService. The Therapist calls `POST /api/goals/assign {patientId, description, dueDate, points}` from the dashboard. The GoalTrackingModule validates the goal data — specifically confirming that the due date is greater than today — and persists the goal via `db.goals.insertOne({patientId, description, status:"ACTIVE", points, dueDate})`. The dashboard receives `HTTP 201 {goalId, status:"ACTIVE"}`.

On the patient side, opening the goals dashboard triggers `GET /api/goals/{patientId}`, which calls `db.goals.find({patientId, status:"ACTIVE"})` and returns the active goal list with progress indicators rendered as visual progress bars in the Patient Portal.

When the patient marks a goal complete, `PATCH /api/goals/complete {goalId, patientId}` is called. The GoalTrackingModule updates the document via `db.goals.updateOne({_id}, {status:"COMPLETED", completedAt:now()})` and immediately calls `addPoints(patientId, goal.points)` to credit the gamification points counter. The Observer Pattern triggers the NotificationService, which dispatches a congratulatory notification to the Patient and a progress update to the Therapist via email and SMS. The patient receives `HTTP 200 {status:"COMPLETED", totalPoints}`.

In the alternate flow for missed deadlines, a daily n8n cron job queries `db.goals.find({dueDate:{$lte:now}, status:"ACTIVE"})` and triggers `NotificationService.sendReminder(patientId)` for each overdue goal. Goals deleted by the Therapist are updated via `PATCH status:"DELETED"` and both dashboards reflect the change immediately.

3.1.5 UC-5: Interact with AI Virtual Assistant

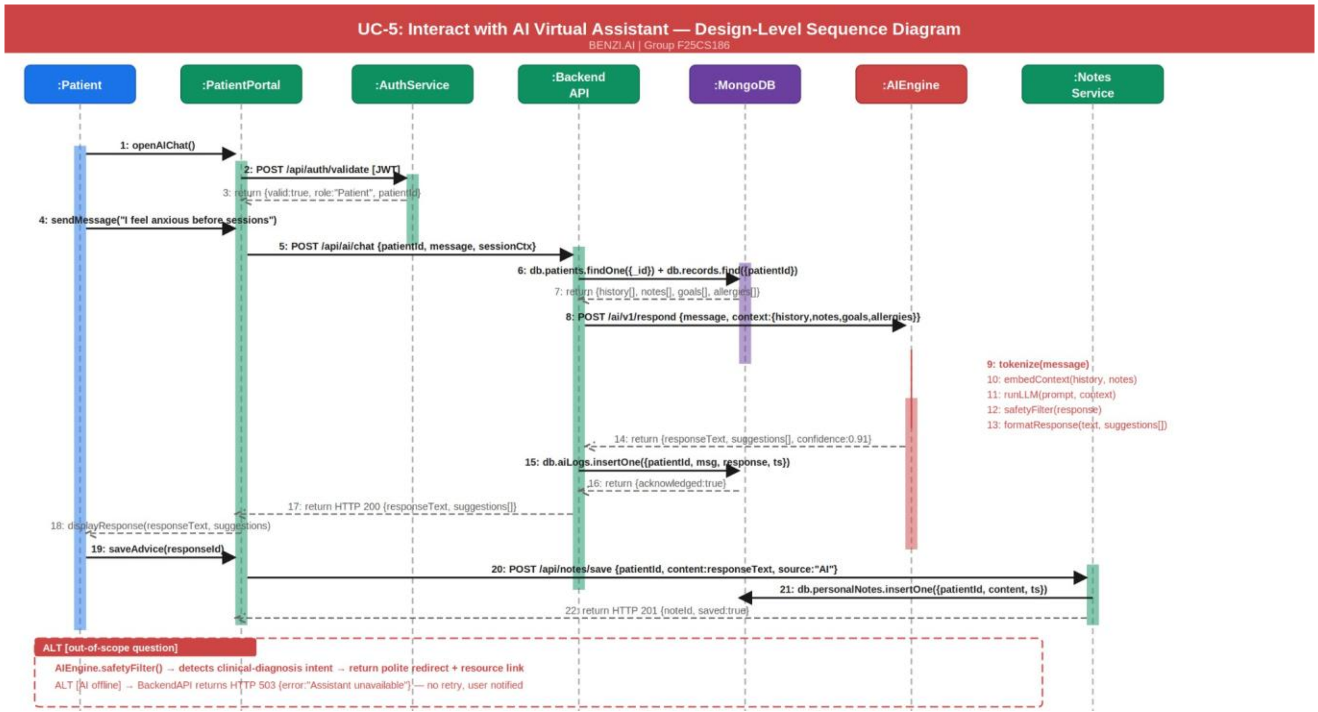


Figure 6: UC-5 AI Virtual Assistant Interaction — Design-Level Sequence Diagram

The AI assistant interaction flow involves seven components: Patient, PatientPortal, AuthService, BackendAPI, MongoDB, AIEngine, and NotesService. The patient opens the AI chat, triggering `POST /api/auth/validate` to confirm `role:"Patient"` and retrieve the `patientId`. The patient submits a free-text message, which the portal sends via `POST /api/ai/chat {patientId, message, sessionCtx}`.

The BackendAPI first assembles the patient context by calling `db.patients.findOne({_id})` combined with `db.records.find({patientId})`, returning `{history[], notes[], goals[], allergies[]}`. This context payload is then sent to the AIEngine via `POST /ai/v1/respond {message, context:{history, notes, goals, allergies}}`.

The AIEngine executes a multi-step processing pipeline: `tokenize(message)` converts the input; `embedContext(history, notes)` builds a contextual embedding; `runLLM(prompt, context)` invokes the language model inference; `safetyFilter(response)` checks the output for harmful or clinically diagnostic content; and `formatResponse(text, suggestions[])` structures the final output. The engine returns `{responseText, suggestions[], confidence:0.91}` to the backend.

The BackendAPI logs the interaction via `db.aiLogs.insertOne({patientId, message, response, timestamp})` before returning `HTTP 200 {responseText, suggestions[]}` to the PatientPortal. The patient optionally saves the advice, triggering `POST /api/notes/save {patientId, content:responseText, source:"AI"}`, which the NotesService persists via `db.personalNotes.insertOne()`.

In the alternate flow where the patient asks a question outside the AI's safe scope — such as requesting a clinical diagnosis — `safetyFilter()` detects the intent and the engine returns a polite redirect response with a resource link rather than attempting to answer. If the AI Engine is offline, the BackendAPI returns `HTTP 503 {error:"Assistant unavailable"}` immediately without retry, and the Patient Portal displays the offline message.

3.1.6UC-6: Manage User Accounts and Subscriptions

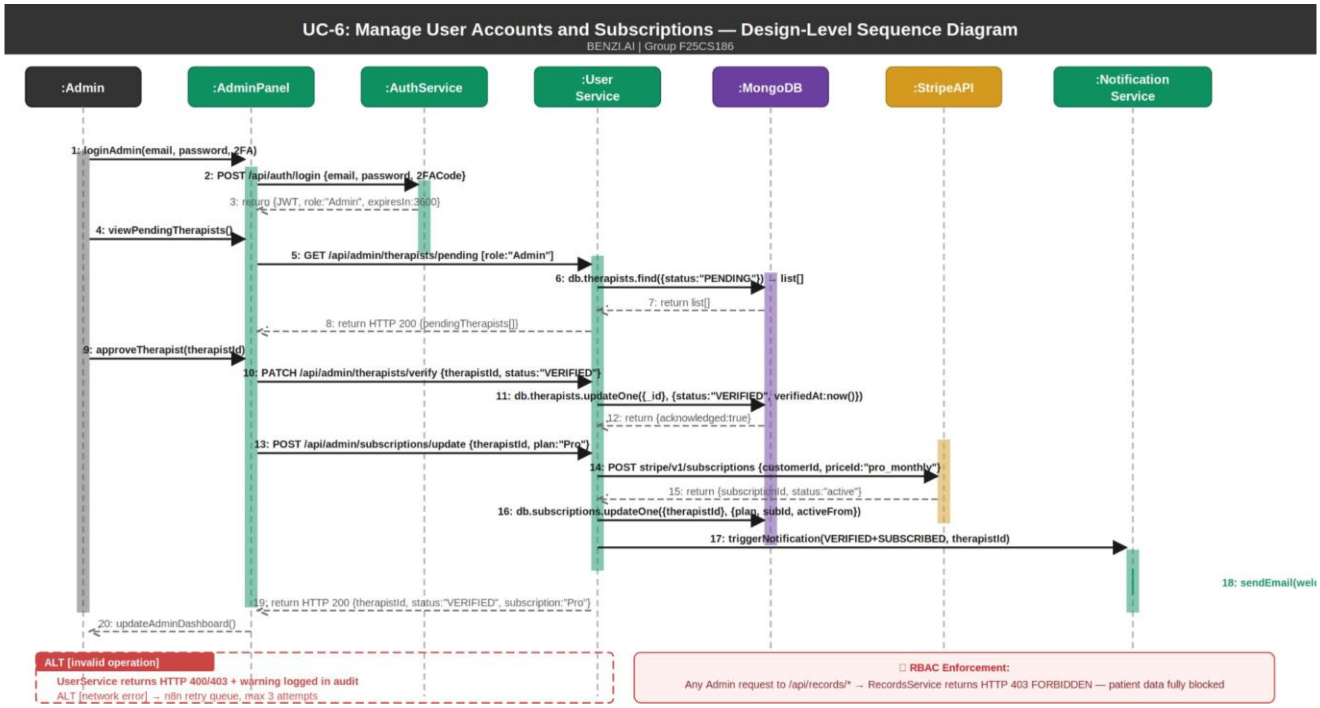


Figure 7: UC-6 Manage Accounts and Subscriptions — Design-Level Sequence Diagram

The account management flow involves seven components: Admin, AdminPanel, AuthService, UserService, MongoDB, StripeAPI, and NotificationService. The Admin logs in via `POST /api/auth/login {email, password, 2FACode}`, receiving `{JWT, role:"Admin", expiresIn:3600}` upon successful two-factor verification.

To view pending therapist registrations, the Admin calls `GET /api/admin/therapists/pending`, which the UserService resolves via `db.therapists.find({status:"PENDING"})`, returning the list to the Admin Panel. The Admin approves a therapist via `PATCH /api/admin/therapists/verify {therapistId, status:"VERIFIED"}`, which the UserService persists using `db.therapists.updateOne({_id}, {status:"VERIFIED", verifiedAt:now()})`.

To update a therapist subscription, the UserService calls Stripe's `POST stripe/v1/subscriptions {customerId, priceId:"pro_monthly"}`, receiving `{subscriptionId, status:"active"}` in return. The subscription record is updated in MongoDB and the NotificationService dispatches a welcome email including the active plan details to the newly verified Therapist.

A critical RBAC enforcement is built into this flow: any Admin-role request directed at `/api/records/*` is intercepted by the RecordsService middleware, which returns `HTTP 403 FORBIDDEN` without processing — the Admin has zero access to patient data by architectural design, not just policy. All invalid Admin operations return `HTTP 400` or `HTTP 403` with the event logged in the system audit trail. Network failures during database or Stripe operations trigger the n8n retry queue with a maximum of three retry attempts.

Component Interaction Summary Table

Insert the following table after the six sequence diagram subsections.

Table 2: Internal Component Interaction Summary

Calling Component	Called Component	Method / Endpoint	Protocol	Returns
PatientPortal	AuthService	POST /api/auth/validate	HTTPS REST	{valid, role, userId}
PatientPortal	AppointmentService	POST /api/appointments/book	HTTPS REST	HTTP 201 {appointmentId}
AppointmentService	MongoDB	db.appointments.insertOne()	MongoDB Driver	{_id, acknowledged}
AppointmentService	StripeAPI	POST stripe/v1/payment_intents	HTTPS REST	{status, paymentId}
AppointmentService	NotificationService	triggerNotification()	Internal Event	void
NotificationService	n8n	scheduleReminder()	HTTP Webhook	{jobId}
TherapistDashboard	AIEngine	POST /api/ai/context	HTTPS REST	{recommendations[]}
BackendAPI	AIEngine	POST /ai/v1/respond	HTTPS REST	{responseText, suggestions[]}
RecordsService	CloudStorage	PUT s3/benzi-records/	AWS SDK	{fileUri, checksum}
RecordsService	MongoDB	db.records.insertOne()	MongoDB Driver	{_id, acknowledged}
GoalTrackingModule	MongoDB	db.goals.updateOne()	MongoDB Driver	{acknowledged}

GoalTrackingModule	NotificationService	triggerNotification()	Internal Event	void
UserService	StripeAPI	POST stripe/v1/subscriptions	HTTPS REST	{subscriptionId, status}
AdminPanel	RecordsService	GET /api/records/*	HTTPS REST	HTTP 403 FORBIDDEN

3.2 Component-External Entities Interface

BENZI.AI communicates with four categories of external systems. All external communications are performed over HTTPS using authenticated API clients with credentials stored securely in environment variables and cloud secret managers (AWS Secrets Manager / Azure Key Vault). No external system has direct access to the BENZI.AI database. All external API calls are made exclusively from the backend Application Layer — the frontend never communicates directly with any third-party service.

External Entity 1: Zoom SDK / Google Meet API (Video Conferencing)

Direction: BENZI.AI Backend → Zoom / Google Meet

Communication: The Appointment Service calls the Zoom REST API or Google Meet API at session start time to dynamically generate an encrypted meeting room. The returned session URL is distributed to the Patient and Therapist through their respective portals. End-to-end encryption of the video session is enforced at the provider level (Zoom E2EE / Google Meet encryption).

Data Exchanged: Session creation request (therapist ID, patient ID, scheduled time, duration) → Encrypted meeting URL and session token returned.

Error Handling: If the API returns an error or timeout, the backend retries once. On second failure, both parties are notified via the Notification Service and the session is flagged for manual rescheduling.

External Entity 2: Stripe API (Payment Processing)

Direction: BENZI.AI Backend → Stripe

Communication: The Appointment Service calls the Stripe API during the online payment path of appointment booking. The backend creates a payment intent, which is processed by Stripe. BENZI.AI never stores raw card data all payment data is handled exclusively by Stripe's PCI-DSS-compliant infrastructure. Webhook callbacks from Stripe notify the backend of payment confirmation or failure.

Data Exchanged: Payment intent creation request (amount, currency, patient reference) → Payment confirmation or failure event returned via webhook.

Error Handling: Payment failure triggers appointment cancellation, patient notified via Notification Service. Network timeout triggers retry; on persistent failure, session flagged as PAYMENT_PENDING.

External Entity 3: Twilio API (SMS Notifications)

Direction: BENZI.AI Notification Service → Twilio

Communication: The Notification Service calls the Twilio REST API to dispatch SMS messages. Triggered events include appointment confirmations, 24-hour session reminders, goal deadline reminders, and system alerts. n8n workflows schedule and manage the timing of all SMS dispatches.

Data Exchanged: SMS dispatch request (recipient phone number, message body) → Delivery status response (DELIVERED / FAILED).

Error Handling: Failed delivery logged in MongoDB audit log. n8n workflow retries failed SMS up to three times with exponential backoff.

External Entity 4: SendGrid API (Email Notifications)

Direction: BENZI.AI Notification Service → SendGrid

Communication: The Notification Service calls the SendGrid API to dispatch HTML-formatted email notifications. Triggered events mirror Twilio triggers and additionally include post-session summaries, AI advice logs (when saved by patient), and subscription receipts.

Data Exchanged: Email dispatch request (recipient address, subject, HTML body, template ID) → Delivery acknowledgement.

Error Handling: Undelivered emails logged in audit system. Bounced emails flagged in patient/therapist profile for admin review.

External Entity Interaction Summary

Table 3.2.1: Component – External Entity Interaction Summary

External System	BENZI.AI Component	Protocol	Purpose	Data Direction
Zoom SDK	Appointment Service	HTTPS REST	Encrypted video session creation	Outbound→Inbound URL
Google Meet API	Appointment Service	HTTPS REST	Encrypted video session creation	Outbound→ Inbound URL
Stripe API	Appointment Service	HTTPS REST+ Webhook	Payment processing	Outbound→ Inbound event

Twilio API	Notification Service	HTTPS REST	SMS dispatch	Outbound→ Status
SendGrid API	Notification Service	HTTPS REST	Email dispatch	Outbound→ Status
AWS KMS / Azure Key Vault	Records Service	SDK	Encryption key management	Bidirectional
AWS S3 / Azure Blob	Records Service	SDK	Encrypted file storage	Outbound→ Inbound URI

3.3 Component – Human Interface

This subsection identifies all screens and interaction points through which users provide input to or receive output from the BENZI.AI system. The interface is entirely browser-based (no thick client required) and is accessible via Chrome, Firefox, Edge, and Safari. The design follows Human-Computer Interaction (HCI) principles including clarity, minimal cognitive load, role-appropriate information presentation, and accessibility-conscious design consistent with WCAG guidelines.

HCI Principles Followed

Role-Based Interface Segregation: Each user role (Patient, Therapist, Admin) accesses a completely distinct portal with only the functions relevant to their role visible. No role sees interface elements belonging to another role, reducing confusion and enforcing the principle of least privilege at the UI level.

Minimalist and Calm Design: Given the mental health context of the application, the UI employs a minimalistic design language with soft color palettes, generous whitespace, and simple navigation. This is intentional to reduce visual fatigue and anxiety for users who may already be in emotionally vulnerable states.

Progressive Disclosure: Complex features (AI assistant configuration, subscription management, analytics) are not surfaced immediately. Users are progressively guided to advanced features only after completing primary actions, preventing overwhelm on first use.

Feedback and Confirmation: All destructive or significant actions (appointment cancellation, goal deletion, account suspension) require explicit confirmation dialogs before execution. Success and error states are communicated with clear, non-technical messages.

Accessibility: Font sizes meet minimum readability standards. Contrast ratios between text and background are maintained at WCAG AA level. All interactive elements are keyboard-navigable.

3.3 Component-Human Interface

BENZI.AI provides a fully browser-based graphical user interface accessible through all major modern browsers without any client-side installation. The interface is organized into three role-specific portal environments Patient Portal, Therapist Portal, and Public/Guest Interface each presenting only the screens and actions relevant to that role. The following HCI design norms are consistently followed throughout all screens.

HCI Principles Applied:

Role-based interface segregation ensures no user sees controls outside their permissions. A persistent right-side navigation panel on all authenticated screens provides one-click access to all sections without page reloads. Soft teal and off-white color palette is intentionally calming for the mental health context. All data tables include keyword search and pagination for managing large record sets. Destructive actions (delete, cancel) require confirmation. Form fields use real-time inline validation. Progress and statistics are communicated through visual charts and circular progress indicators rather than raw numbers alone.

The following table lists all screens identified from the actual implemented prototype.

Table 3.1: Screen Inventory — Input/Output and Component Mapping

#	Screen Name	User Role	Input Received	Output Provided	Backend Component
1	Home	Public / Guest	Navigation clicks, newsletter email subscription	Platform overview, care offerings (NLP, Emotion Recognition, Personalized Recommendations, Risk Assessment, 24/7 Availability, Continuous Learning), subscription plan tiers, CTA buttons	None (static + SendGrid)
2	Auth — Role Selection	Public / Guest	Role card selection (Register As Therapist / Register As User)	Redirects to role-specific Register or Login screen	AuthService

3	**Login**	**All Roles**	**Email / username, password, Remember Me checkbox**	**JWT session token, role-based dashboard redirect, Forgot Password link**	**AuthService**
4	**Register**	**Patient / Therapist**	**Full name, email, password, role selection; Therapist additionally submits credentials**	**Account creation confirmation, email verification link dispatched**	**AuthService, SendGrid**
5	**Patient — Dashboard**	**Patient**	**Mood selection (Happy / Good / Normal / Bad / Awful), Submit button**	**Task score counter, mood log confirmation, Current Progress rings (Mental Health / Self Care / Therapy), weekly Task Progress bar chart, Overall Report annual line chart**	**RecordsService, GoalTrackingModule**
6	**Patient — Conversations (AI Chat)**	**Patient**	**Free-text message, voice input button**	**BENZI AI response bubbles, chat history panel (Today / Yesterday / Previous), New Chat button, search, save option**	**AIEngine, NotesService**

7	**Patient — Goals**	**Patient**	**Goal preset selection (Enhance Sleep Quality / Improve Stress Management / Improve Communication Skills), current status sliders (0–100), custom goal text input, Submit**	**Active goals with progress percentage, Completed / In Progress / Pending pie chart (70% / 20% / 10%), Community Reviews sentiment (Negative / Neutral / Positive), AI Insights & Recommendations feed**	**GoalTrackingModule, AIEngine**
8	**Patient — Progress**	**Patient**	**Date range filter, Monthly / Annual toggle**	**Individual goal progress bars (Stress / Anxiety / Depression / Improve Sleep at 100% / 50% / 30% / 10%), Overall Goal Progress area chart (Jan–May), Benzi Usage hours bar chart, Chatbot Usage weekly bar chart, Overall Report line chart**	**RecommendationEngine, MongoDB**
9	**Patient — Appointment**	**Patient**	**+ Appointment button, keyword search, per-row action icons**	**Appointments table (ID, Therapist, Date & Time, Duration, Location, Status: Pending / Completed), Video Call link generation panel, Clinic / Hospital address generation panel, paginated list**	**AppointmentService, ZoomAPI**

10	**Patient — Reports**	**Patient**	**Keyword search, Download action, Feedback action, per-row menu**	**Reports table (Report ID, Therapist, Date & Time, PDF File, Location, Status: Reviewed / Not review / Half review), Task generation section, Notes / Guidance generation section**	**RecordsService, MongoDB**
11	**Patient — Profile**	**Patient**	**Profile photo upload, Full Name, Email, Password change (old / new / confirm)**	**Updated profile confirmation, password change confirmation**	**AuthService, MongoDB**
12	**Patient — Help & Support**	**Patient**	**Support query input, topic selection**	**Help articles, support ticket submission, response confirmation**	**NotificationService, SendGrid**
13	**Subscription (Public)**	**Public / Guest**	**Plan selection (Basic $100 / Standard $200 / Premium $300), Monthly / Annual billing toggle, Get Started button**	**Plan comparison cards with feature lists, redirect to payment / registration**	**StripeAPI**

14	**Therapist — Dashboard**	**Therapist**	**Navigation clicks, calendar date selection**	**KPI cards: Active Services (12, +4.2%), New Services (2, −1.2%), Avg Reviews (20, +3.4%), Avg Reply Time (15 min, +3.4%); Today's Appointments card with patient name and session topic; monthly calendar view; Most Bought Package donut charts (Stress Management 60%, Career Counseling 75%, Couples Counseling); Generated Revenue horizontal bar chart (Aug $1000 / Sep $700 / Oct $800)**	**AppointmentService, MongoDB**
15	**Therapist — Appointment**	**Therapist**	**Keyword search, + Calendar button, per-row action icons**	**Appointments table (Appointment ID, Patient, Date & Time, Duration, Location, Status: Confirmed / Pending / Cancelled), paginated list**	**AppointmentService, MongoDB**
16	**Therapist — Clients**	**Therapist**	**+ New Client button, keyword search, per-row messaging and action icons**	**Clients table (Appointment ID, Patient name, Date & Time, Duration, Location, Status: Confirmed / Pending /**	**RecordsService, MongoDB**

				Cancelled), paginated list	
17	**Therapist — Services**	**Therapist**	**+ New Service button, Edit / Delete per service card**	**Service cards displaying: service name, type, price per 60-minute session, description (e.g. Mindful Counseling — Individual Therapy $100/60min; Career Counseling $120/60min; Couple Counseling $120/60min)**	**MongoDB**
18	**Therapist — Subscription**	**Therapist**	**Plan selection, upgrade / downgrade action**	**Active subscription plan details, plan comparison, Stripe payment redirect**	**StripeAPI, MongoDB**
19	**Therapist — Payments**	**Therapist**	**Date range filter**	**Payment history list, revenue summary by period**	**StripeAPI, MongoDB**

20	**Therapist — Profile**	**Therapist**	**Full Name, WhatsApp, Email (Personal Information section); Specialization, Qualification, Practice, Experience, Session count, Client count, Bio / Message (Professional Information section); Old Password, New Password, Confirm Password (Change Password section); Change Photo**	**Updated profile confirmation, password change confirmation**	**AuthService, MongoDB**
21	**Therapist — Profile Setup**	**Therapist**	**Initial onboarding fields (name, credentials, specialization)**	**Profile activation, onboarding completion confirmation**	**AuthService, MongoDB**

22	**Meditation Counselor (Public)**	**Public / Guest**	**Navigation clicks, See Our Benefits / See more buttons**	**AI approach explanation ("Your journey begins with an initial consultation"), How it Works 6-step process, feature highlights (Risk Assessment, Emotion Recognition, 24/7 Availability), Interactive Therapy Modules section, AI feedback & learning description**	**None (static)**
23	**Resources**	**Public / Guest**	**Navigation / category filter clicks**	**Mental health resource articles and materials library**	**None (static / CMS)**
24	**About Us**	**Public / Guest**	**Navigation click**	**Team information, mission statement, company background**	**None (static)**
25	**About BENZI.AI**	**Public / Guest**	**Navigation click**	**Product overview, brand values, platform description**	**None (static)**
26	**FAQs**	**Public / Guest**	**Navigation click, question expansion**	**Frequently asked questions list with expandable answers**	**None (static)**

27	**Contact Us**	**Public / Guest**	**Full name, email address, message text, Submit**	**Support ticket creation, submission confirmation message dispatched**	**NotificationService, SendGrid**
28	**Admin — Dashboard**	**Admin**	**Navigation clicks, date range filter (7 days / 30 days / 12 months)**	**KPI cards: Total Doctors (48, +3.2%), Total Patients (1,240, +5.1%), Monthly Revenue ($24,800, +4.2%), Active Subscriptions (36, +1.8%); Most Sold Packages donut charts (Standard Plan 60%, Pro Plan 75%, Enterprise Plan 45%); Revenue Overview 12-month line chart; Today's Doctor Appointments card with calendar**	**UserService, MongoDB, StripeAPI**
29	**Admin — Doctors**	**Admin**	**Keyword search, + Add Doctor button, per-row actions (Edit / View Profile / Send Credentials / Delete)**	**Doctors table: Doctor ID, Name, Specialization, Patients count, Subscription plan (Pro / Standard / Enterprise), Status (Active / Pending / Inactive), action menu; paginated list of 6 doctors per page**	**UserService, MongoDB**

30	**Admin — Patients**	**Admin**	**Keyword search, view (eye icon) and menu actions per row**	**Patients table grouped by doctor (Doctor Name, Specialization, Total Patients, Active count, Inactive count, Last Session date); Patient Distribution donut chart (Mental Health 40% / Self Care 35% / Therapy 25%); New Patients This Month daily bar chart**	**UserService, MongoDB**
31	**Admin — Subscriptions**	**Admin**	**+ New Plan button, Edit / Delete per plan card, Manage per assignment row**	**Plan cards: Standard ($299/mo, 36 active doctors), Pro ($499/mo, Most Popular, 12 active), Enterprise ($799/mo, 8 active) with full feature lists; Subscription Assignments table (Doctor ID, Name, Plan, Start Date, Expiry Date, Status: Active / Pending / Inactive)**	**StripeAPI, MongoDB**

32	**Admin — Revenue**	**Admin**	**Date range filter, per-payment actions (Edit / Delete)**	**Revenue KPI cards: Total Revenue ($124,800, +8.3%), This Month ($24,800, +4.2%), Pending Payouts ($3,200, −1.1%); Payments table (Payment ID, Doctor, Patient, Date, Service, Amount, Status: Completed / Pending); Monthly Revenue horizontal bar chart; Revenue by Plan donut chart (Standard / Pro / Enterprise)**	**StripeAPI, MongoDB**
33	**Admin — Send Credentials**	**Admin**	**First Name, Last Name, Email, Phone No, Specialization, Subscription Plan, Temporary Password (manual or Auto-generate), Welcome Message text, Send Credentials button**	**Credential dispatch confirmation; Recently Sent log showing doctor avatar, name, email, timestamp and delivery status (Sent ✓ / Failed ✗)**	**AuthService, NotificationService, SendGrid**
34	**Admin — Send Credentials (Nav Active State)**	**Admin**	**Same as #33 — Send Credentials nav item highlighted**	**Same output as #33 — shows active navigation state for Send Credentials section**	**AuthService, NotificationService, SendGrid**

35	**Admin — Profile**	**Admin**	**(In development — no input fields active)**	**Placeholder message: "Admin profile settings coming soon."**	**AuthService**
36	**Admin — About Benzi**	**Admin**	**Navigation click**	**Platform description: "Benzi is a mental wellness platform connecting patients and therapists."**	**None (static)**

4. Screenshots/Prototype

4.1 Workflow

The BENZI.AI system operates across three parallel user journeys that converge at the AI Engine and database layer. The Public/Guest user lands on the Home screen and navigates to the Auth screen to select their role. A Patient registers, logs in, and accesses the Patient Portal where they can track mood, book appointments, chat with the BENZI AI assistant, view goals assigned by their therapist, monitor progress analytics, and download session reports. A Therapist registers with credentials, awaits admin verification, then accesses the Therapist Portal where they manage their service listings, view and confirm appointments, monitor their client list, and track their generated revenue. All authenticated interactions mood submissions, AI chat messages, appointment bookings, report downloads pass through the Node.js backend to MongoDB and, where applicable, to the AI Engine or third-party APIs. The n8n automation engine runs in the background managing reminders, notifications, and analytics workflows.

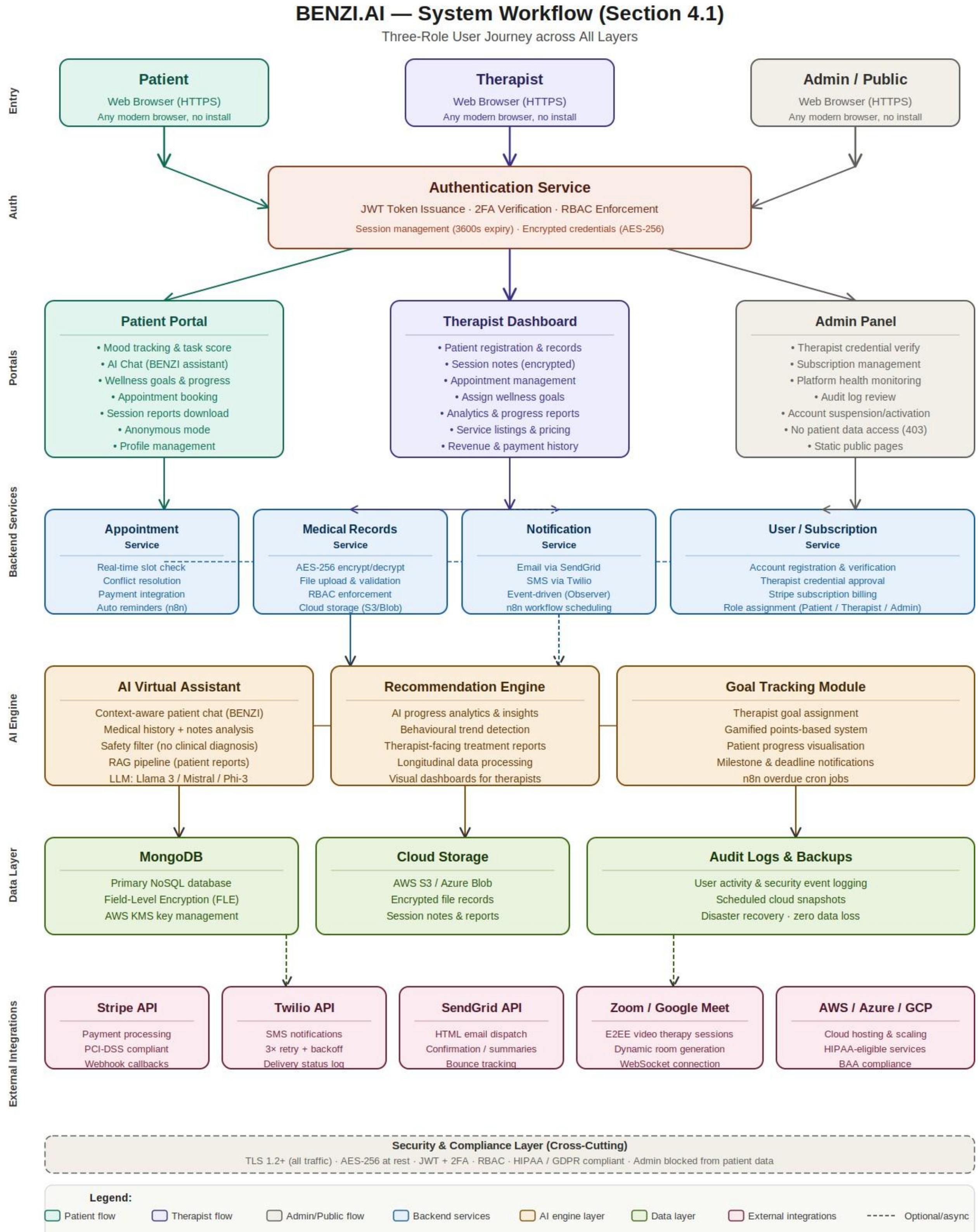
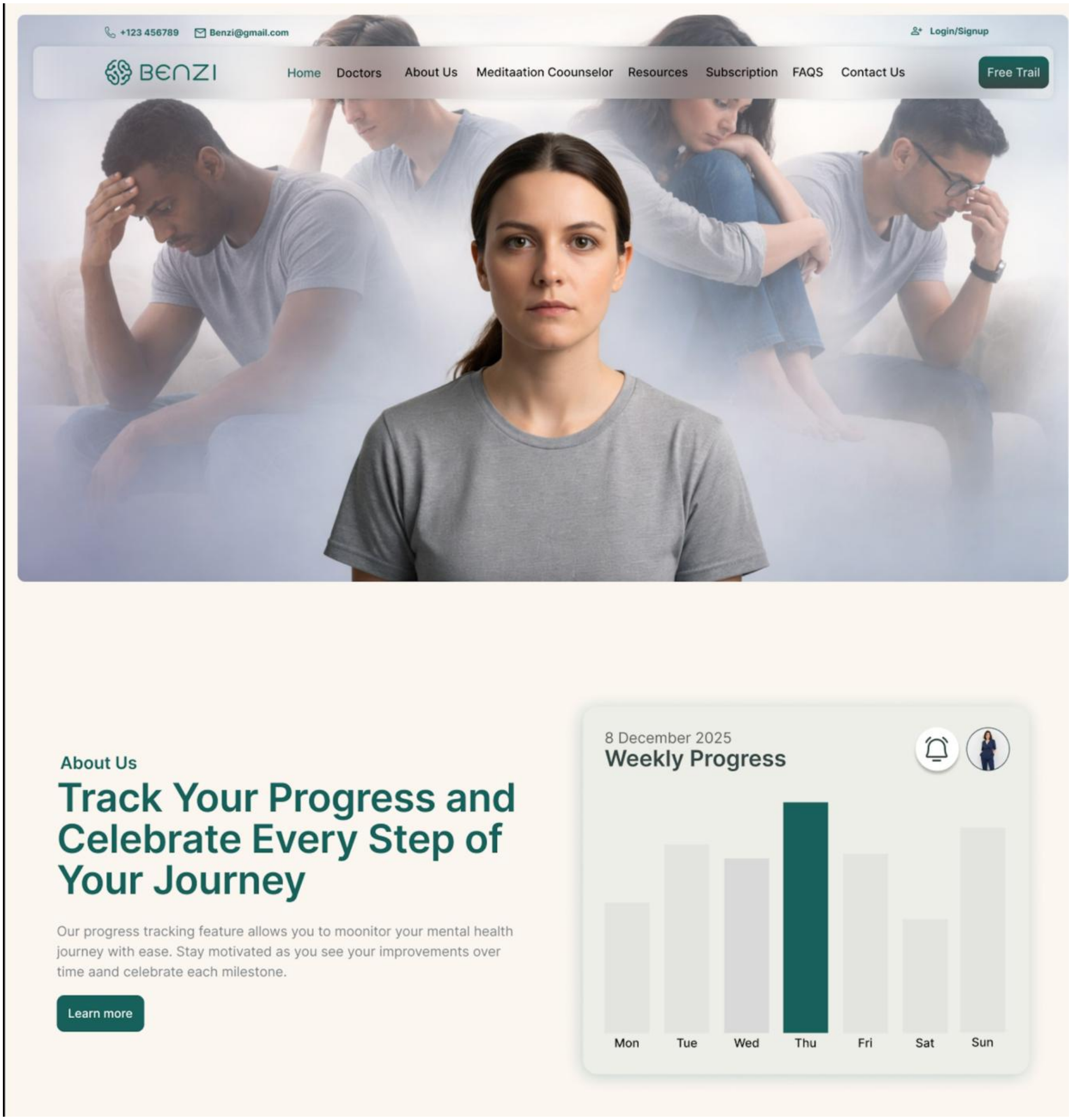
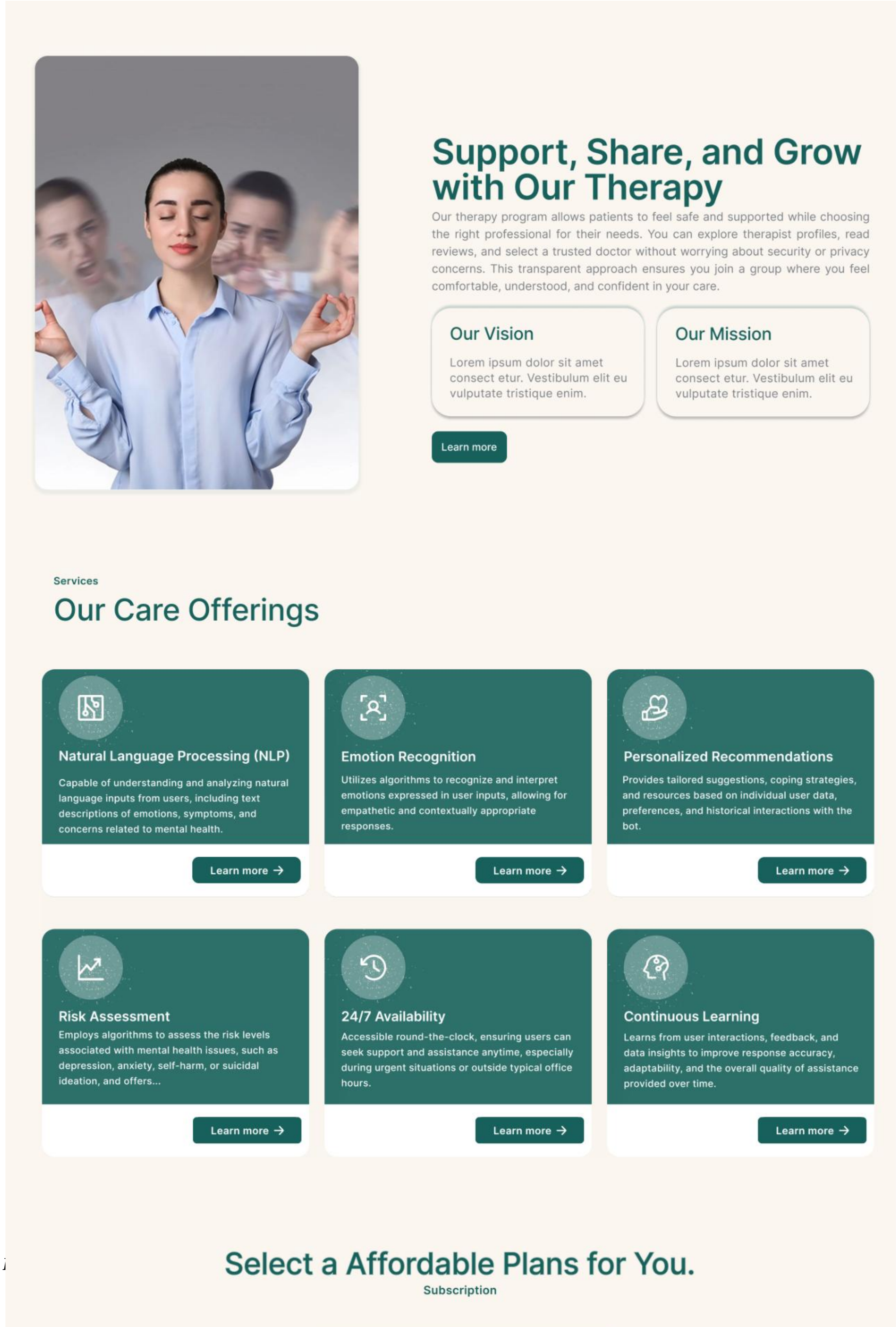


Figure 4.1: BENZI.AI System Workflow — Three-Role User Journey across All Layers | Group F25CS186

4.2 Screens


+123 456789
Benzi@gmail.com
Login/Signup
BENZI
Home
Doctors
About Us
Meditaation Coounselor
Resources
Subscription
FAQS
Contact Us
Free Trail
About Us
Track Your Progress and Celebrate Every Step of Your Journey
Our progress tracking feature allows you to moonitor your mental health journey with ease. Stay motivated as you see your improvements over time aand celebrate each milestone.
Learn more
8 December 2025
Weekly Progress
Mon
Tue
Wed
Thu
Fri
Sat
Sun


Support, Share, and Grow with Our Therapy
Our therapy program allows patients to feel safe and supported while choosing the right professional for their needs. You can explore therapist profiles, read reviews, and select a trusted doctor without worrying about security or privacy concerns. This transparent approach ensures you join a group where you feel comfortable, understood, and confident in your care.
Our Vision
Lorem ipsum dolor sit amet consect etur. Vestibulum elit eu vulputate tristique enim.
Our Mission
Lorem ipsum dolor sit amet consect etur. Vestibulum elit eu vulputate tristique enim.
Learn more
Services
Our Care Offerings
Natural Language Processing (NLP)
Capable of understanding and analyzing natural language inputs from users, including text descriptions of emotions, symptoms, and concerns related to mental health.
Learn more
Emotion Recognition
Utilizes algorithms to recognize and interpret emotions expressed in user inputs, allowing for empathetic and contextually appropriate responses.
Learn more
Personalized Recommendations
Provides tailored suggestions, coping strategies, and resources based on individual user data, preferences, and historical interactions with the bot.
Learn more
Risk Assessment
Employs algorithms to assess the risk levels associated with mental health issues, such as depression, anxiety, self-harm, or suicidal ideation, and offers...
Learn more
24/7 Availability
Accessible round-the-clock, ensuring users can seek support and assistance anytime, especially during urgent situations or outside typical office hours.
Learn more
Continuous Learning
Learns from user interactions, feedback, and data insights to improve response accuracy, adaptability, and the overall quality of assistance provided over time.
Learn more
Select a Affordable Plans for You.
Subscription

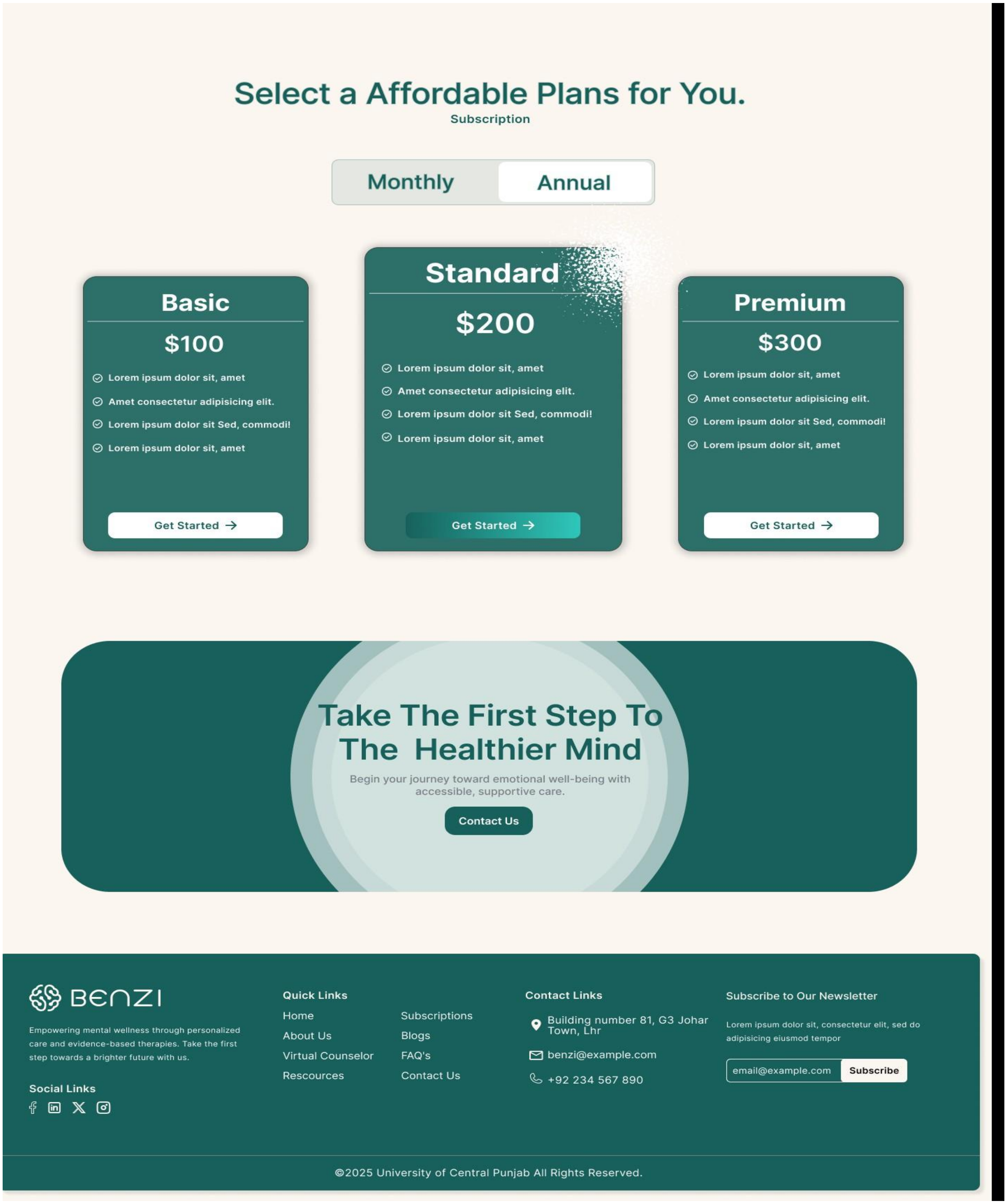


Figure 4.1: Home Page Platform landing page with care offerings and subscription plans

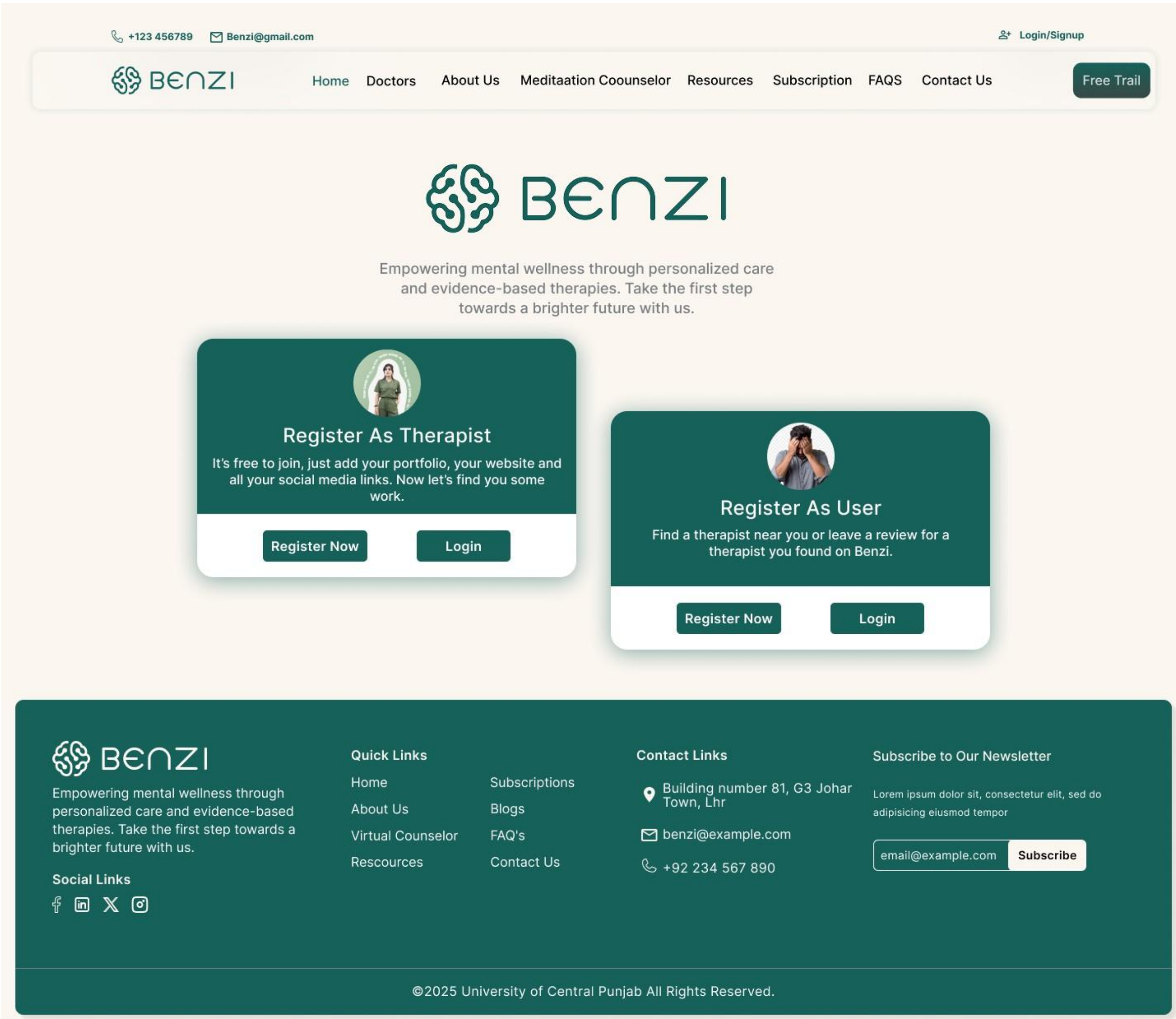


Figure 4.2: Auth Screen Role selection for Therapist or Patient registration/login

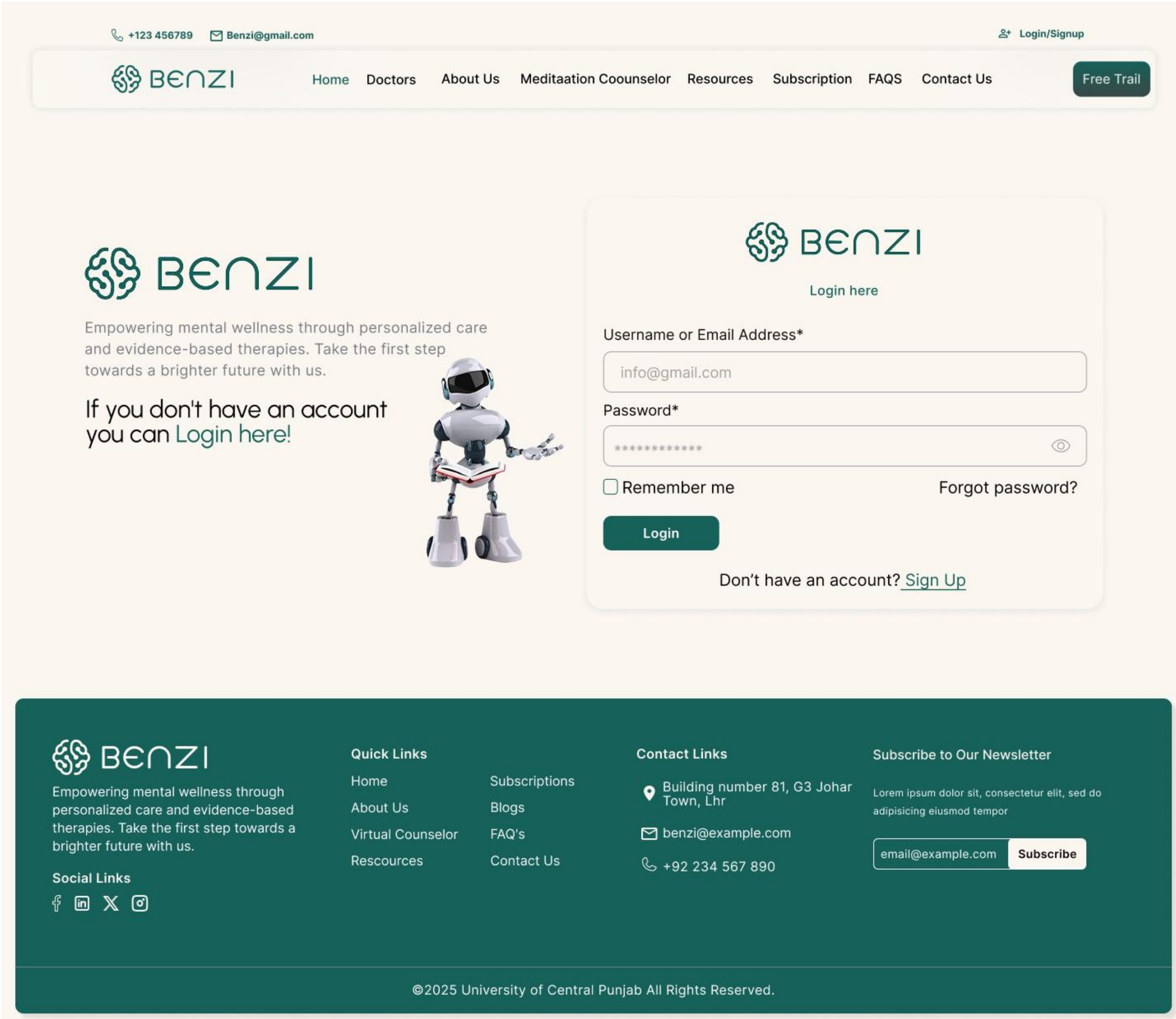


Figure 4.3: Login Screen Credential entry with Remember Me and Forgot Password

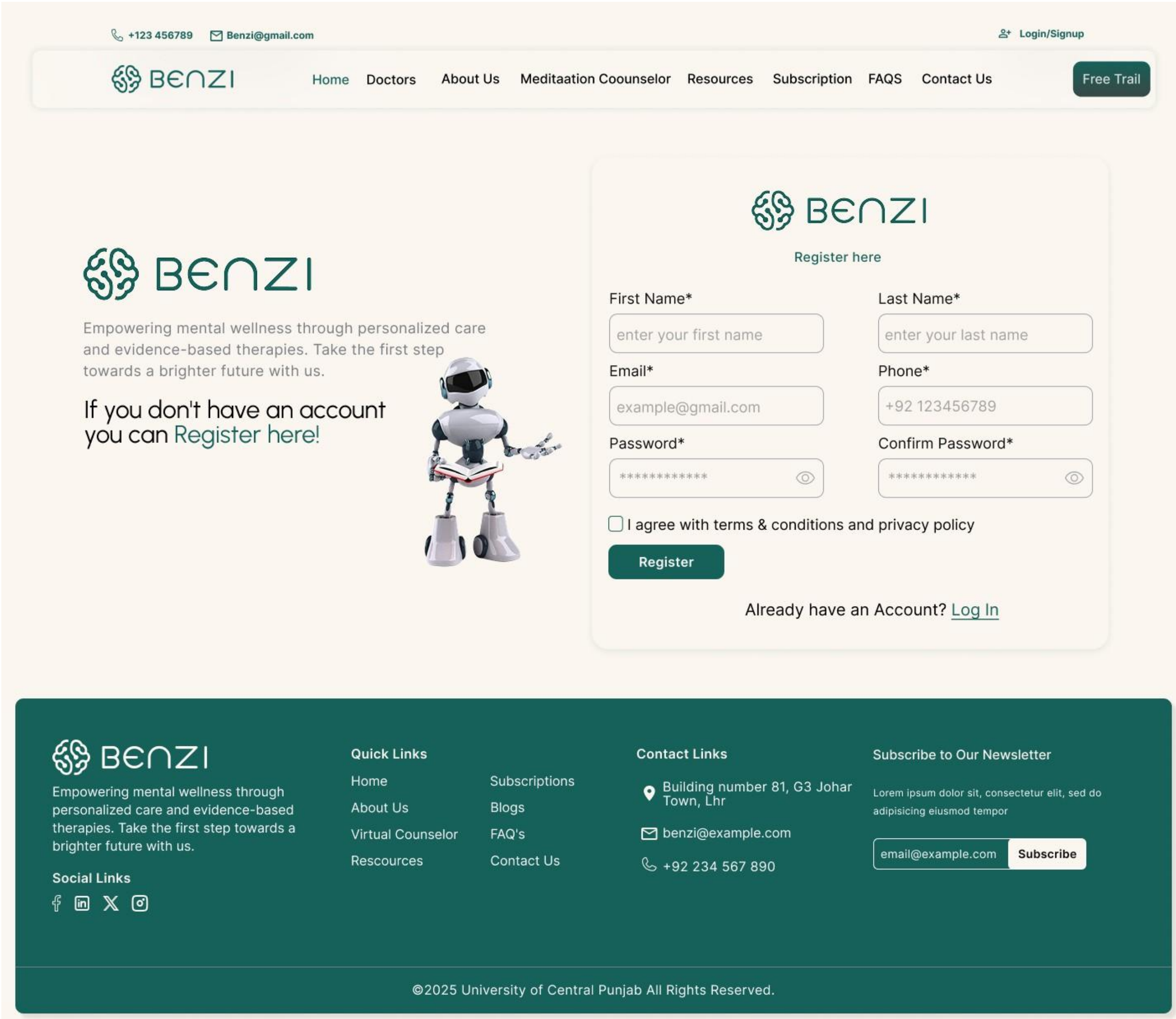


Figure 4.4: Register Screen New user account creation

Figure 4.5: Patient Dashboard Mood tracker, task score, progress rings and weekly chart

Figure 4.6: Patient Conversations BENZI AI chatbot interface with history panel

Figure 4.7: Patient Goals — Goal selection, status sliders, completion pie chart and AI insights

Figure 4.8: Patient Progress — Individual goal bars, Benzi usage, chatbot activity charts

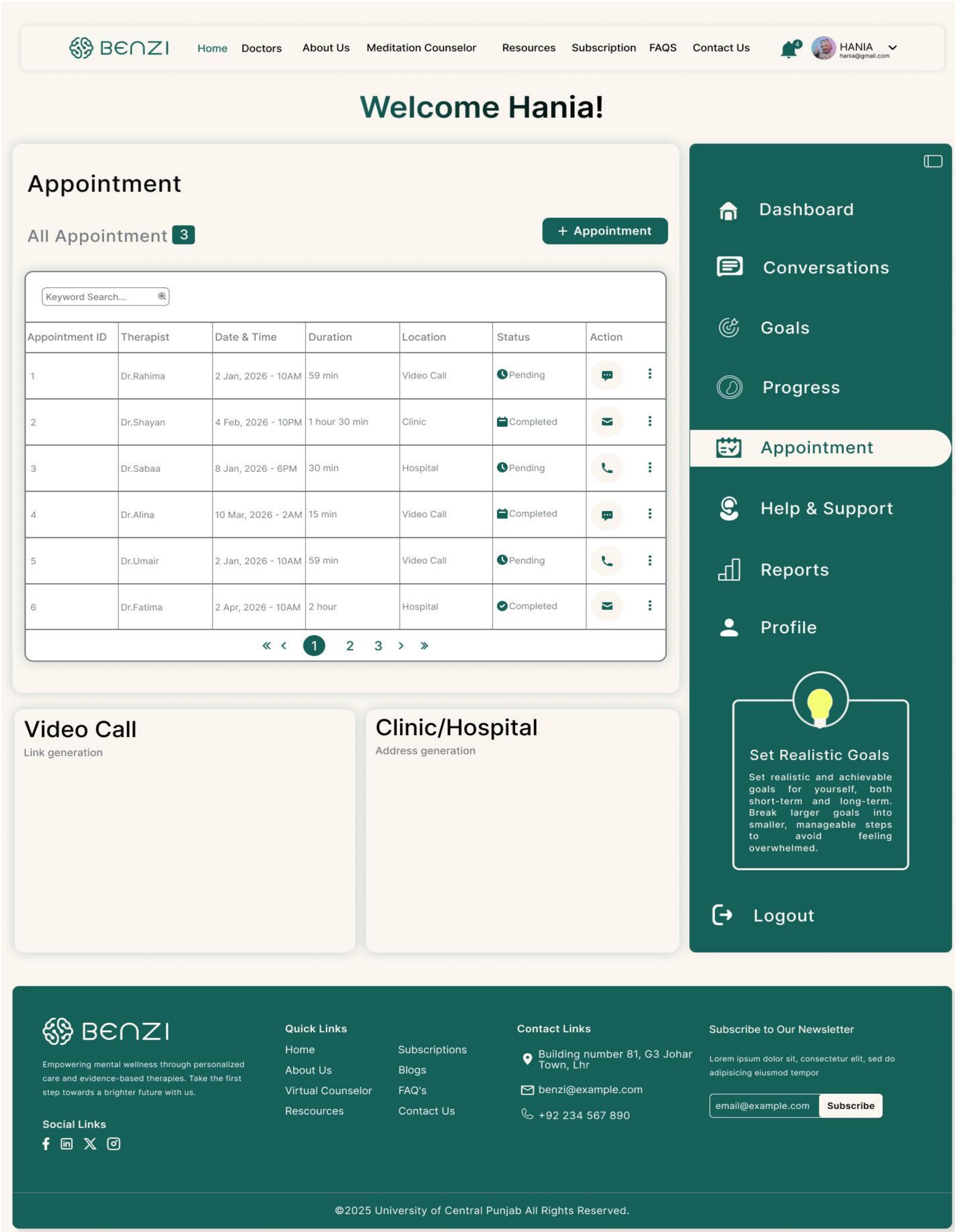


Figure 4.9: Patient Appointments — Appointment table with video call and clinic link generation

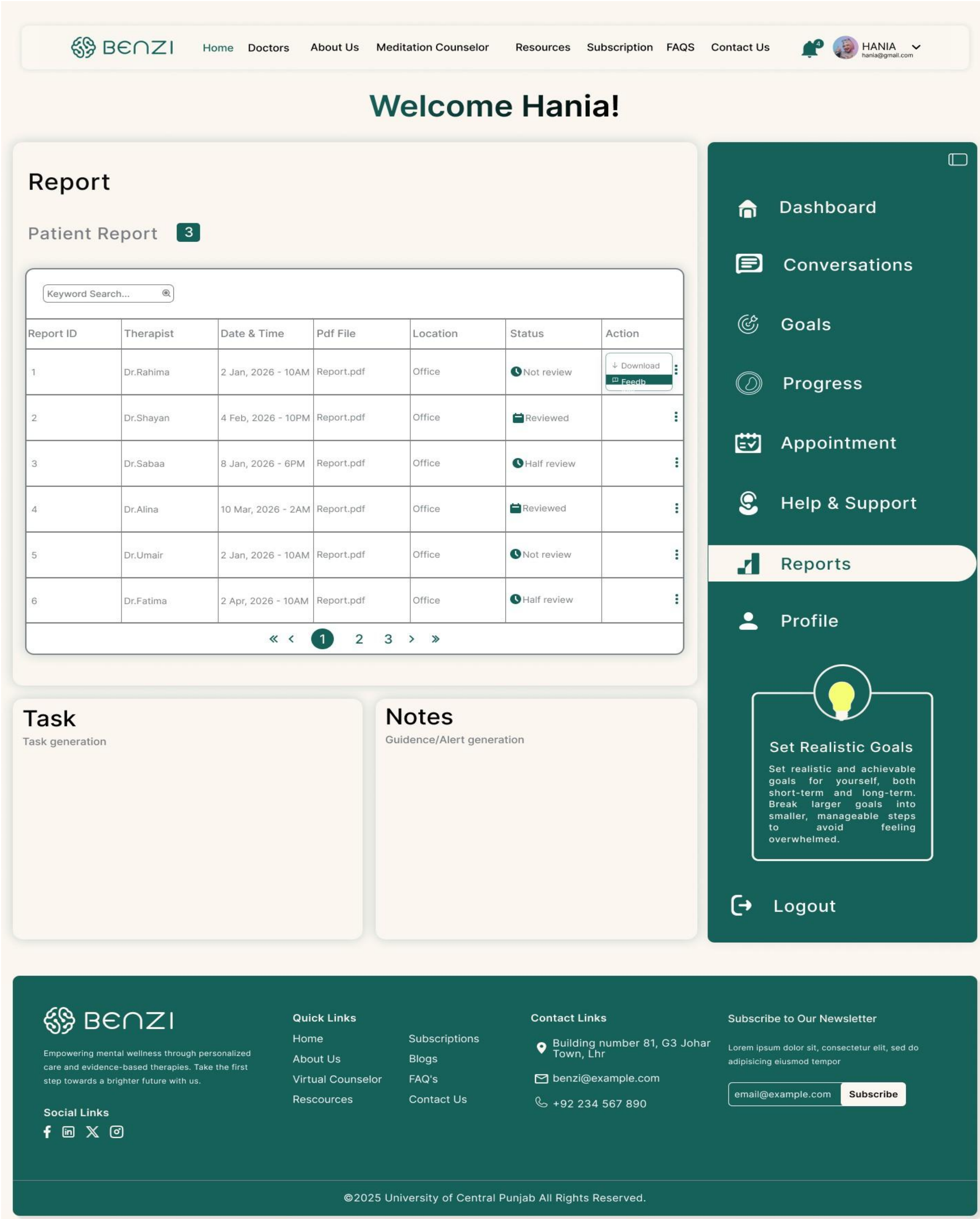


Figure 4.10: Patient Reports Report table with download, feedback, task and notes sections

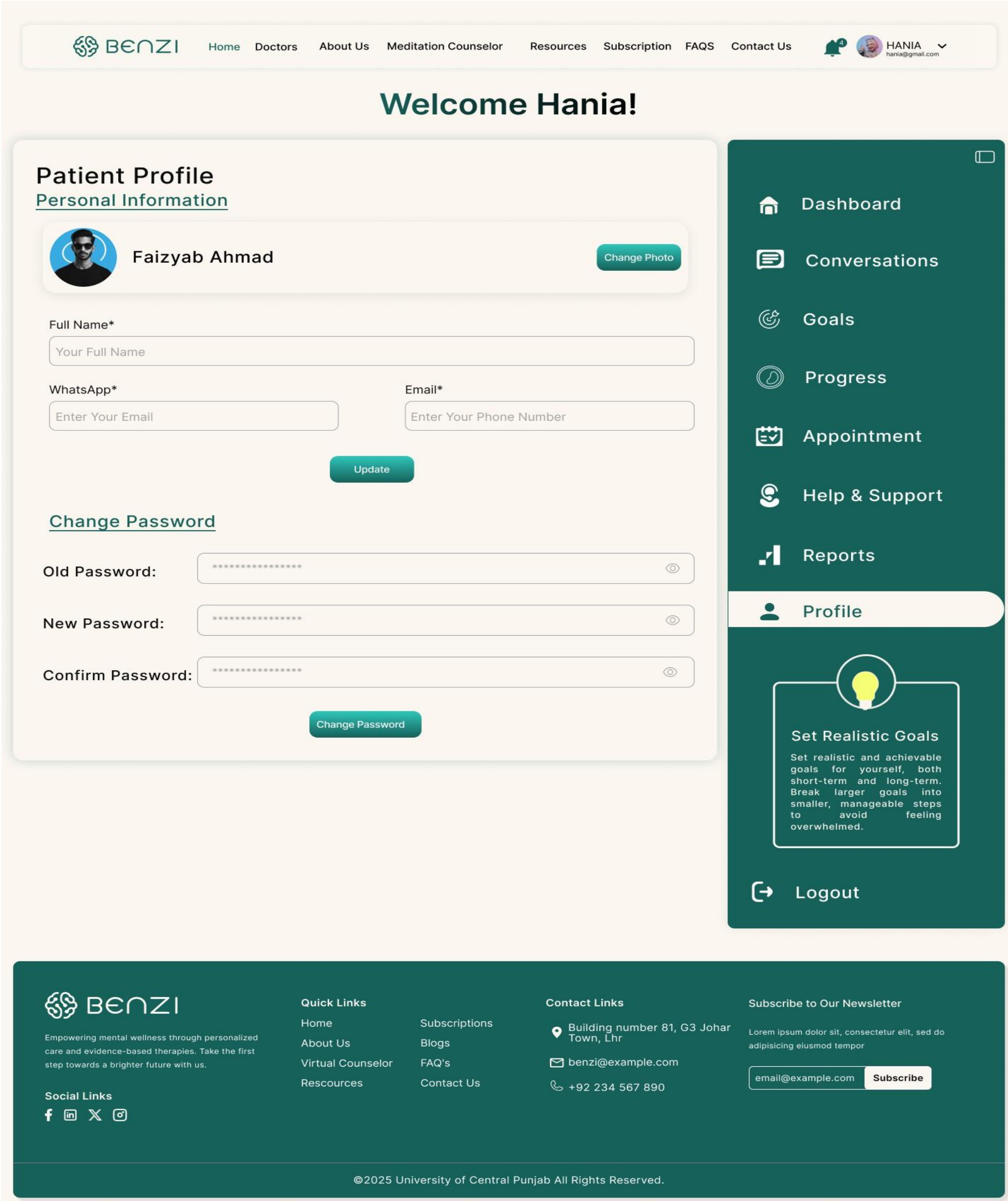


Figure 4.11: Patient Profile — Personal info and password management

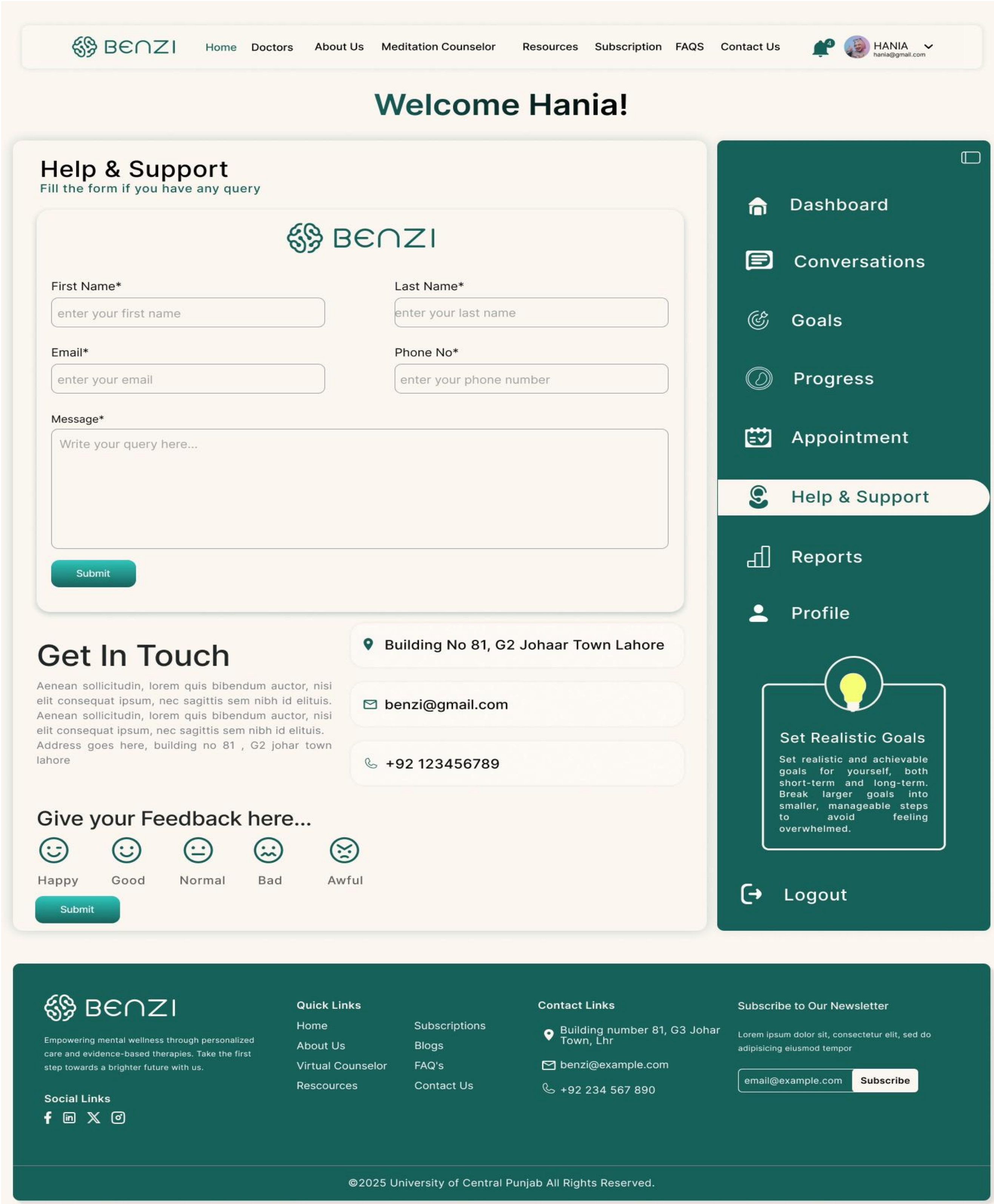


Figure 4.12: Patient Help & Support — Support access screen

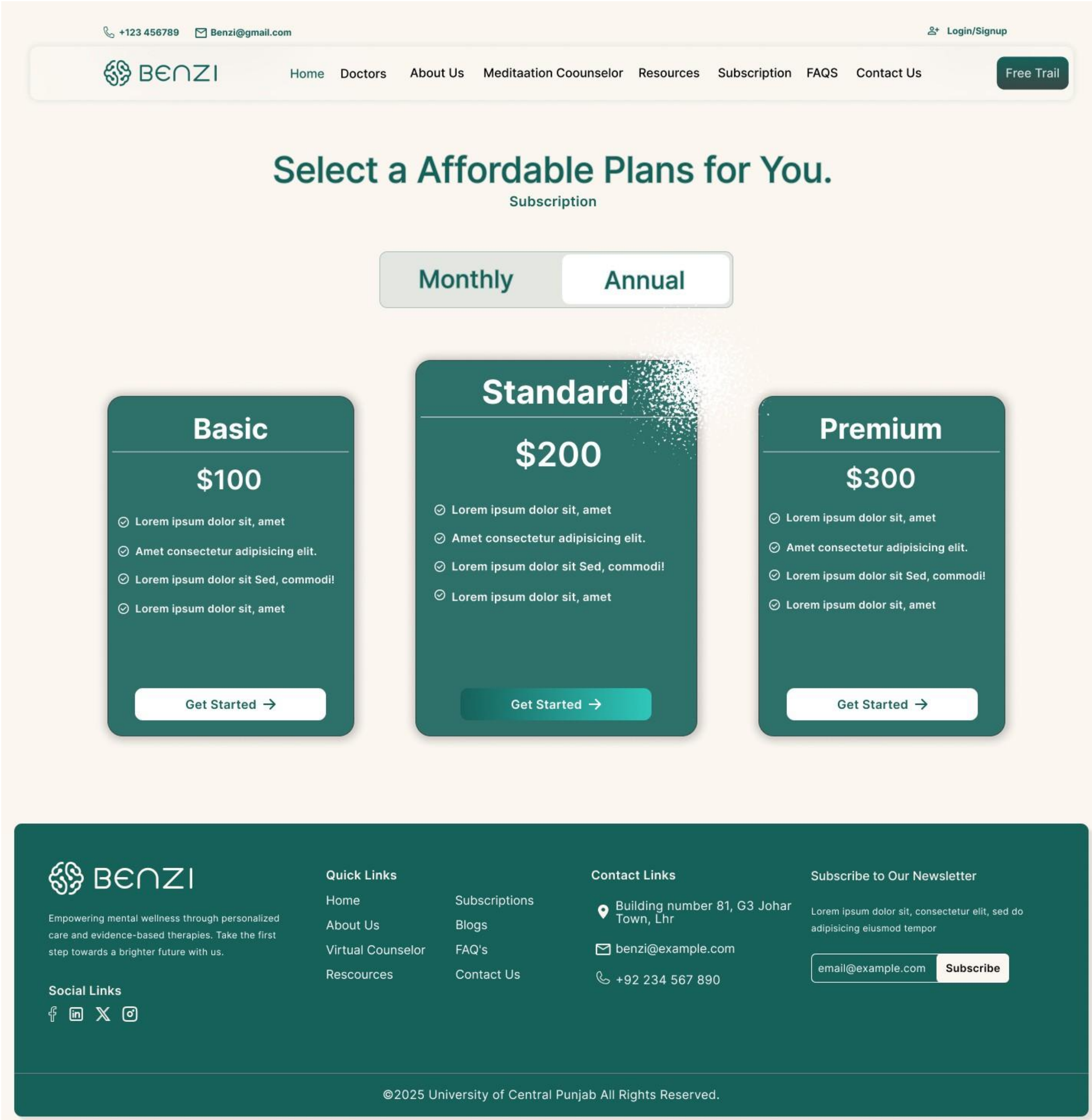


Figure 4.13: Subscription Plans Basic/Standard/Premium with Monthly/Annual toggle

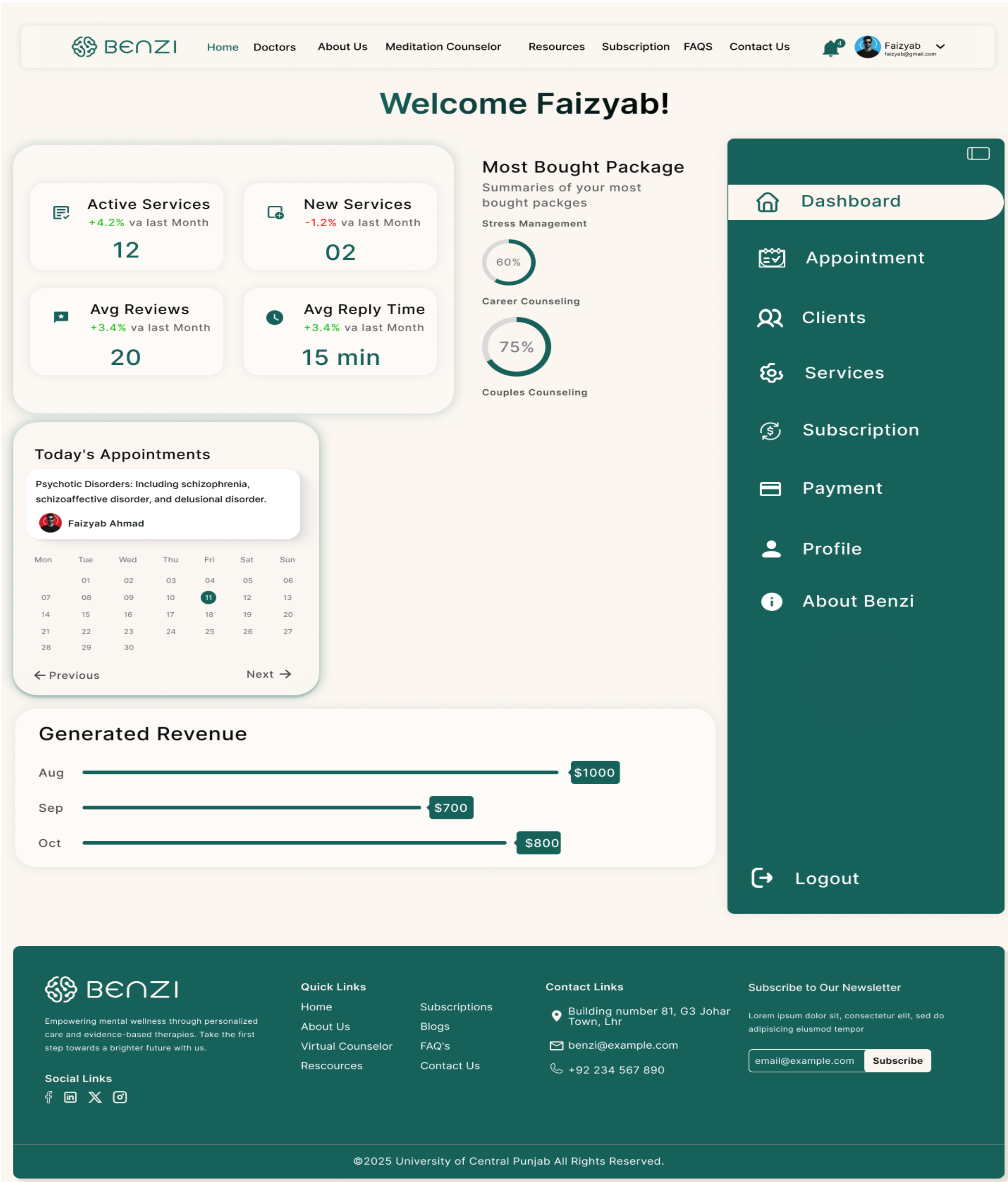


Figure 4.14: Therapist Dashboard KPI cards, today's appointments, revenue chart

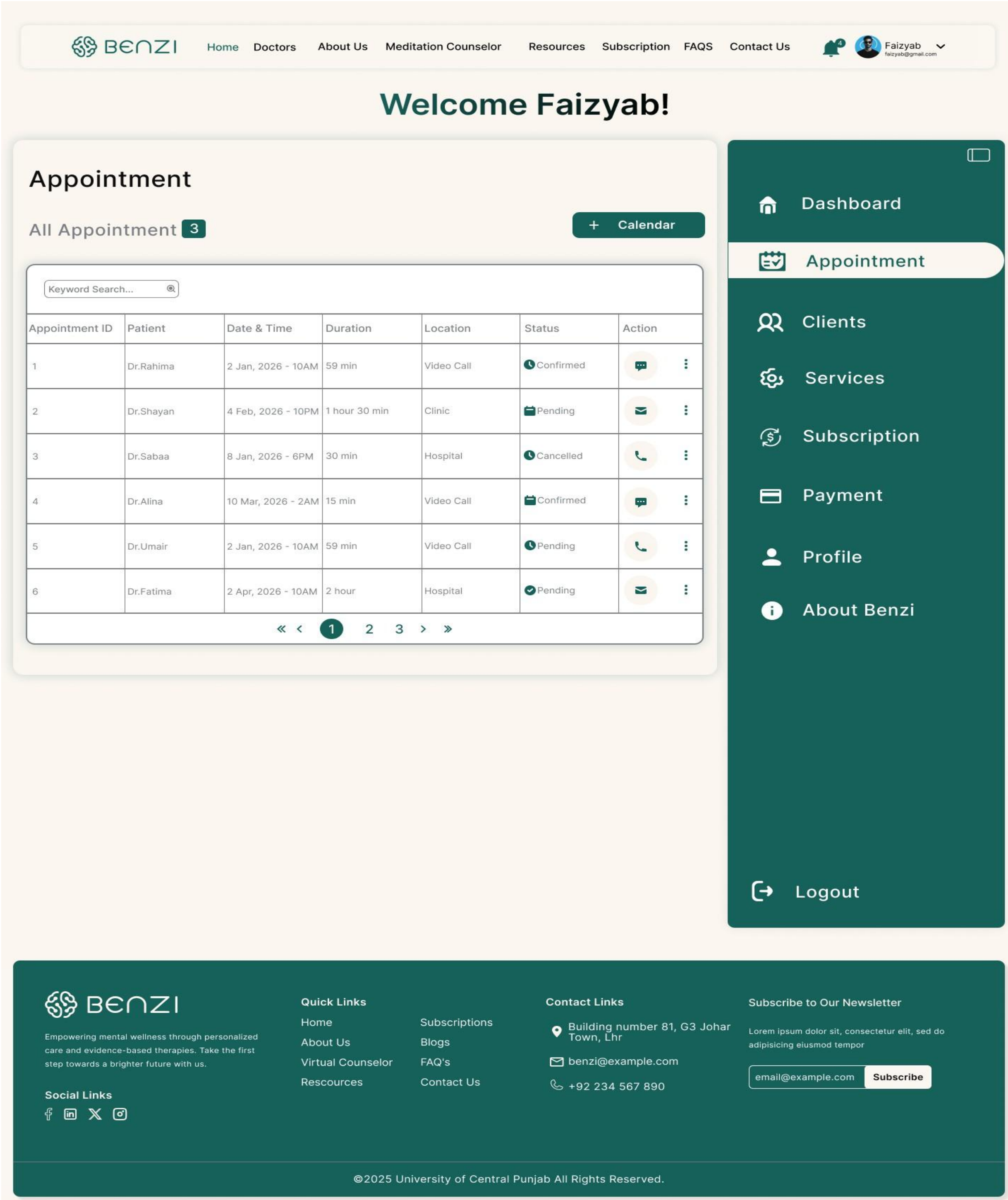


Figure 4.15: Therapist Appointments Full appointment list with calendar integration

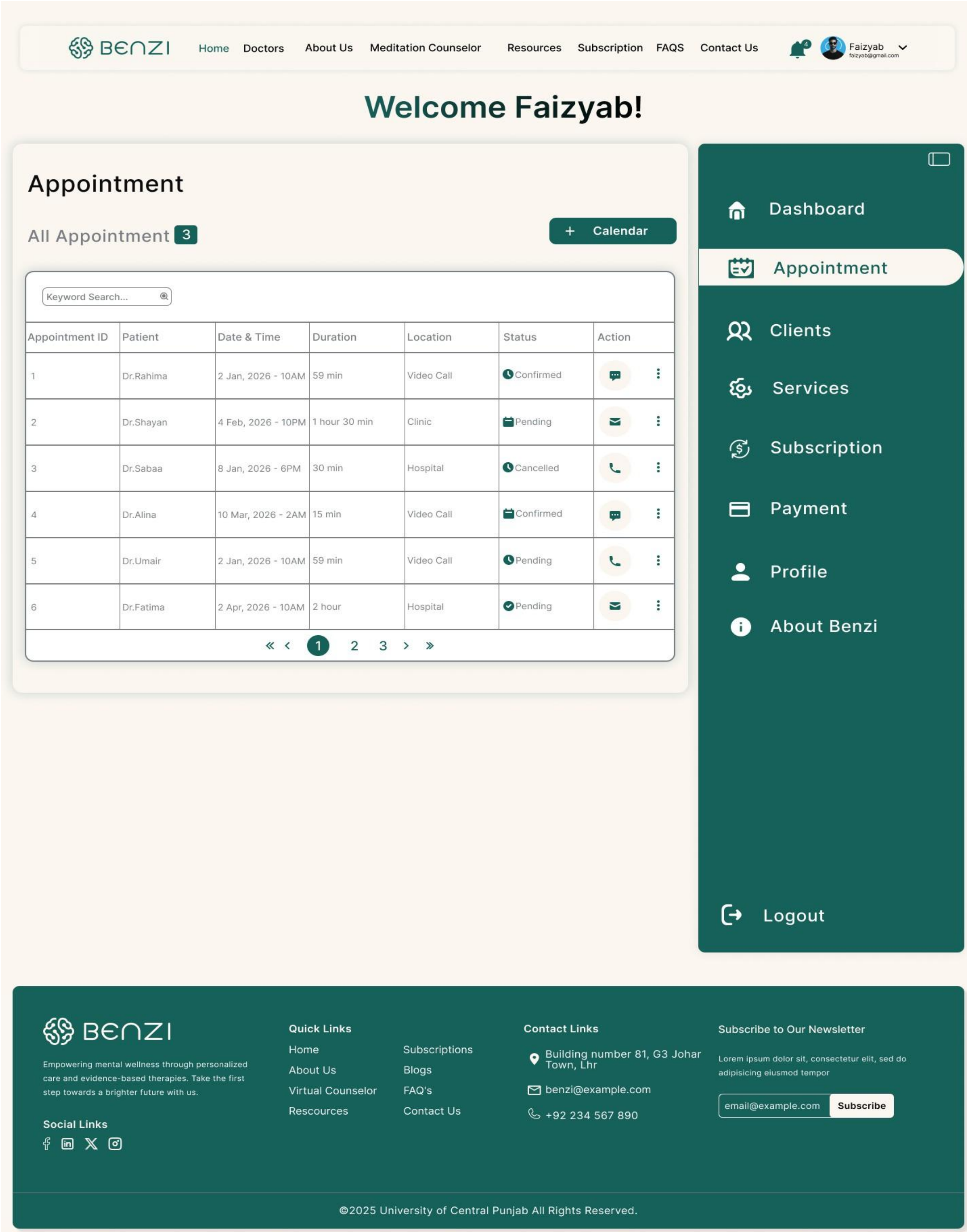


Figure 4.16: Therapist Clients — Client management table with status tracking

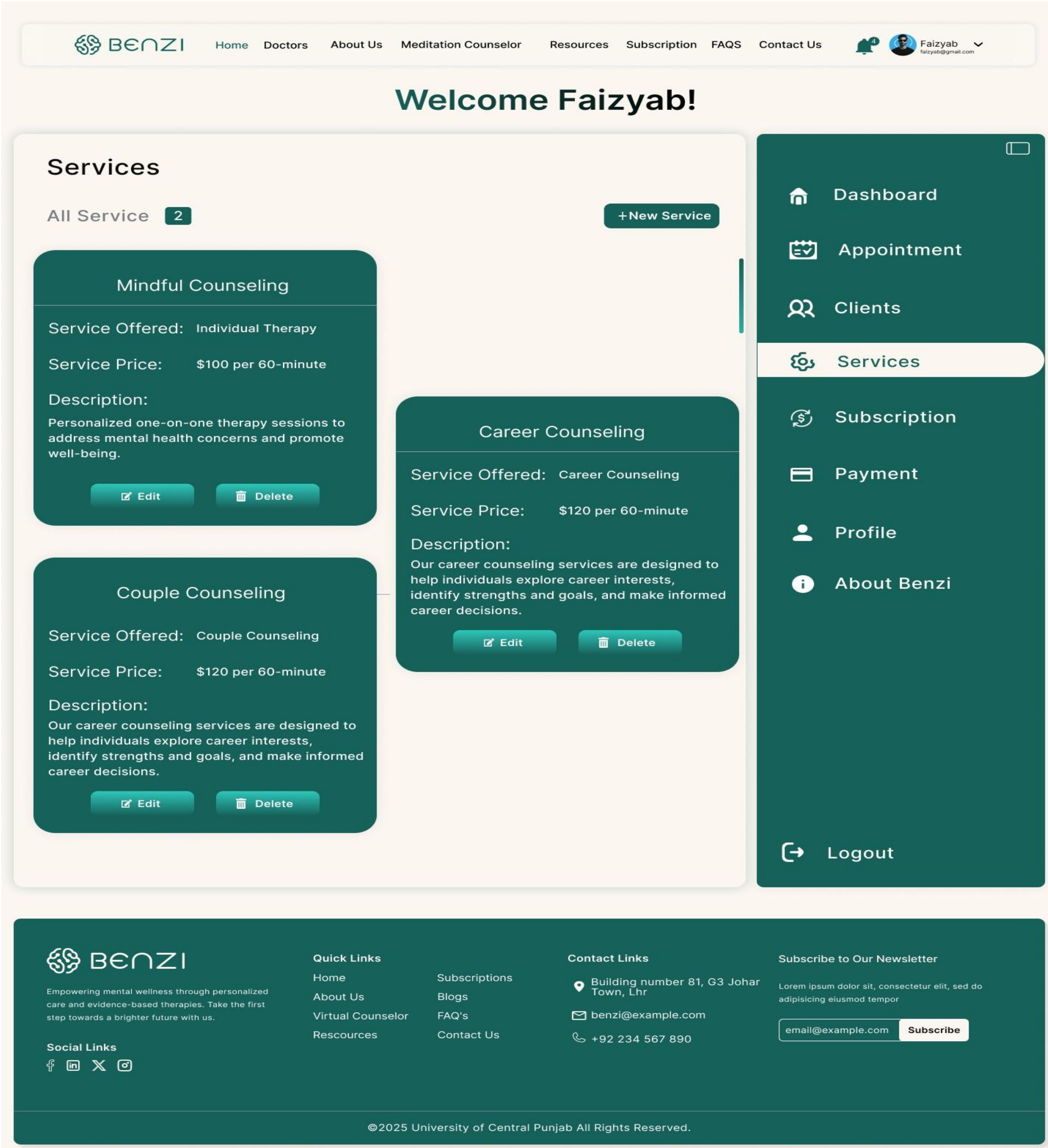


Figure 4.17: Therapist Services — Service listing with pricing, edit and delete controls

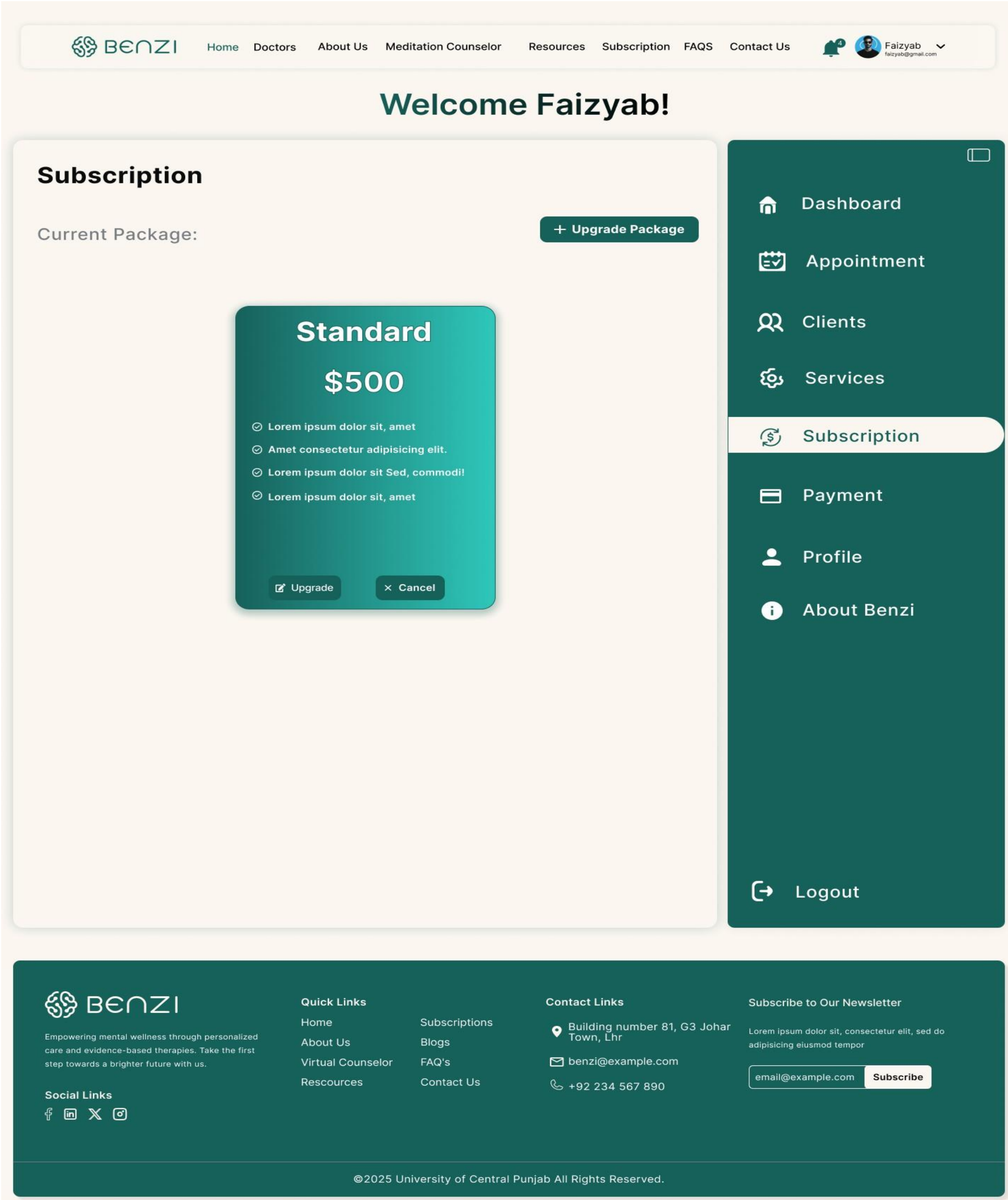


Figure 4.18: Therapist Subscription — Plan management for therapist accounts

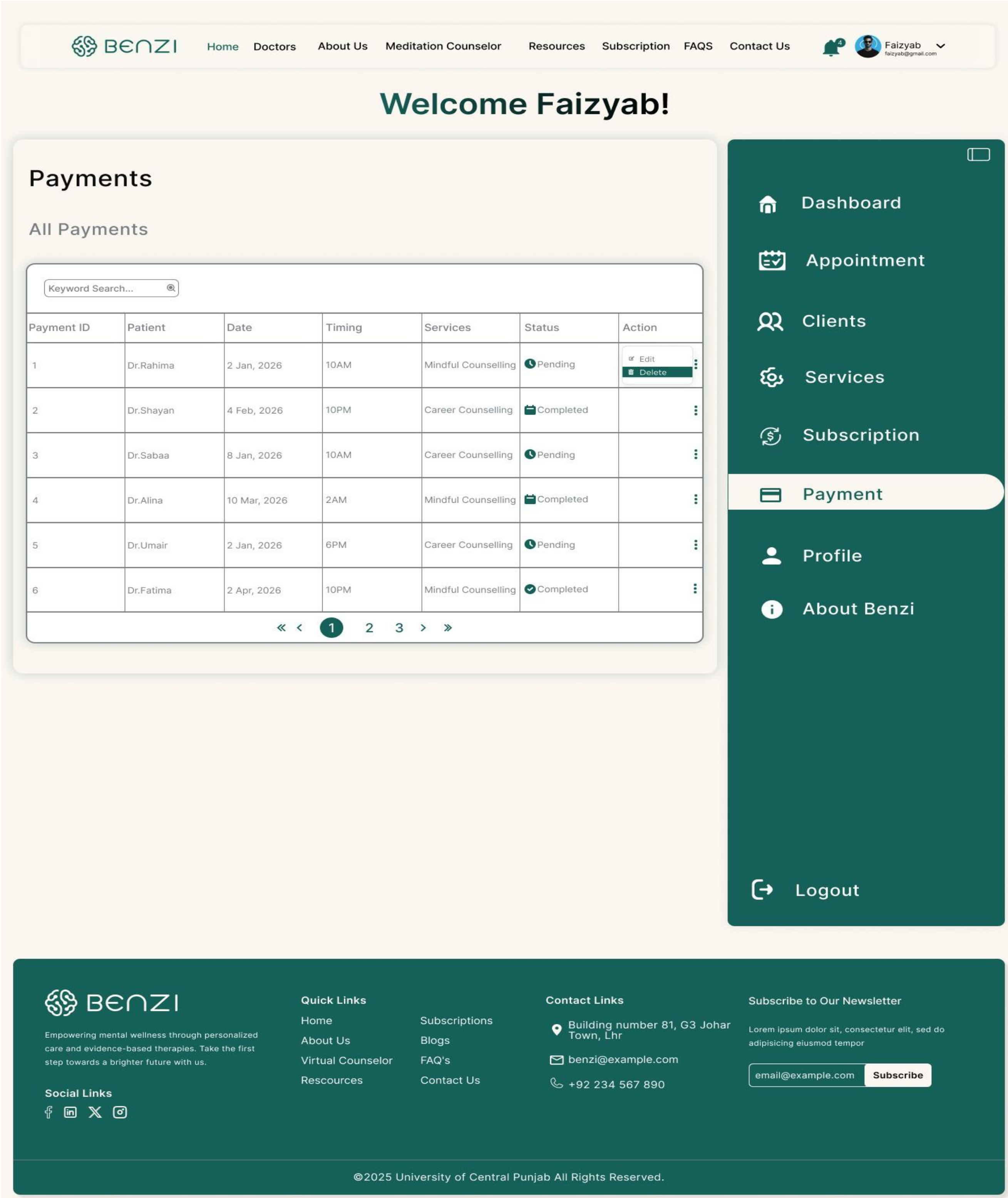


Figure 4.19: Therapist Payments — Revenue and payment history

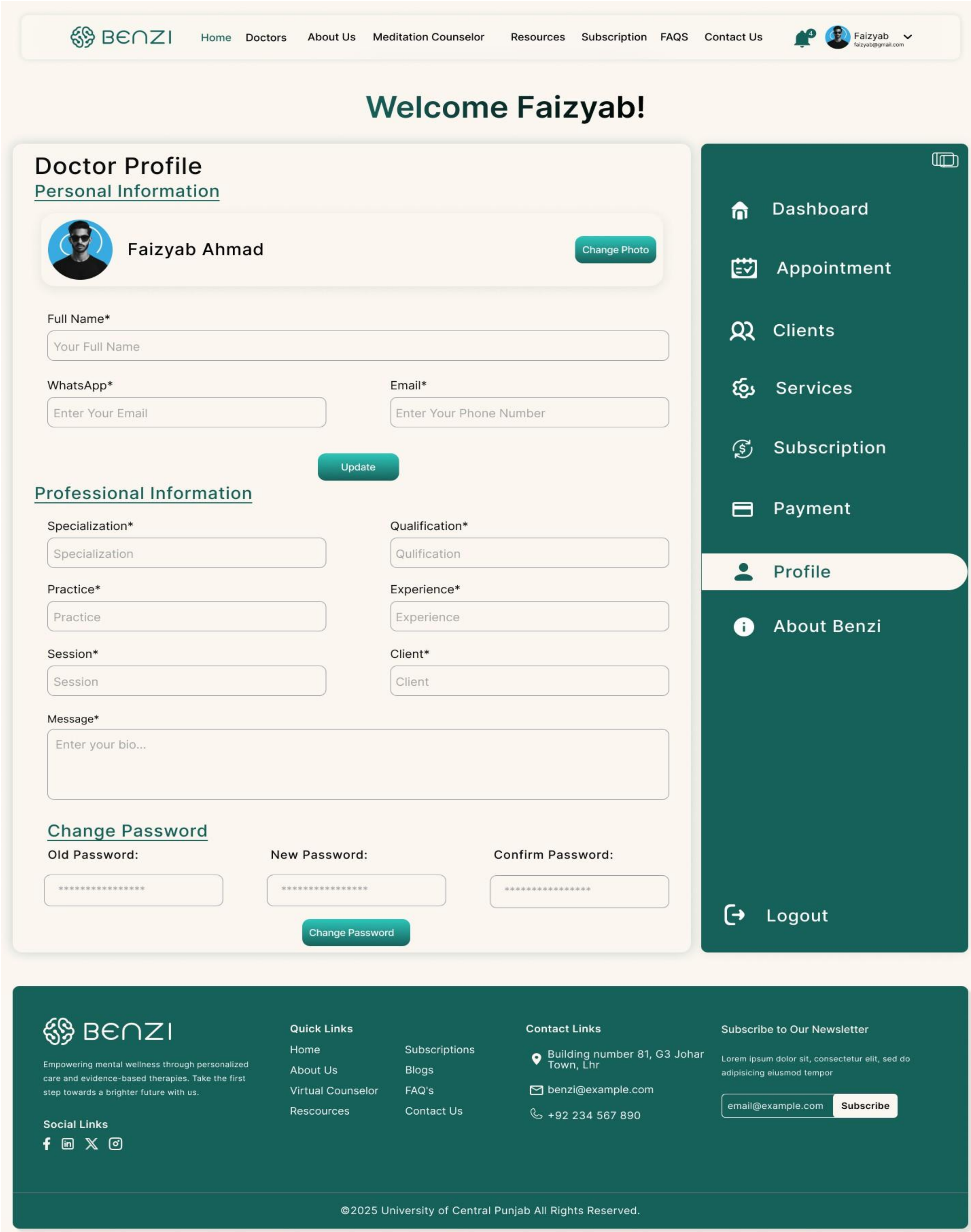


Figure 4.20: Therapist Profile — Personal and professional info with credential fields

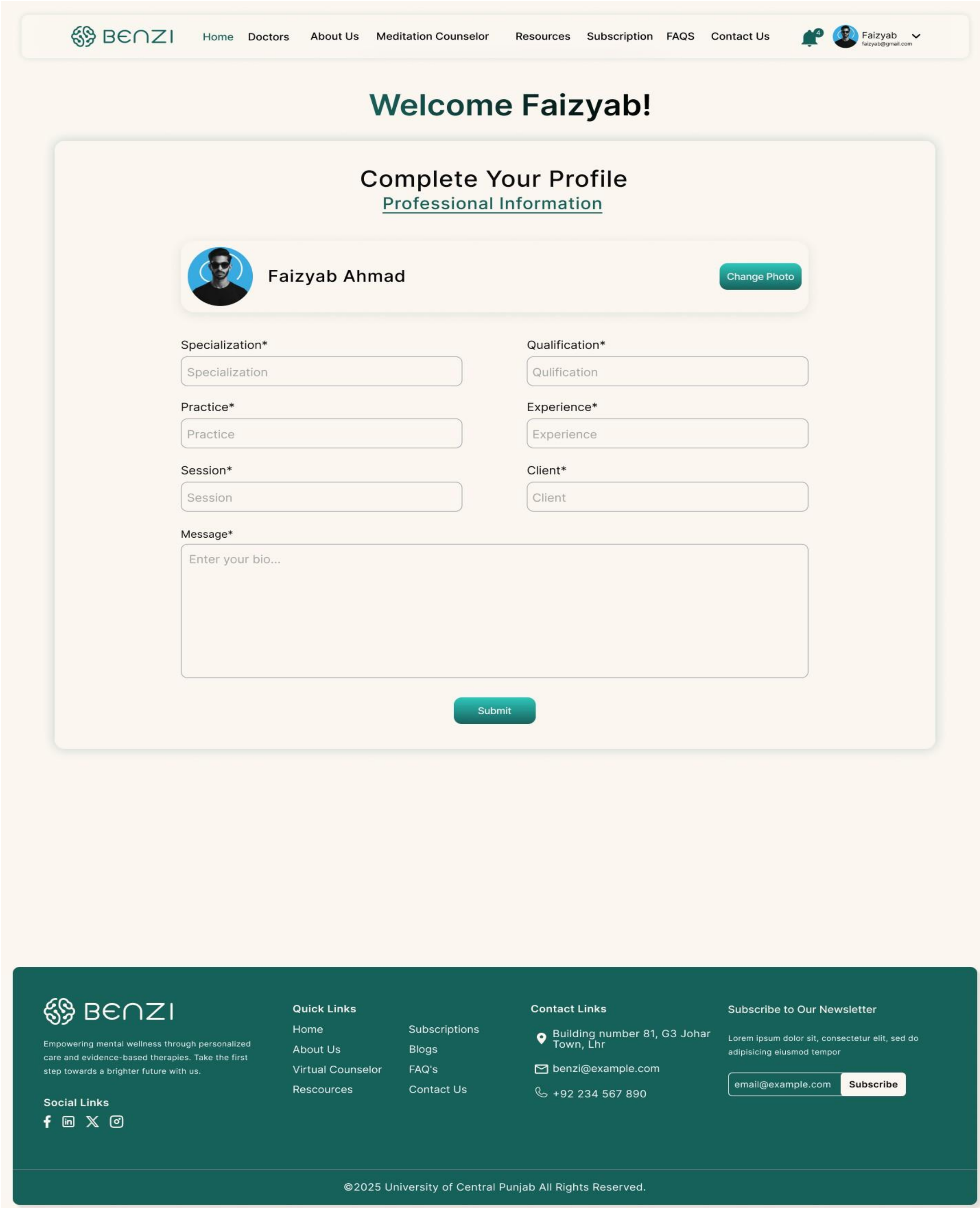


Figure 4.21: Therapist Profile Setup — Initial onboarding form

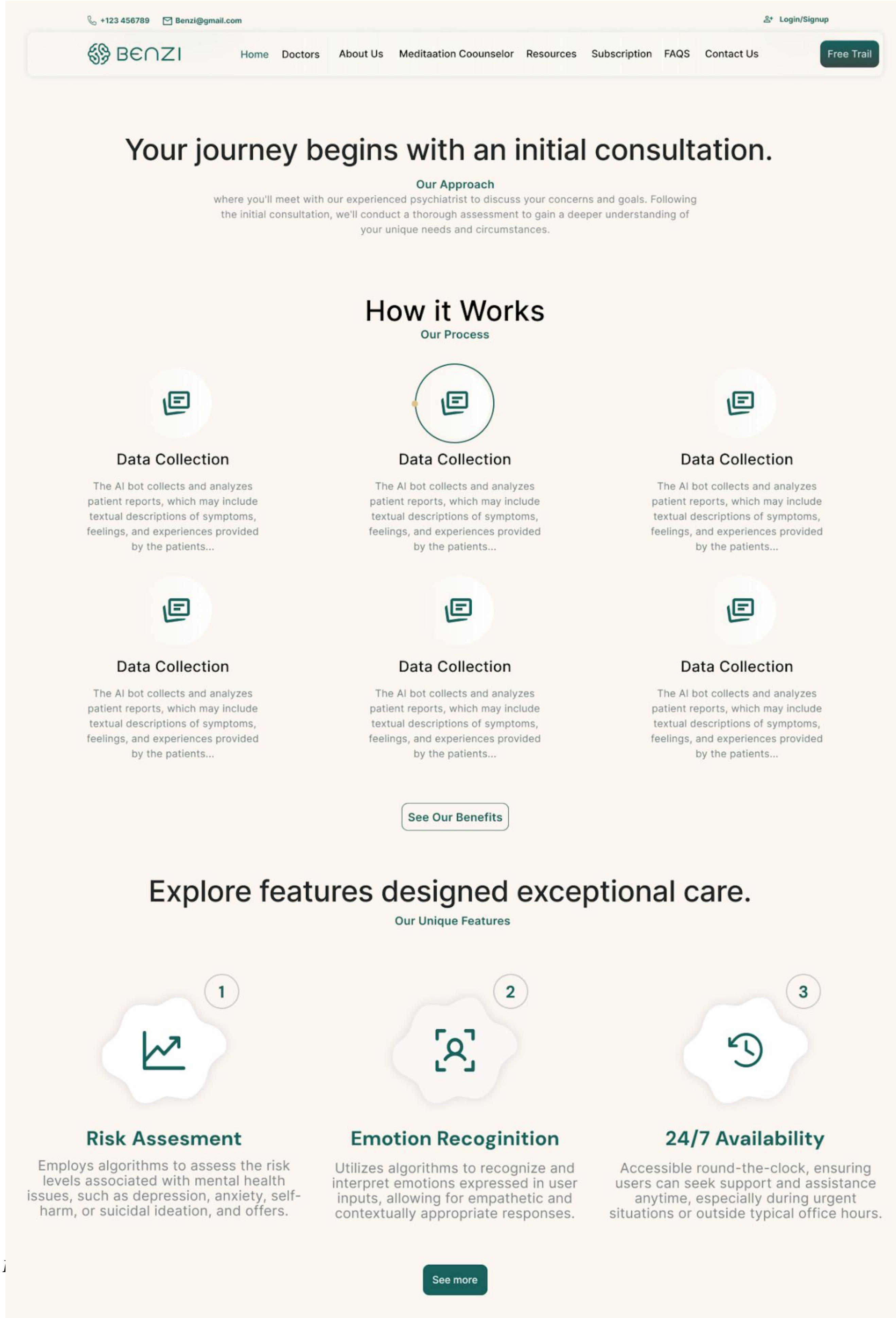

+123 456789
Benzi@gmail.com
Login/Signup
BENZI
Home
Doctors
About Us
Meditaation Coounselor
Resources
Subscription
FAQS
Contact Us
Free Trail
Your journey begins with an initial consultation.
Our Approach
where you'll meet with our experienced psychiatrist to discuss your concerns and goals. Following the initial consultation, we'll conduct a thorough assessment to gain a deeper understanding of your unique needs and circumstances.
How it Works
Our Process
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
Data Collection
The AI bot collects and analyzes patient reports, which may include textual descriptions of symptoms, feelings, and experiences provided by the patients...
See Our Benefits
Explore features designed exceptional care.
Our Unique Features
1
2
3
Risk Assesment
Employs algorithms to assess the risk levels associated with mental health issues, such as depression, anxiety, self-harm, or suicidal ideation, and offers.
Emotion Recoginition
Utilizes algorithms to recognize and interpret emotions expressed in user inputs, allowing for empathetic and contextually appropriate responses.
24/7 Availability
Accessible round-the-clock, ensuring users can seek support and assistance anytime, especially during urgent situations or outside typical office hours.
See more

Figure 4.22: Meditation Counselor Page — AI approach, How it Works, feature highlights

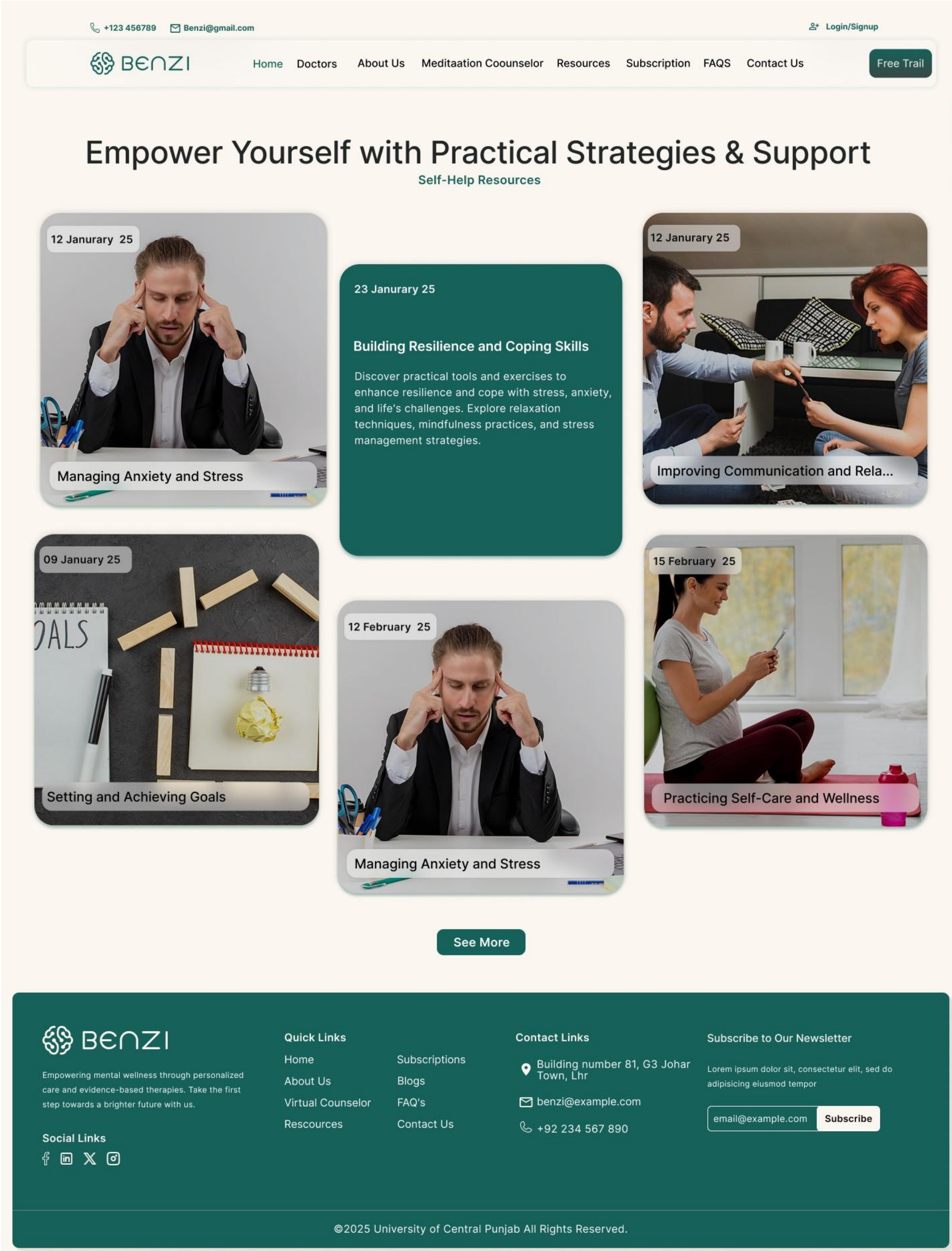
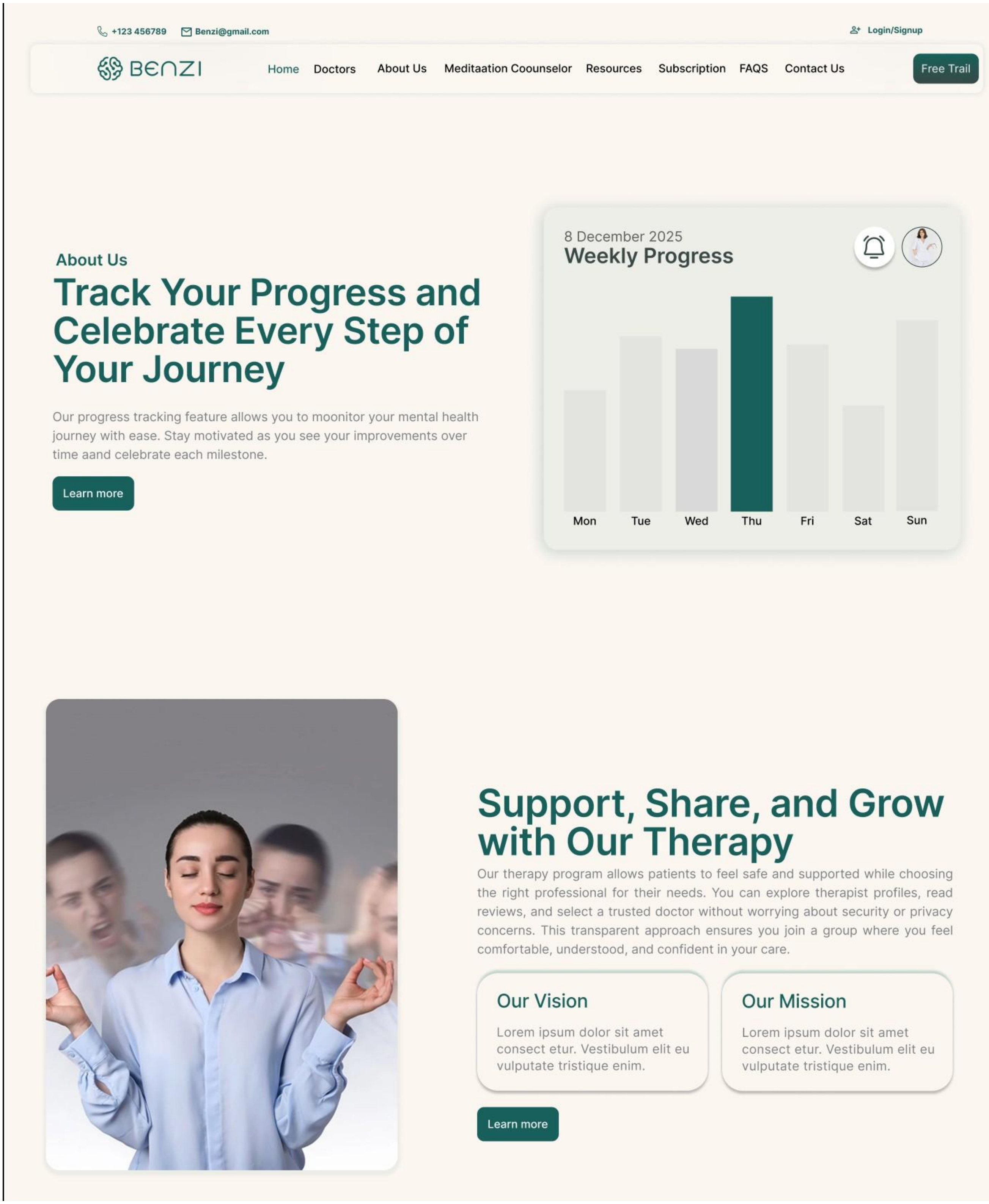


Figure 4.23: Resources Page — Mental health resource library


+123 456789
Benzi@gmail.com
Login/Signup
BENZI
Home
Doctors
About Us
Meditaation Coounselor
Resources
Subscription
FAQS
Contact Us
Free Trail
About Us
Track Your Progress and Celebrate Every Step of Your Journey
Our progress tracking feature allows you to moonitor your mental health journey with ease. Stay motivated as you see your improvements over time aand celebrate each milestone.
Learn more
8 December 2025
Weekly Progress
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Support, Share, and Grow with Our Therapy
Our therapy program allows patients to feel safe and supported while choosing the right professional for their needs. You can explore therapist profiles, read reviews, and select a trusted doctor without worrying about security or privacy concerns. This transparent approach ensures you join a group where you feel comfortable, understood, and confident in your care.
Our Vision
Lorem ipsum dolor sit amet consect etur. Vestibulum elit eu vulputate tristique enim.
Our Mission
Lorem ipsum dolor sit amet consect etur. Vestibulum elit eu vulputate tristique enim.
Learn more

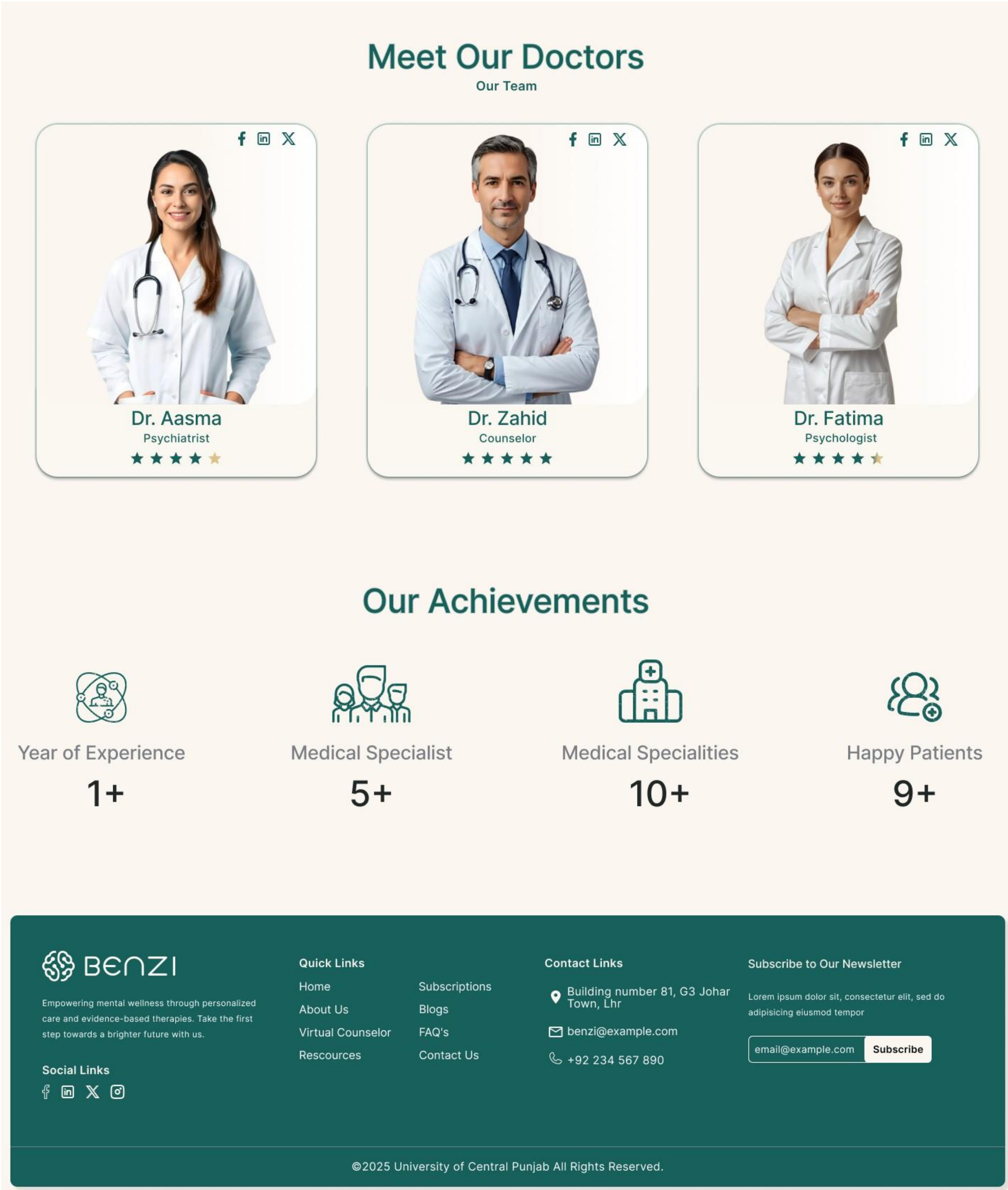


Figure 4.24: About Us — Team and mission information

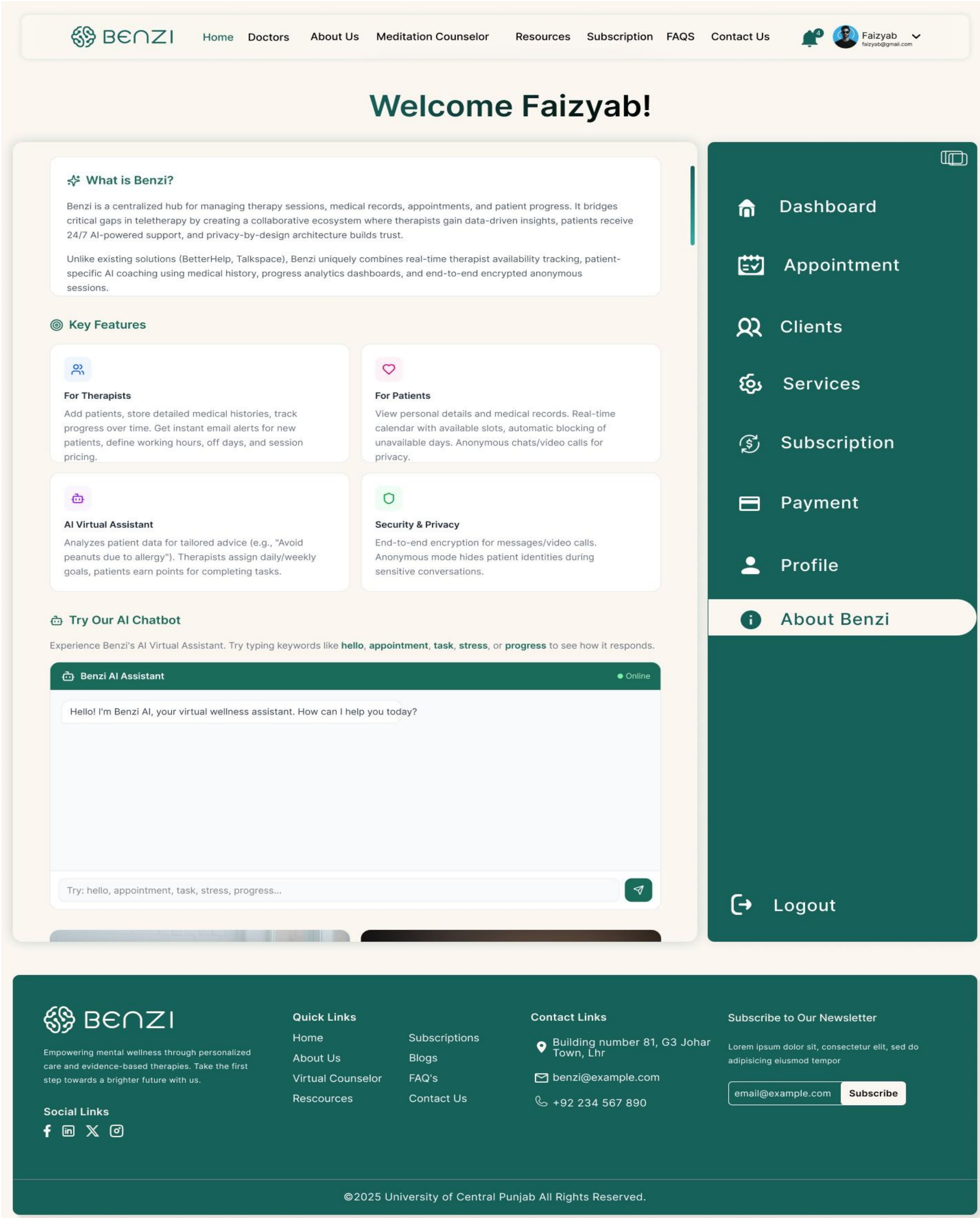


Figure 4.25: About BENZI.AI — Product overview page

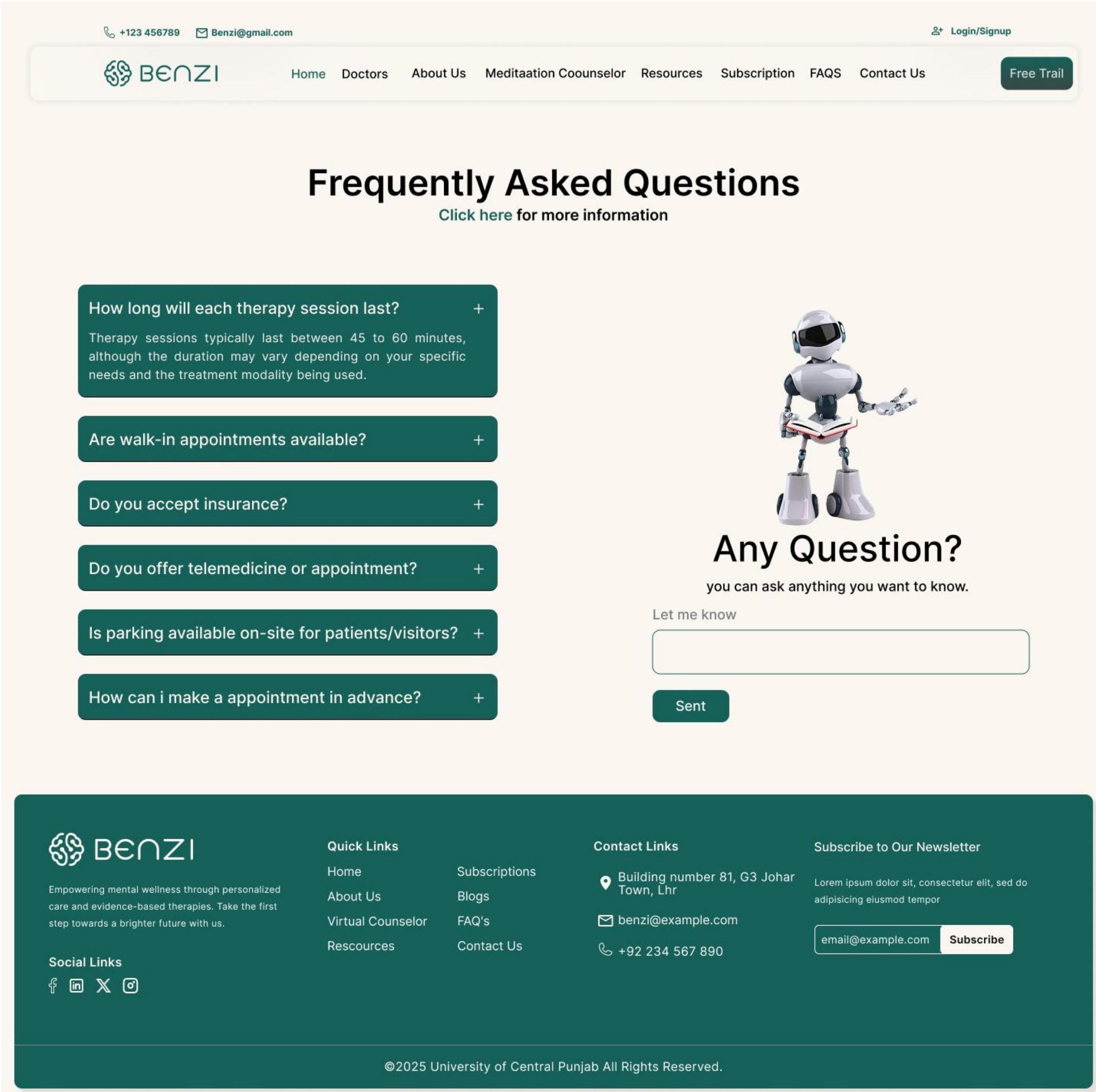


Figure 4.26: FAQs Page — Frequently asked questions

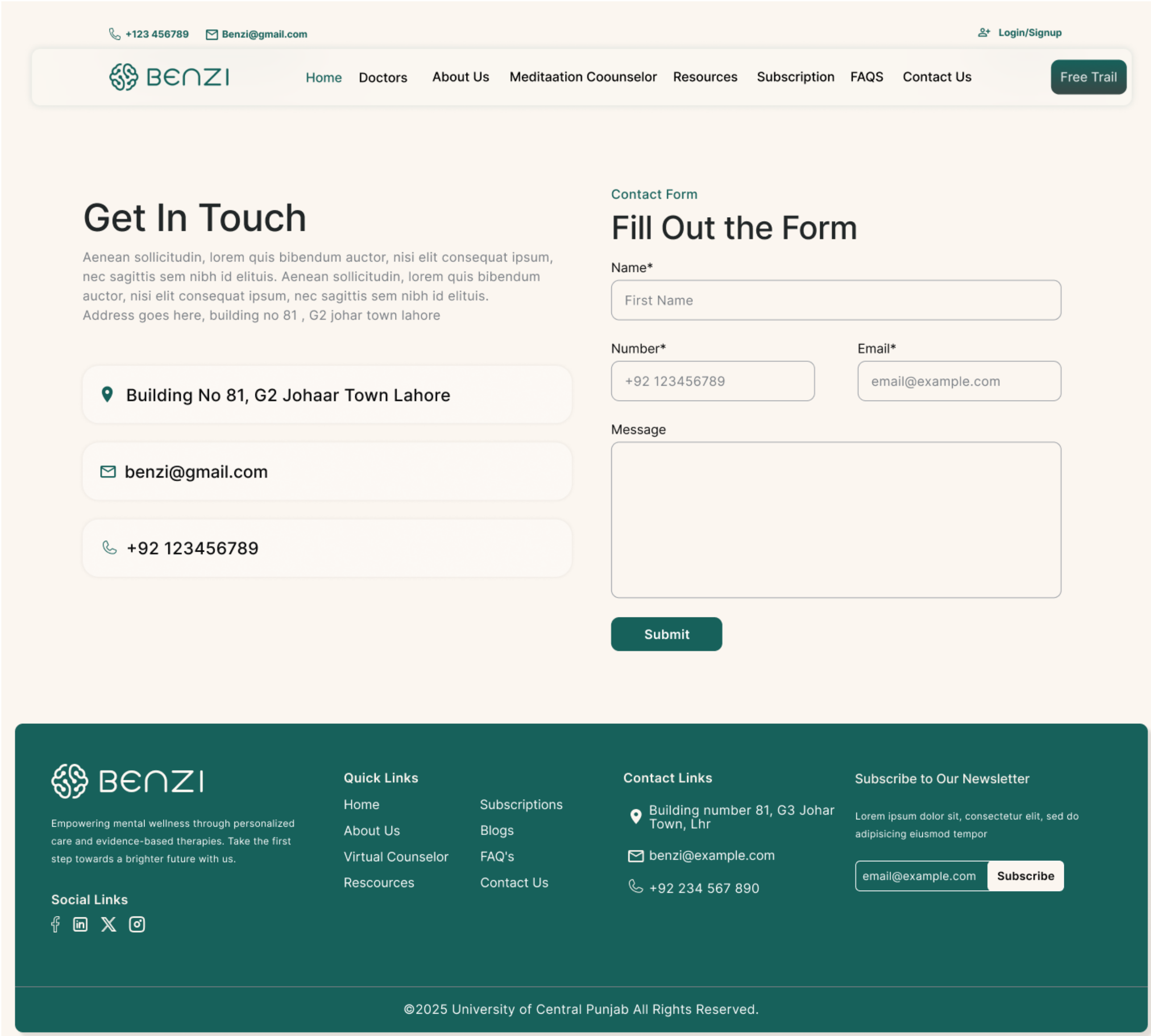


Figure 4.27: Contact Us — Support enquiry submission form

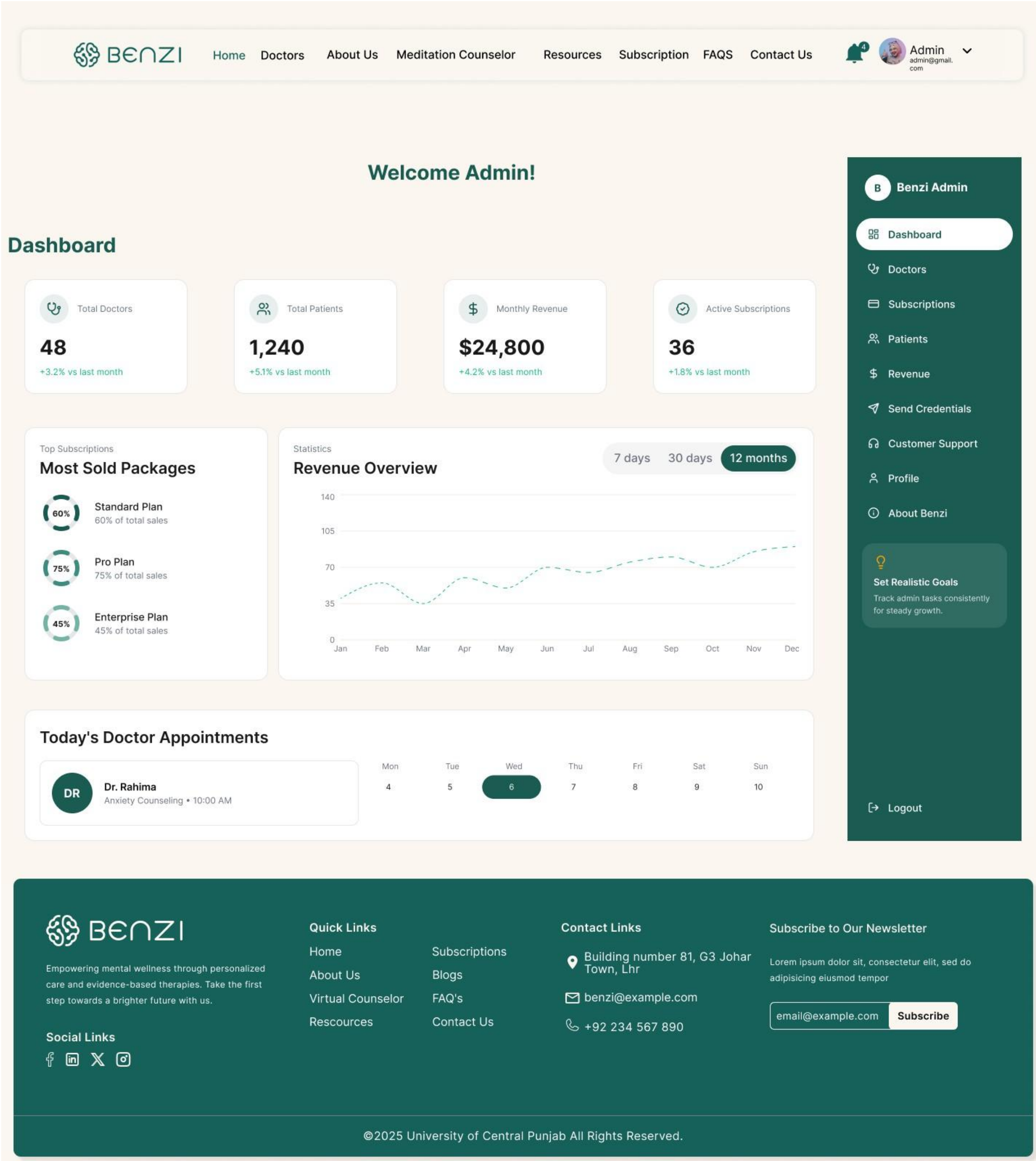


Figure 4.28: Admin Dashboard — KPI overview, revenue chart, most sold packages and today's appointments

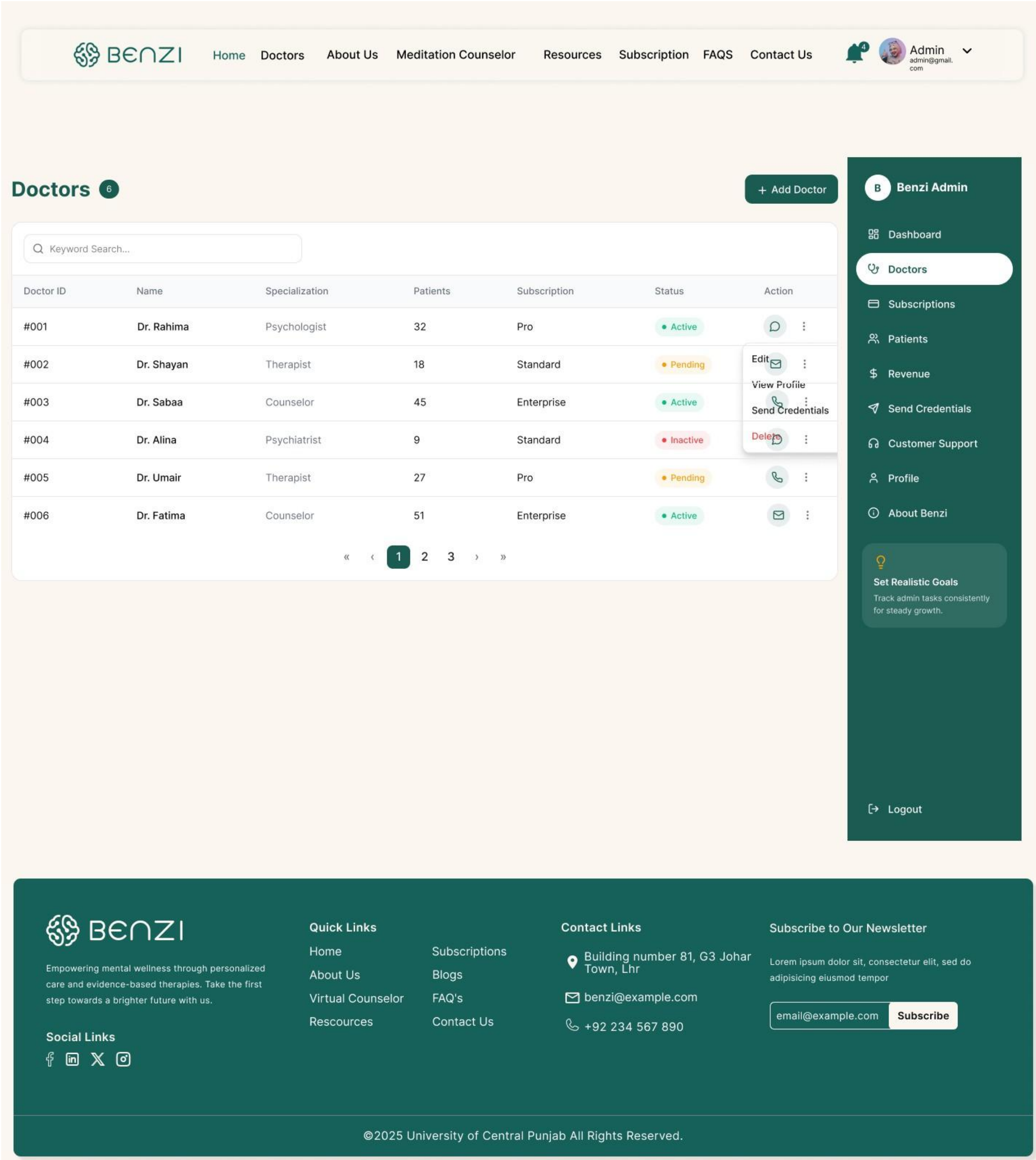


Figure 4.29: Admin Doctors Management — Doctor list with specialization, subscription plan, status and action controls

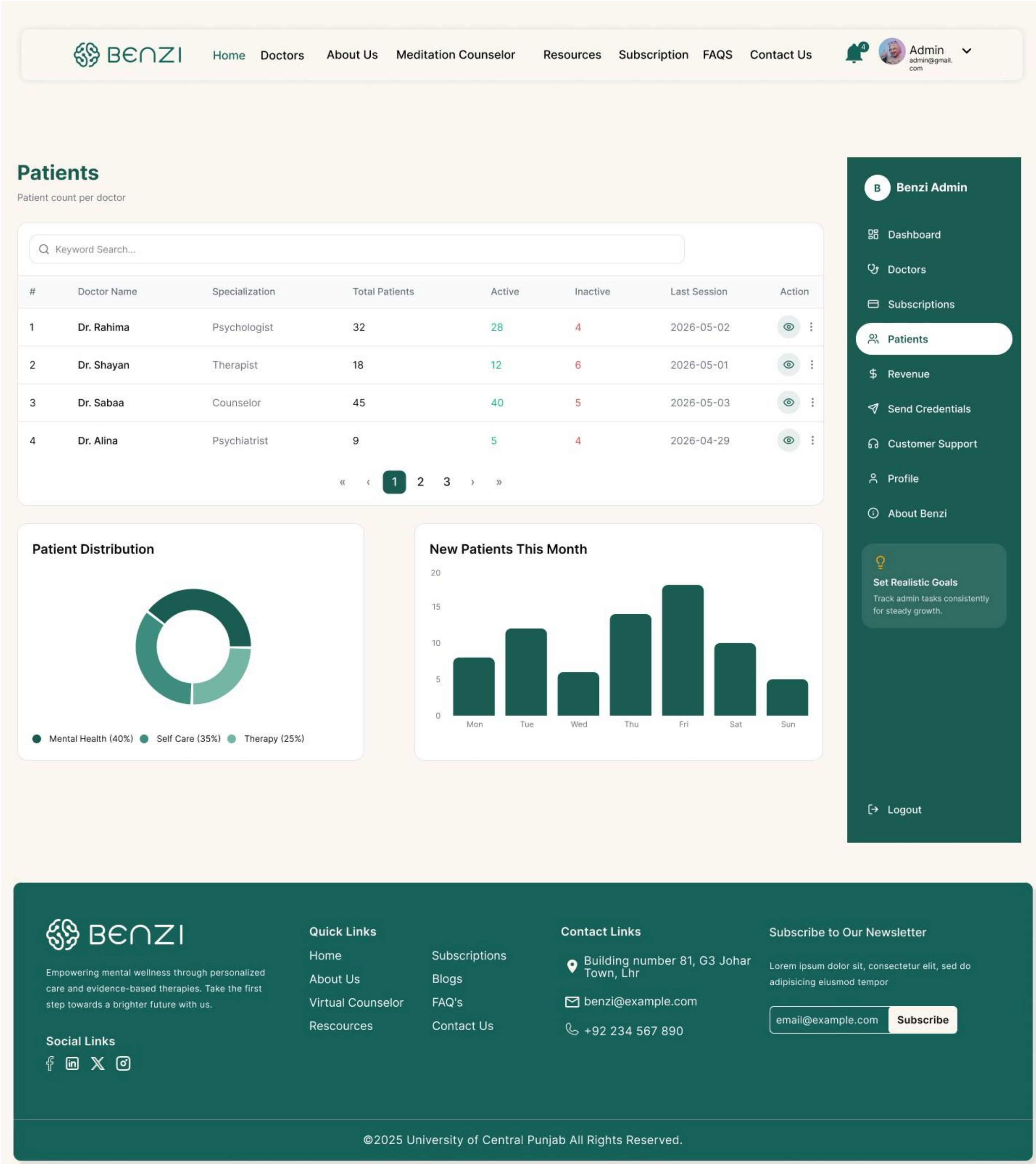


Figure 4.30: Admin Patients Overview — Patient counts per doctor, distribution chart and new patient trend

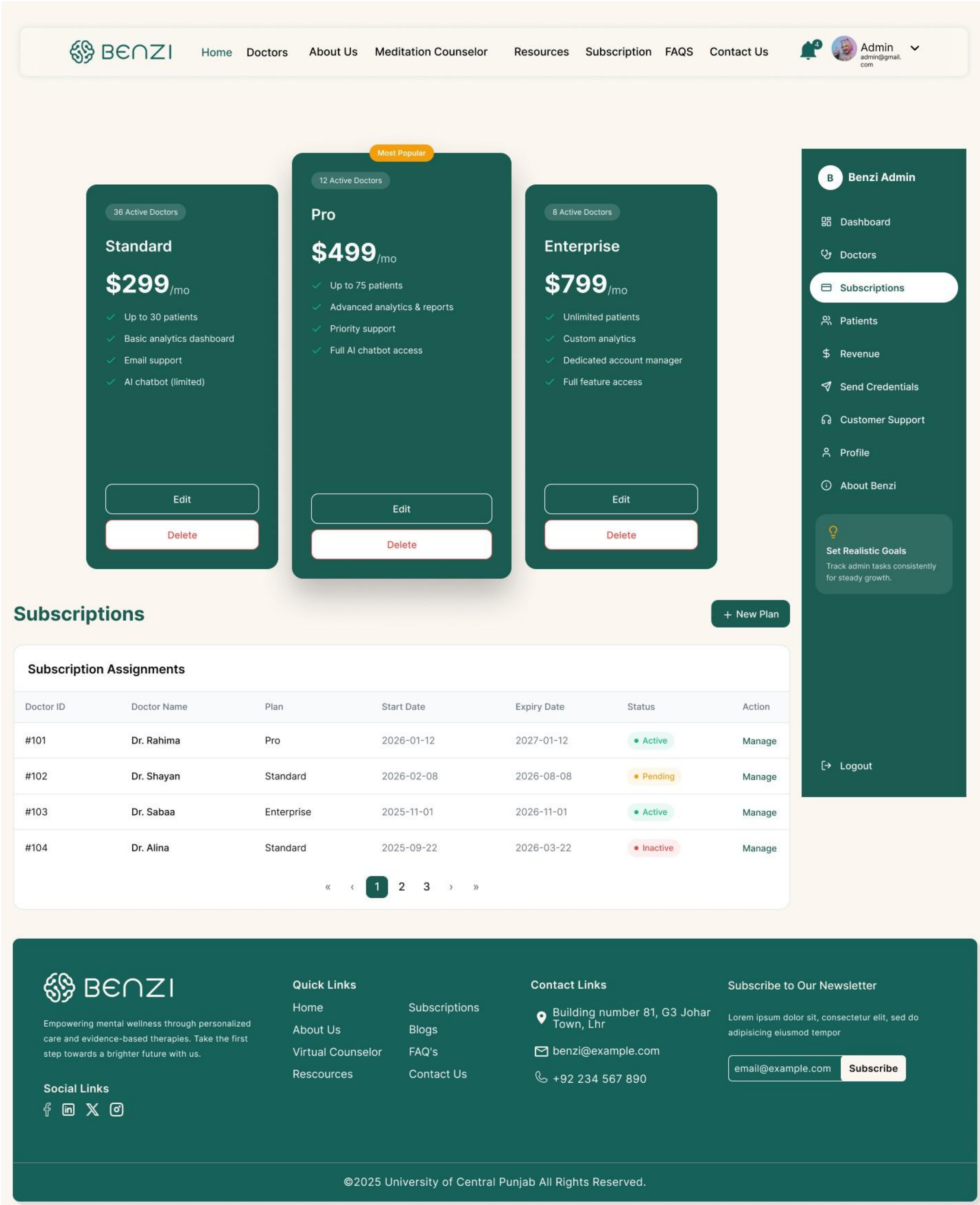


Figure 4.31: Admin Subscriptions — Plan management (Standard/Pro/Enterprise) and subscription assignment table

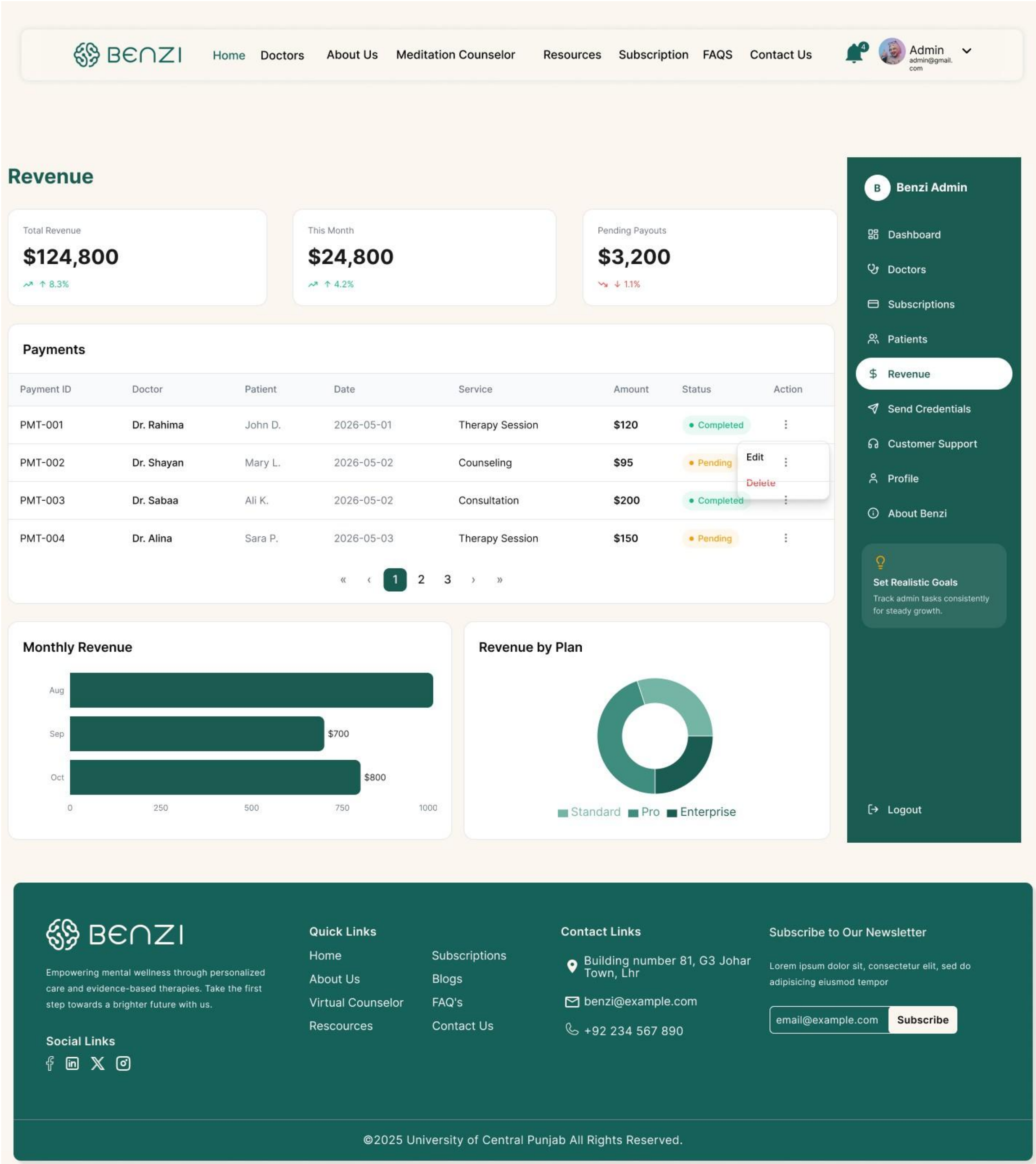


Figure 4.32: Admin Revenue — Total revenue KPIs, payments table, monthly bar chart and plan-based breakdown

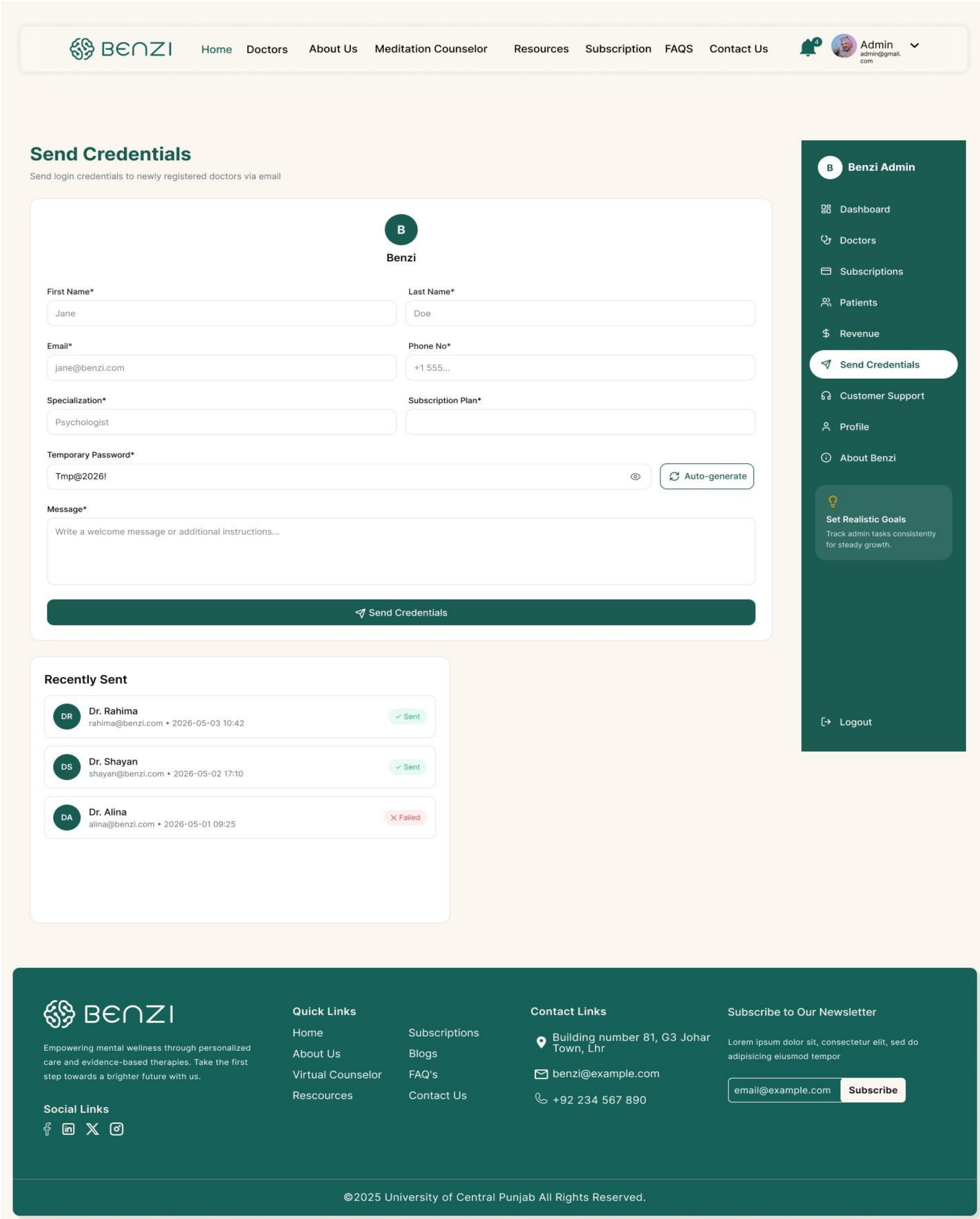


Figure 4.33: Admin Send Credentials (Active State) — Send Credentials panel with highlighted navigation state

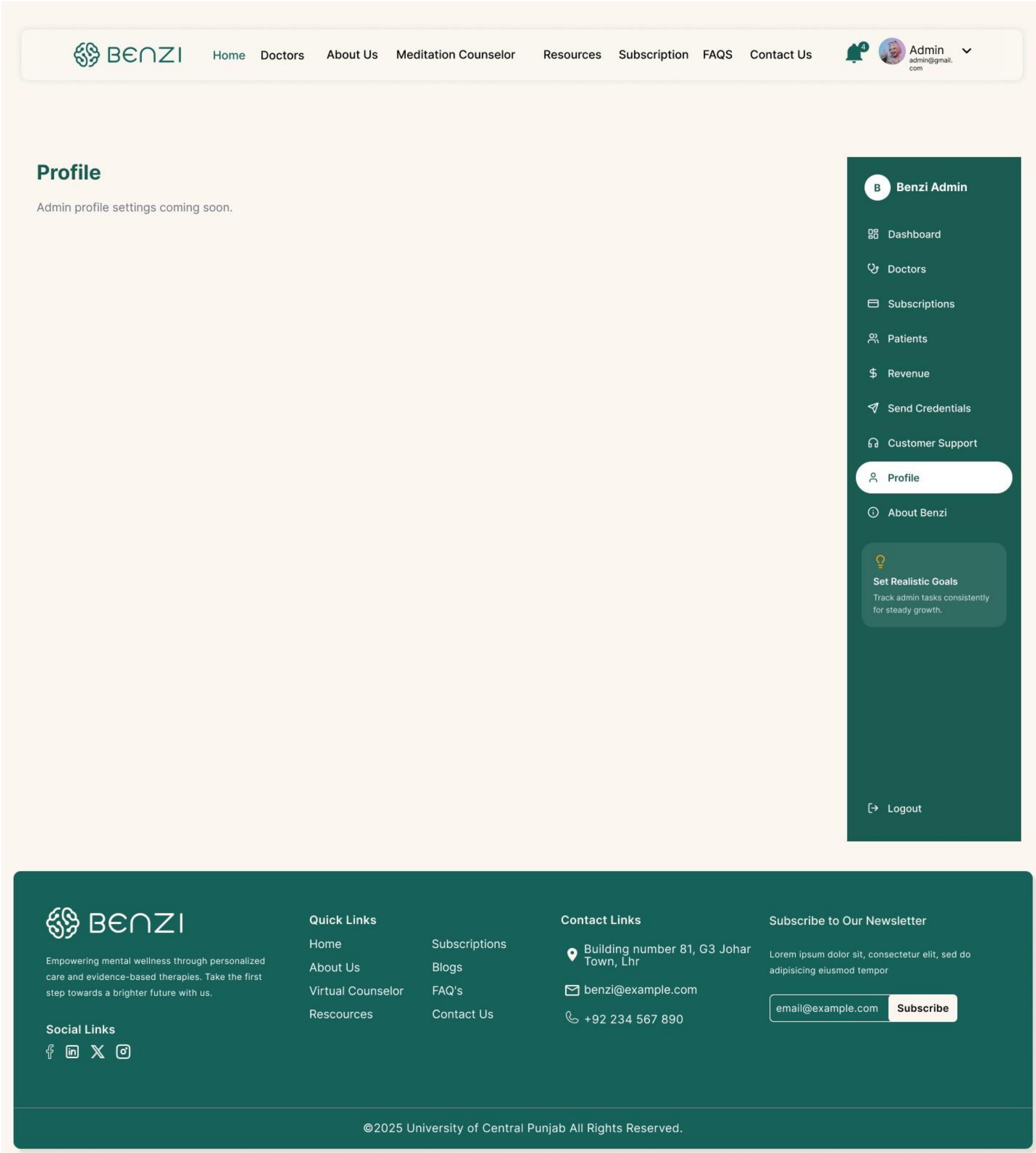


Figure 4.34: Admin Profile — Admin profile settings page (in development)

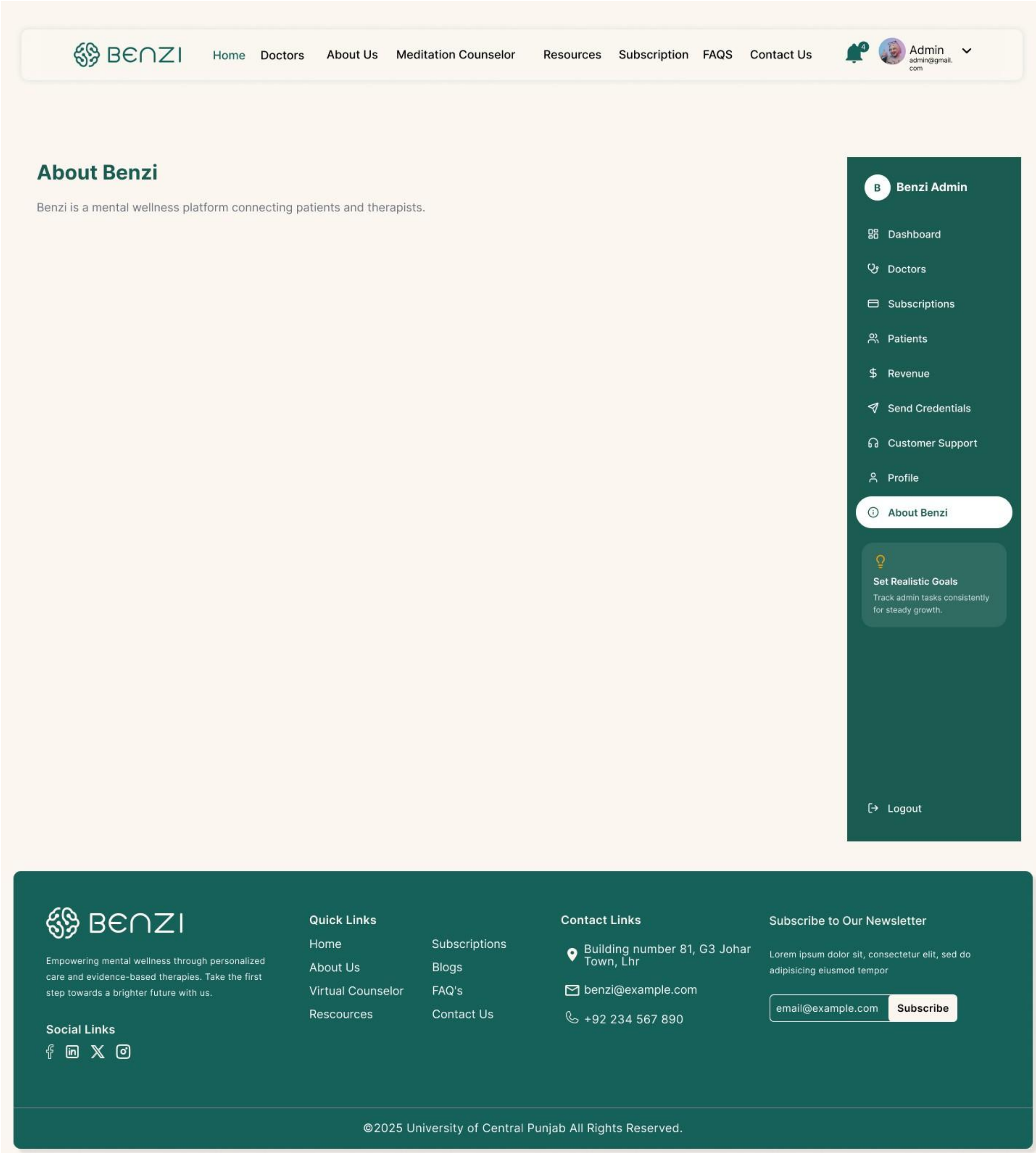


Figure 4.35: Admin About Benzi — Platform description future under developement for admin reference

4.3 Additional Information

The complete interactive prototype of BENZI.AI is accessible via Figma at the following link: https://www.figma.com/design/WUv3qWjLqaIE71mxhA2ReW/Final-Year-Project

The prototype covers all 27 screens documented in Section 4.2. All Patient Portal screens share a consistent collapsible right-side navigation panel containing: Dashboard, Conversations, Goals, Progress, Appointment, Help & Support, Reports, Profile, and Logout. All Therapist Portal screens share a consistent right-side panel containing: Dashboard, Appointment, Clients, Services, Subscription, Payment, Profile, About Benzi, and Logout. This persistent navigation pattern eliminates the need for back-button navigation and ensures zero-depth access to any function from any screen.

The color scheme of deep teal (`#0F5C52` approximately) on white is applied consistently across all authenticated screens. Public screens use the same palette with an off-white/cream background to distinguish pre-login from post-login environments.

5. Other Design Details

5.1 Security and Privacy Design

BENZI.AI implements a defense-in-depth security model designed to meet HIPAA and GDPR compliance requirements. All data transmitted between client browsers and the backend is encrypted using TLS 1.2 or higher over HTTPS no plain HTTP communication is permitted in any environment. Session management relies on JWT tokens with a 3600-second expiry, requiring re-authentication after timeout. Two-Factor Authentication (2FA) is supported for all user roles. All patient healthcare data including session notes, medical history, uploaded reports, and AI interaction logs — is encrypted at rest using AES-256, with encryption keys managed through a cloud-based Key Management Service (AWS KMS or Azure Key Vault). The Admin role is architecturally blocked from all patient data at the RecordsService middleware level, returning HTTP 403 FORBIDDEN regardless of any token or credential presented.

5.2 Security and Privacy Design

The BENZI AI virtual assistant is designed strictly as a supportive guidance tool, not a diagnostic or prescriptive clinical system. The following AI governance rules are enforced by the `safetyFilter()` module within the AI Engine. First, the AI will never produce a clinical diagnosis, medication prescription, or crisis intervention directive any such query is detected and redirected to professional resources. Second, all AI recommendations are presented transparently with contextual attribution (e.g., "Based on your recent sessions…"). Third, user consent for AI feature activation is explicitly obtained at first use. Fourth, all AI interaction logs are stored in MongoDB and are accessible to the treating Therapist the patient is informed of this at onboarding. Fifth, the AI model is not trained on individually identifiable patient data from the live system; training uses controlled and anonymized datasets only.

5.3 and Maintainability

The modular service-oriented architecture ensures that any individual component — the AI Engine, the Notification Service, the Appointment Service — can be independently scaled on the cloud platform without redeployment of the full system. MongoDB's horizontal sharding capability supports growth in patient and session data volume. The n8n automation platform supports addition of new workflow types without code changes to the backend. All React.js UI components are independently importable modules, allowing new screens to be added without modifying existing portal layouts.

5.4 Accessibility Design

All text elements meet WCAG AA contrast ratio requirements. Font sizes are set at minimum 12pt for body content across all screens, with headings at 16pt or above. All interactive elements — buttons, form fields, navigation items — are keyboard-navigable with visible focus indicators. The mood tracker on the Patient Dashboard uses universally recognizable emoji-based icons with text labels to eliminate ambiguity. Color alone is never used as the sole indicator of status — all appointment statuses (Confirmed/Pending/Cancelled) additionally use distinct icons alongside color coding.

5.5 Real-Time Features Design

Live therapy sessions use WebSocket connections maintained between the client browser and the Node.js backend for low-latency bidirectional communication. The WebSocket layer implements a three-retry auto-reconnect protocol with 2-second intervals on connection drop, minimizing session interruption. The Zoom SDK integration creates E2EE-encrypted meeting rooms with the session token distributed by the backend — the meeting URL is never stored in MongoDB after session completion, only the session metadata (ID, duration, status) is retained.

5.6 Gamification Design

The wellness goal system implements a points-based engagement mechanism to encourage patient adherence to therapeutic goals. Each goal assigned by a therapist carries a configurable point value. On completion, the `addPoints()` function credits the patient's profile with the earned points, and a visual counter on the Patient Goals screen animates to reflect the new total. A community sentiment indicator (Negative / Neutral / Positive counters visible on the Goals screen) provides social proof of platform engagement without exposing individual patient data. Goal completion progress is visualized as a pie chart showing Completed (70%), In Progress (20%), and Pending (10%) breakdown across all active goals.

6. Test Specification and Results

6.1 Test Case Specification

This section presents test case specifications for the BENZI.AI system covering all major implemented and partially implemented modules. Test cases are designed to validate functional requirements corresponding to the primary use cases defined in the Phase 2 SDS. Modules covered include: User Authentication & Registration, Appointment Booking & Scheduling, Therapist Dashboard & Patient Records, Wellness Goals & Gamification, Notifications, Payment Processing, Admin Panel & Subscription Management, and the AI Virtual Assistant module.

Table 6.1: TC-1 — User Login with Valid Credentials

Field	Details
Identifier	TC-1
Related Requirement(s)	UC-6, SRS Section 3.1 — User Authentication & RBAC
Short Description	Verify that a registered user can log in with valid credentials and is redirected to the correct role-based dashboard
Pre-condition(s)	Verified user account exists in MongoDB. System is running. Login page is accessible.
Input Data	Email: testpatient@benzi.ai / Password: Test@1234 / Role: Patient / 2FA Code: Valid OTP
Detailed Steps	1. Open the BENZI.AI login page. 2. Enter registered email and password. 3. Enter valid 2FA code when prompted. 4. Click Login. 5. Observe the redirect destination and dashboard content displayed.
Expected Result(s)	JWT token is issued. User is redirected to the Patient Dashboard. Role-specific navigation is shown (appointments, goals, AI chat). No therapist or admin elements are visible.
Post-condition(s)	Active JWT session exists. Login event is recorded in system audit log.
Actual Result(s)	Login successful. Patient Dashboard loaded correctly. Role isolation confirmed.
Test Case Result	Pass

Table 6.2: TC-2 — New Patient Registration

Field	Details
Identifier	TC-2
Related Requirement(s)	UC-6, SRS Section 3.1 — User Authentication & RBAC

Short Description	Verify that a new patient can register successfully and receives an email verification link
Pre-condition(s)	No existing account with test email. SendGrid API is operational. Registration page is accessible.
Input Data	Full Name: Sara Khan / Email: sarakhan@test.com / Password: Sara@5678 / Role: Patient / Phone: 03001234567
Detailed Steps	1. Navigate to Registration page. 2. Select role as Patient. 3. Fill all required fields. 4. Click Register. 5. Open registered email inbox. 6. Click the verification link received.
Expected Result(s)	Account created in MongoDB with status PENDING_VERIFICATION. Verification email dispatched via SendGrid. After clicking link, status updates to VERIFIED. User redirected to login page with success message.
Post-condition(s)	Verified patient account exists. User can log in with registered credentials.
Actual Result(s)	Registration successful. Verification email received and link functional. Account status updated correctly.
Test Case Result	Pass

Table 6.3: TC-3 — Book Appointment with Online Payment

Field	Details
Identifier	TC-3
Related Requirement(s)	UC-1, SRS Section 3.1 — Appointment Scheduling
Short Description	Verify that a patient can successfully book an available appointment slot using online payment
Pre-condition(s)	Patient is logged in. At least one verified therapist with available slots exists. Stripe API operational in test mode.
Input Data	Therapist: Dr. Ahmed (T001) / Date: Next available weekday / Time: 10:00 AM / Payment: Stripe test card 4242 4242 4242 4242
Detailed Steps	1. Navigate to Book Appointment from Patient Dashboard. 2. Select therapist Dr. Ahmed. 3. Pick an available date and time slot. 4. Choose Pay Online. 5. Enter Stripe test card details. 6. Click Confirm Booking. 7. Check confirmation screen and email/SMS inbox.

Expected Result(s)	Stripe payment returns status succeeded. Appointment record inserted in MongoDB with status CONFIRMED. Patient and Therapist receive confirmation email (SendGrid) and SMS (Twilio). Appointment visible on Patient Dashboard. n8n schedules 24-hour reminder.
Post-condition(s)	Confirmed appointment in database. Slot marked unavailable for other patients. Reminder workflow active in n8n.
Actual Result(s)	Booking confirmed. Payment processed. Notifications received by both parties. Minor delay observed in SMS delivery (~45 seconds).
Test Case Result	Pass

Table 6.4: TC-4 — Upload and Access Encrypted Patient Record

Field	Details
Identifier	TC-4
Related Requirement(s)	UC-3, SRS Section 3.1 — Secure Data Practices
Short Description	Verify that a therapist can upload an encrypted patient record, the patient can view it read-only, and the admin is denied access
Pre-condition(s)	Therapist is logged in. Linked patient profile exists. Cloud storage (AWS S3 / Azure Blob) is operational.
Input Data	File: session_notes.pdf (valid PDF, under 10MB) / Patient ID: P001 / Therapist ID: T001
Detailed Steps	1. Therapist opens patient P001 profile. 2. Clicks Upload Record and selects session_notes.pdf. 3. Clicks Upload and waits for confirmation. 4. Log in as Patient P001 and navigate to My Records. 5. Attempt to open the uploaded file. 6. Log in as Admin and attempt to access patient records of P001.
Expected Result(s)	File validated, AES-256 encrypted, and stored in cloud storage. File URI persisted in MongoDB. Patient views file in read-only mode with no edit or delete options. Admin receives HTTP 403 FORBIDDEN — no patient data accessible.
Post-condition(s)	Encrypted file record exists in MongoDB and cloud storage. RBAC enforcement confirmed for all three roles.
Actual Result(s)	Upload and encryption successful. Patient read-only access confirmed. Admin access correctly blocked with 403 response.
Test Case Result	Pass

Table 6.5: TC-5 — Assign Wellness Goal and Award Gamification Points

Field	Details
Identifier	TC-5
Related Requirement(s)	UC-4, SRS Section 3.1 — Goal Assignment & Progress Tracking
Short Description	Verify that a therapist can assign a wellness goal, a patient can mark it complete, and gamification points are awarded
Pre-condition(s)	Therapist and Patient are verified and linked. Both are logged in on separate sessions.
Input Data	Goal: "Practice 10 minutes of mindfulness daily" / Due Date: 7 days from today / Points: 50 / Patient: P001
Detailed Steps	1. Therapist navigates to Patient P001 profile and clicks Assign Goal. 2. Enters goal description, due date, and points value. Clicks Save. 3. Switch to Patient P001 session. 4. Open Wellness Goals and confirm new goal appears with status ACTIVE. 5. Click Mark as Complete. 6. Observe points counter update and notification.
Expected Result(s)	Goal created in MongoDB with status ACTIVE. Goal visible on Patient Dashboard with progress indicator. On completion, status updates to COMPLETED. Points counter increments by 50. Congratulatory notification sent to Patient. Progress update sent to Therapist.
Post-condition(s)	Goal status COMPLETED in database. Patient points balance updated. Both dashboards reflect the change.
Actual Result(s)	Goal assigned and visible to patient. Completion and points award functioning correctly. Notifications dispatched successfully.
Test Case Result	Pass

Table 6.6: TC-6 — Automated Notification Dispatch

Field	Details
Identifier	TC-6
Related Requirement(s)	UC-1, UC-4, SRS Section 3.1 — Notifications & Reminders
Short Description	Verify that automated email and SMS notifications are dispatched correctly for appointment confirmation and goal completion events
Pre-condition(s)	Twilio and SendGrid APIs operational. Confirmed appointment (TC-3) and completed goal (TC-5) exist. Notification Service running.
Input Data	Patient Email: sarakhan@test.com / Phone: 03001234567 / Therapist Email: dr.ahmed@benzi.ai / Events: Appointment Confirmation, Goal Completion
Detailed Steps	1. Complete appointment booking (TC-3) and observe email and SMS receipt. 2. Complete goal marking (TC-5) and observe notification. 3. Check SendGrid activity dashboard for delivery status. 4. Check Twilio logs for SMS delivery. 5. Verify n8n workflow shows reminder scheduled 24 hours before appointment.
Expected Result(s)	Confirmation email and SMS received by both Patient and Therapist within 60 seconds of booking. Goal completion notification received by Patient. n8n reminder workflow visible in automation logs with correct trigger time. All delivery statuses show DELIVERED in SendGrid and Twilio dashboards.
Post-condition(s)	All notification events logged in MongoDB audit trail. No failed delivery entries for the test events.
Actual Result(s)	Email notifications delivered within expected time. SMS delivered with ~45 second delay. n8n reminder job scheduled correctly.
Test Case Result	Pass

Table 6.7: TC-7 — Payment Failure Handling

Field	Details
Identifier	TC-7
Related Requirement(s)	UC-1, SRS Section 3.1 — Appointment Scheduling, Payment Processing
Short Description	Verify that a failed Stripe payment prevents appointment confirmation and the patient is notified appropriately without creating an orphaned record
Pre-condition(s)	Patient logged in. Therapist with available slots exists. Stripe test mode active.
Input Data	Therapist: Dr. Ahmed (T001) / Slot: Available / Payment: Stripe test decline card 4000 0000 0000 0002
Detailed Steps	1. Patient selects available slot for Dr. Ahmed. 2. Chooses Pay Online and enters Stripe decline test card. 3. Clicks Confirm Booking. 4. Observe on-screen response. 5. Check database for any new appointment record. 6. Check patient email for failure notification.
Expected Result(s)	Stripe returns payment failure event. No appointment record inserted in MongoDB. Selected slot remains available for other patients. On-screen error message displayed: "Payment unsuccessful. Please try a different payment method." Failure notification email sent via SendGrid. No SMS dispatched.
Post-condition(s)	Database state unchanged. Slot still available. No orphaned appointment record exists.
Actual Result(s)	Payment failure handled correctly. No record created. Slot remained available. Error message displayed as expected. Failure email received.
Test Case Result	Pass

Table 6.8: TC-8 — Admin Therapist Verification and Subscription Assignment

Field	Details
Identifier	TC-8

Related Requirement(s)	UC-6, SRS Section 3.1 — Admin Panel & Subscription Management
Short Description	Verify that admin can verify a pending therapist account and assign a subscription plan via Stripe
Pre-condition(s)	Admin account active. Therapist registration with status PENDING exists. Stripe test mode active.
Input Data	Admin: admin@benzi.ai / Therapist ID: T002 (PENDING) / Plan: Pro Monthly
Detailed Steps	1. Admin logs in and opens Admin Panel. 2. Views pending therapist list and selects T002. 3. Reviews credentials and clicks Approve. 4. Confirms action in dialog. 5. Assigns Pro Monthly plan to T002 via Subscription Management. 6. Verify T002 receives welcome email. 7. Log in as T002 and verify dashboard access.
Expected Result(s)	T002 status updated to VERIFIED in MongoDB with timestamp. Stripe subscription created with status active. Welcome email dispatched to T002. Therapist T002 can log in and access full Therapist Dashboard. Admin blocked from accessing any patient record throughout (HTTP 403 confirmed).
Post-condition(s)	T002 fully onboarded. Subscription record in Stripe referenced in MongoDB. Admin verification action recorded in system audit log.
Actual Result(s)	Verification and subscription assignment successful. Welcome email received by therapist. Dashboard access confirmed. Admin patient-data isolation verified.
Test Case Result	Pass

Table 6.9: TC-9 — AI Virtual Assistant Context-Aware Response

Field	Details
Identifier	TC-9
Related Requirement(s)	UC-5, SRS Section 3.1 — AI-Powered Virtual Assistant
Short Description	Verify that the AI Virtual Assistant returns a contextually relevant, guardrail-compliant response based on the patient's uploaded report and message, and correctly rejects out-of-scope clinical queries

Pre-condition(s)	Patient logged in. Therapist has uploaded a patient report (PDF) which has been processed through the RAG pipeline. AI module (Python/FastAPI) and LLM (Llama 3 / Mistral / Phi-3 via Ollama) are running. Guardrails configured with topic limits and crisis detection rules.
Input Data	Patient Message 1 (valid): "What relaxation techniques have been recommended for me?" / Patient Message 2 (out-of-scope): "Do I have anxiety disorder?" / Patient Report: Uploaded PDF containing therapist notes and treatment plan
Detailed Steps	1. Patient opens the AI Virtual Assistant chat. 2. Submits Message 1 asking about relaxation techniques. 3. Observe AI response and verify it references content from the uploaded patient report. 4. Submit Message 2 asking for a clinical diagnosis. 5. Observe AI response and verify guardrail behaviour. 6. Optionally save AI response to personal notes. 7. Check MongoDB for logged interaction entries.
Expected Result(s)	For Message 1: AI retrieves relevant chunks from the RAG-processed patient report, generates a contextually accurate supportive response, and returns it within the topic boundaries set by the clinician. Response does not include any clinical diagnosis. For Message 2: Guardrail layer detects out-of-scope clinical query and returns a polite redirect response (e.g., "I'm not able to provide a diagnosis — please speak with your therapist directly."). No LLM-generated diagnosis is returned. Interaction log for both messages stored in MongoDB with timestamp, patientId, message, and response fields.
Post-condition(s)	Both interaction records exist in MongoDB AI logs. No clinically diagnostic content present in any stored response. Patient notes updated if save action was triggered.
Actual Result(s)	Context-aware response for Message 1 correctly referenced report content. Guardrail for Message 2 triggered successfully — redirect response returned. Interaction logs persisted in database.
Test Case Result	Pass

6.2 Summary of Test Results

Table 6.10: Summary of Test Results

Module Name	Test Cases Run	Defects Found	Defects Corrected	Defects Remaining
User Authentication & Registration	TC-1, TC-2	1	1	0

Appointment Booking & Scheduling	TC-3	1	1	0
Therapist Dashboard & Patient Records	TC-4	1	0	1
Wellness Goals & Gamification	TC-5	0	0	0
Notifications (Email / SMS)	TC-6	1	1	0
Payment Processing (Stripe)	TC-7	0	0	0
Admin Panel & Subscription Management	TC-8	1	1	0
AI Virtual Assistant	TC-9	1	0	1
Complete System	**TC-1 to TC-9**	**6**	**4**	**2**

7. Revised Project Plan

This section presents the current progress of the BENZI.AI project against the original plan submitted in the project proposal. The Gantt chart (Figure 7.1) illustrates the planned and actual timelines for all project phases from September 2025 to June 2026, with the current date marker at April 2026.

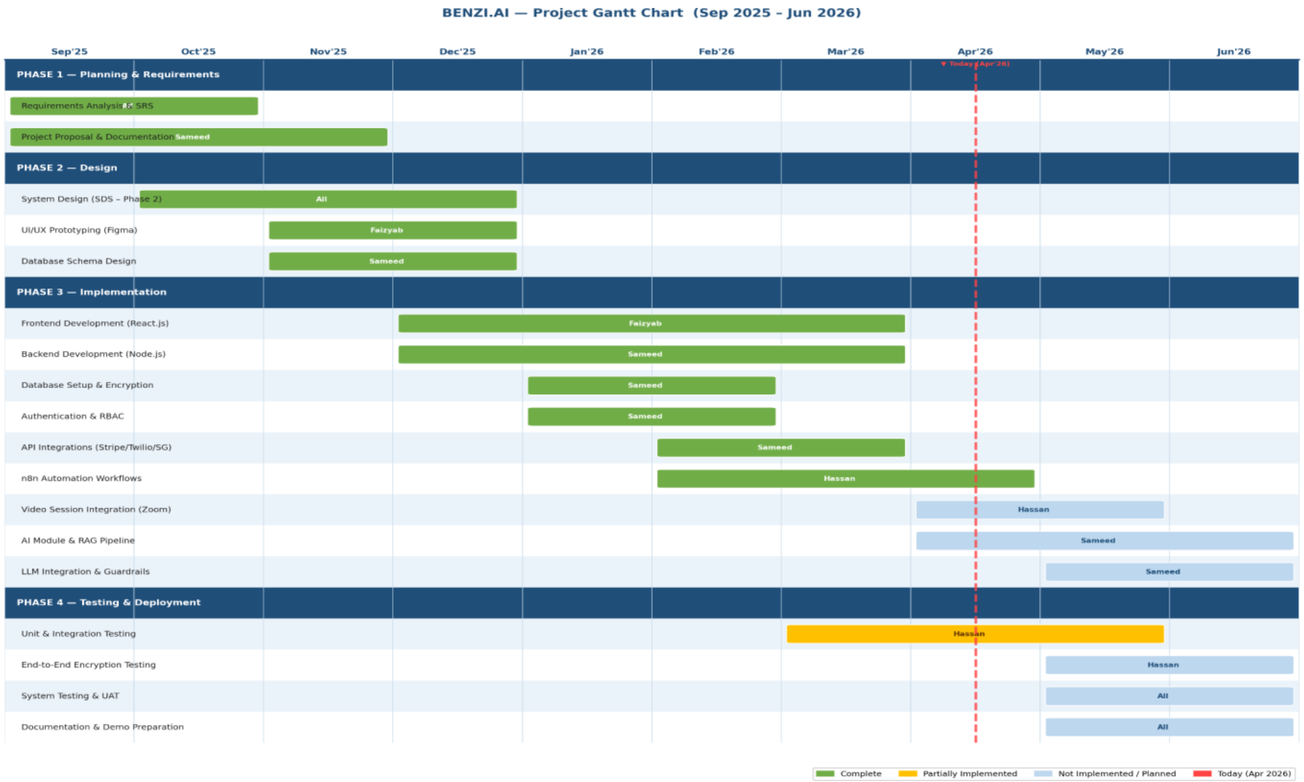


Figure 7.1: BENZI.AI Project Gantt Chart (Sep 2025 – Jun 2026)

The project has successfully completed all planning, design, and core implementation tasks on schedule. The frontend, backend, database, authentication, API integrations, and automation workflows are fully implemented. The remaining work — video session integration, AI module development, RAG pipeline, LLM integration with guardrails, and full system testing — is planned for completion by June 2026.

Table 6.2: Project Completion Status

Module Name	Status
Requirements Analysis & SRS Documentation	**Complete**
Project Proposal & Planning	**Complete**
System Design – SDS (Phase 2)	**Complete**
UI/UX Prototyping (Figma)	**Complete**
Database Schema Design	**Complete**

Frontend Development – Patient Portal (React.js)	**Complete**
Frontend Development – Therapist Dashboard (React.js)	**Complete**
Frontend Development – Admin Panel (React.js)	**Complete**
Backend Development – REST API (Node.js / Express.js)	**Complete**
Database Setup & Encryption (MongoDB + AES-256)	**Complete**
User Authentication & RBAC (JWT, 2FA)	**Complete**
API Integrations (Stripe, Twilio, SendGrid)	**Complete**
n8n Automation Workflows (Reminders, Notifications)	**Complete**
Unit & Integration Testing	**Partially Implemented**
Video Session Integration (Zoom / Google Meet)	**Not Implemented**
AI Virtual Assistant Module (Python / FastAPI)	**Not Implemented**
RAG Pipeline & Context Loader	**Not Implemented**
LLM Integration & Guardrails (Llama 3 / Mistral)	**Not Implemented**
End-to-End Encryption Testing	**Not Implemented**
System Testing & UAT	**Not Implemented**
Documentation & Demo Preparation	**Not Implemented**
Complete System	**Partially Implemented (~55%)**

8. References

[1] Health Insurance Portability and Accountability Act (HIPAA) Security Rule, U.S. Department of Health & Human Services, 45 C.F.R. Parts 160 and 164, 2003. Available: https://www.hhs.gov/hipaa/for-professionals/security/index.html

[2] General Data Protection Regulation (GDPR), Regulation (EU) 2016/679 of the European Parliament and of the Council, Official Journal of the European Union, 2016. Available: https://gdpr-info.eu/

[3] OWASP Application Security Verification Standard (ASVS) 4.0, Open Web Application Security Project, 2019. Available: https://owasp.org/www-project-application-security-verification-standard/

[4] National Institute of Standards and Technology (NIST), "Recommendation for Block Cipher Modes of Operation," NIST Special Publication 800-38A, AES-256 Standard, 2001. Available: https://csrc.nist.gov/publications/detail/sp/800-38a/final

[5] IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Computer Society, 1998.

[6] ISO/IEC/IEEE 29148:2018, "Systems and Software Engineering — Life Cycle Processes — Requirements Engineering," International Organization for Standardization, 2018. Available: https://www.iso.org/standard/72089.html

[7] MongoDB, Inc., "MongoDB Manual — Field Level Encryption," MongoDB Documentation, 2024. Available: https://www.mongodb.com/docs/manual/core/field-level-encryption/

[8] Zoom Video Communications, Inc., "Zoom End-to-End Encryption Whitepaper," 2020. Available: https://github.com/zoom/zoom-e2e-whitepaper

[9] n8n GmbH, "n8n Documentation — Workflow Automation Platform," 2024. Available: https://docs.n8n.io/

[10] Stripe, Inc., "Stripe API Reference," Stripe Developer Documentation, 2024. Available: https://stripe.com/docs/api

[11] Twilio Inc., "Twilio REST API Documentation — SMS," 2024. Available: https://www.twilio.com/docs/sms/api

[12] Twilio SendGrid, "SendGrid Email API Documentation," 2024. Available: https://docs.sendgrid.com/api-reference/

[13] Google LLC, "Google Meet REST API Reference," Google Workspace Developer Documentation, 2024. Available: https://developers.google.com/meet/api/reference/rest

[14] Meta AI, "Llama 3 Model Card," Meta AI Research, 2024. Available: https://ai.meta.com/blog/meta-llama-3/

[15] Mistral AI, "Mistral 7B Technical Report," arXiv:2310.06825, 2023. Available: https://arxiv.org/abs/2310.06825

[16] Microsoft Research, "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," arXiv:2404.14219, 2024. Available: https://arxiv.org/abs/2404.14219

[17] Ollama, "Ollama — Run Large Language Models Locally," 2024. Available: https://ollama.com/

[18] Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, pp. 9459–9474, 2020. Available: https://arxiv.org/abs/2005.11401

[19] Facebook AI Research, "React.js — A JavaScript Library for Building User Interfaces," Meta Open Source, 2024. Available: https://react.dev/

[20] OpenJS Foundation, "Node.js Documentation," 2024. Available: https://nodejs.org/en/docs/

[21] Pallets Projects, "Flask Documentation (3.x)," 2024. Available: https://flask.palletsprojects.com/

[22] Sebastián Ramírez, "FastAPI Documentation," 2024. Available: https://fastapi.tiangolo.com/

[23] Amazon Web Services, "AWS HIPAA Eligible Services Reference," AWS Compliance Documentation, 2024. Available: https://aws.amazon.com/compliance/hipaa-eligible-services-reference/

[24] Microsoft Azure, "Azure Healthcare APIs Documentation," Microsoft, 2024. Available: https://learn.microsoft.com/en-us/azure/healthcare-apis/

[25] JSON Web Token (JWT), "RFC 7519 — JSON Web Token," Internet Engineering Task Force (IETF), M. Jones, J. Bradley, N. Sakimura, 2015. Available: https://datatracker.ietf.org/doc/html/rfc7519

[26] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. Available: https://www.w3.org/TR/WCAG21/

[27] HL7 International, "Fast Healthcare Interoperability Resources (FHIR) R4," 2019. Available: https://hl7.org/fhir/R4/

Appendix A: Glossary

Term	Definition
2FA(Two-Factor Authentication)	A security mechanism requiring users to provide two separate verification factors (e.g., password and a one-time OTP code) before access is granted. Used across all BENZI.AI user roles to prevent unauthorised login.
AES-256(Advanced Encryption Standard – 256-bit)	A symmetric encryption algorithm used to encrypt all sensitive patient data, session notes, medical records, and uploaded files stored in the BENZI.AI database and cloud storage.
AI Engine	The Python-based (Flask/FastAPI) artificial intelligence processing layer of BENZI.AI, comprising three sub-modules: the AI Virtual Assistant, the Recommendation Engine, and the Goal Tracking Module.
AI Virtual Assistant (BENZI)	The context-aware conversational chatbot integrated into the Patient Portal. It analyses patient medical history, therapist notes, and wellness goals to deliver personalised, guardrail-compliant mental health support and recommendations.
Anonymous Mode	A privacy feature allowing patients to interact with therapists or the AI assistant without revealing personally identifying information, reducing stigma associated with mental health care.
API (Application Programming Interface)	A defined set of rules and endpoints through which software components communicate. BENZI.AI uses RESTful APIs over HTTPS for all inter-layer communication.
AppointmentService	A backend microservice responsible for checking therapist availability, preventing double-booking, managing slot confirmations, and invoking payment and notification workflows during the appointment lifecycle.

AWS KMS (Amazon Web Services Key Management Service)	A cloud-based encryption key management service used to securely generate, store, and control access to the AES-256 encryption keys protecting BENZI.AI's healthcare data.
Backend	The server-side application layer of BENZI.AI, implemented in Node.js and Express.js, that handles all business logic, API routing, database interaction, and integration with external third-party services.
BENZI.AI	The branded name for the Intelligent Secure Meditation Counsellor–Patient Care System developed by Group F25CS186. A cloud-hosted, browser-based web platform connecting patients, therapists, and administrators in a secure mental health care ecosystem.
Cloud Storage	The encrypted object storage service (AWS S3 or Azure Blob Storage) used to store uploaded files such as session reports, prescriptions, and clinical PDFs after AES-256 encryption.
COTS (Commercial Off-The-Shelf)	Pre-built software solutions available for purchase or licensing. BENZI.AI was built as a fully custom system because no COTS product adequately addressed its combined requirements of healthcare privacy, AI personalisation, and role-based multi-portal access.
DFD (Data Flow Diagram)	A diagramming technique showing how data moves through a system. The Phase 2 SDS submission included a DFD that received feedback indicating it required further decomposition.
E2EE (End-to-End Encryption)	An encryption approach in which data is encrypted on the sender's device and only decrypted by the intended recipient, with no intermediate party able to access plaintext. Used for all video therapy sessions via Zoom/Google Meet.
Express.js	A lightweight web application framework for Node.js used to build the BENZI.AI backend RESTful API server.

Factory Pattern	A software design pattern used in BENZI.AI to dynamically instantiate the correct user context object (Patient, Therapist, or Admin) and AI response handler at runtime based on the authenticated session role.
FastAPI	A high-performance Python web framework used as an alternative to Flask for implementing the BENZI.AI AI Engine's REST API endpoints.
FLE (Field-Level Encryption)	A MongoDB feature that encrypts individual fields within a document rather than the entire database, providing granular protection for sensitive healthcare data such as diagnoses, allergies, and treatment notes.
Flask	A lightweight Python web microframework used to build the BENZI.AI AI Engine service, exposing REST endpoints consumed by the Node.js backend.
Frontend	The browser-based presentation layer of BENZI.AI, implemented in React.js, providing three distinct role-specific portals: the Patient Portal, the Therapist Dashboard, and the Admin Panel.
Gamification	The application of game-design elements (points, progress bars, milestone notifications) to non-game contexts. In BENZI.AI, a points-based system rewards patients for completing therapist-assigned wellness goals to encourage therapy adherence.
GDPR (General Data Protection Regulation)	A European Union data protection law (Regulation 2016/679) establishing user rights and organisational obligations regarding personal data. BENZI.AI is designed to comply with GDPR standards for handling patient health information.
GCP (Google Cloud Platform)	One of three major cloud infrastructure options (alongside AWS and Microsoft Azure) on which BENZI.AI can be deployed for scalable, highly available hosting.

Goal Tracking Module	An AI Engine sub-module managing the lifecycle of therapist-assigned wellness goals, including goal creation, patient progress tracking, gamification point allocation, and overdue deadline reminders via n8n cron jobs.
HIPAA (Health Insurance Portability and Accountability Act)	A United States federal law establishing national standards for the protection of sensitive patient health information. BENZI.AI is architected to meet HIPAA compliance requirements for encryption, access control, and audit logging.
HTTPS (Hypertext Transfer Protocol Secure)	The secure version of HTTP, using TLS encryption to protect data in transit between client browsers and the BENZI.AI server. All platform communication is restricted to HTTPS; plain HTTP is not permitted.
JWT (JSON Web Token)	A compact, digitally signed token used to securely transmit authentication and authorisation information between the BENZI.AI client and backend. Tokens expire after 3600 seconds to enforce session security.
Layered Architecture Pattern	The overarching software design pattern of BENZI.AI, organising the system into four distinct, non-bypassing layers: Presentation (React.js), Application (Node.js/Express.js), AI Engine (Python), and Data (MongoDB).
LLM (Large Language Model)	A machine learning model trained on large text corpora, capable of generating contextually coherent natural language responses. BENZI.AI's AI Virtual Assistant uses LLM inference (e.g., Llama 3, Mistral, or Phi-3 via Ollama) to generate patient-facing guidance.
Medical Record	A structured set of patient health data including diagnoses, allergies, therapy session notes, treatment plans, and uploaded clinical files. Records are stored in encrypted form and subject to strict RBAC access controls.

MongoDB	The NoSQL document-oriented database used as the primary data store for BENZI.AI. Selected for its schema flexibility, native field-level encryption support, and suitability for semi-structured healthcare data.
MVC (Model-View-Controller)	A software architectural pattern applied to the BENZI.AI backend. MongoDB schemas serve as Models, Express.js controllers encapsulate business logic, and React.js components constitute the View layer.
n8n	A self-hosted workflow automation platform used by BENZI.AI to orchestrate background tasks such as automated appointment reminders, post-session notifications, AI analytics execution, and retry logic for failed operations.
Node.js	An asynchronous, event-driven JavaScript runtime used to build the BENZI.AI backend application server, selected for its suitability for real-time WebSocket-based therapy sessions and concurrent API request handling.
NotificationService	A backend service responsible for dispatching all automated communications in BENZI.AI, including appointment confirmations, session reminders, goal completion alerts, and subscription receipts via Twilio (SMS) and SendGrid (email).
Observer Pattern	A software design pattern used in BENZI.AI's notification subsystem, where internal events (such as appointment confirmation or goal completion) automatically trigger notification dispatch without tight coupling between business logic and messaging infrastructure.
Ollama	A tool enabling local deployment and inference of large language models (e.g., Llama 3, Mistral, Phi-3) on-premise, used by the BENZI.AI AI Engine to run LLM inference within a controlled, HIPAA-compliant environment.

Patient Portal	The React.js-based browser interface for authenticated patients, providing access to mood tracking, wellness goals, AI chat (BENZI), progress analytics, appointment booking, session reports, and profile management.
RAG (Retrieval-Augmented Generation)	An AI technique combining information retrieval from a knowledge base (such as patient reports) with LLM generation, enabling the BENZI.AI virtual assistant to produce personalised responses grounded in a specific patient's uploaded clinical documents.
RBAC (Role-Based Access Control)	An access control model in which system permissions are assigned based on a user's role. BENZI.AI enforces three roles — Patient, Therapist, and Admin — each with strictly defined permissions. The Admin role is architecturally blocked from accessing any patient health records (HTTP 403).
React.js	A JavaScript library for building component-based user interfaces, used to implement all three BENZI.AI portals: the Patient Portal, Therapist Dashboard, and Admin Panel.
Recommendation Engine	An AI Engine sub-module that analyses longitudinal patient data and session history to generate behavioural trend insights and treatment progress reports for therapist-facing dashboards.
RecordsService	A backend service managing the secure creation, retrieval, encryption, and storage of patient medical records, session notes, and uploaded clinical files, with strict RBAC enforcement at the middleware level.
REST / RESTful API	Representational State Transfer; an architectural style for designing networked APIs using standard HTTP methods (GET, POST, PATCH, DELETE). All BENZI.AI component communication is implemented as RESTful APIs over HTTPS.

SafetyFilter	A module within the BENZI.AI AI Engine that intercepts AI-generated responses before delivery, detecting and blocking out-of-scope or harmful content such as clinical diagnoses, medication prescriptions, or crisis intervention attempts.
SDS (Software Design Specification)	A Phase 2 document detailing the system's architecture, data models, use-case diagrams, and component-level design decisions for BENZI.AI. Received feedback noting the USD design was not as per the standard diagram and the DFD was not well decomposed.
SendGrid	A cloud-based email delivery service used by the BENZI.AI NotificationService to dispatch HTML-formatted notification emails, including appointment confirmations, session summaries, and subscription receipts.
Singleton Pattern	A software design pattern ensuring that system-wide resources such as the MongoDB connection pool, configuration managers, and logging services are instantiated only once and shared across all modules, preventing resource duplication.
SOA (Service-Oriented Architecture)	An architectural approach in which core system functions are implemented as independent, loosely coupled services. BENZI.AI applies SOA by isolating Authentication, Appointment, Records, AI, and Notification functions as separate services.
SRS (Software Requirements Specification)	A structured document, following IEEE 830-1998 guidelines, that formally defines the functional and non-functional requirements of a software system. The Phase 1 SRS is the foundational document for the BENZI.AI project.
Stripe	A PCI-DSS-compliant payment processing platform used by BENZI.AI to handle online appointment payments and therapist subscription billing. Raw card data is never stored by BENZI.AI.

Therapist Dashboard	The React.js-based browser interface for authenticated and verified therapists, providing access to patient registration, session management, medical records, appointment scheduling, analytics, service listings, and revenue tracking.
TLS (Transport Layer Security)	A cryptographic protocol that encrypts data in transit between clients and the BENZI.AI server. The platform enforces TLS 1.2 or higher and prohibits all unencrypted HTTP communication.
Twilio	A cloud communications platform used by the BENZI.AI NotificationService to dispatch SMS notifications for appointment confirmations, reminders, goal deadline alerts, and system messages, with up to three retry attempts on failure.
UML (Unified Modelling Language)	A standardised visual modelling language for software design. BENZI.AI design documentation uses UML sequence diagrams, activity diagrams, use-case diagrams, and class-level descriptions throughout Phases 2 and 3.
WebSocket	A full-duplex communication protocol enabling real-time, bidirectional data exchange between the BENZI.AI client and backend server. Used for live therapy session chat and video session coordination, with a three-retry auto-reconnect protocol on connection drop.
WCAG (Web Content Accessibility Guidelines)	W3C guidelines defining accessibility standards for web content. BENZI.AI targets WCAG AA compliance, including minimum contrast ratios, keyboard navigability, and readable font sizes across all portals.
Zoom SDK	A software development kit from Zoom Video Communications used by BENZI.AI to dynamically create end-to-end encrypted video meeting rooms for therapy sessions directly from the Appointment Service. Meeting URLs are never stored in MongoDB after session completion.

Appendix B: IV & V Report

(Independent verification & validation)
IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
...					

Table 1: List of non-trivial defects

This document has been adapted from the following:

- Previous project templates at UCP
- High-level Technical Design, Centers for Medicare & Medicaid Services. (www.cms.gov)